

Principles and Practice of Scalable and Distributed AI Training and Inference

Tutorial at Hoti '26

by

Dhabaleswar K. (DK) Panda

The Ohio State University

panda@cse.ohio-state.edu

<http://www.cse.ohio-state.edu/~panda>

Nawras Alnaasan

The Ohio State University

alnaasan.1@osu.edu

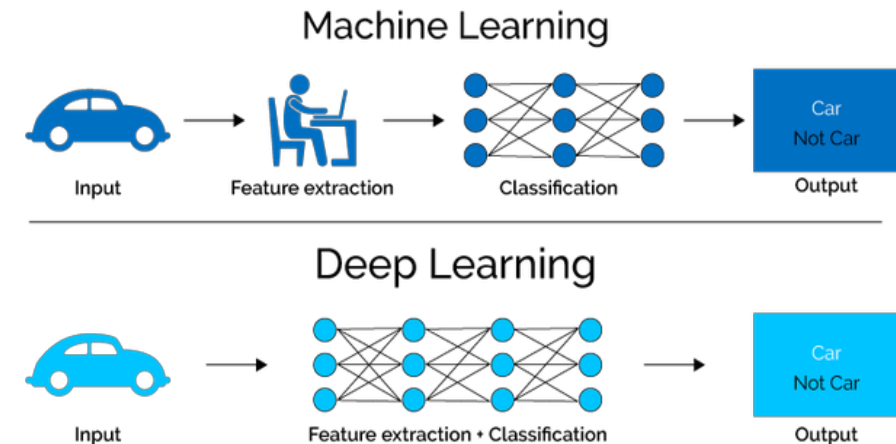
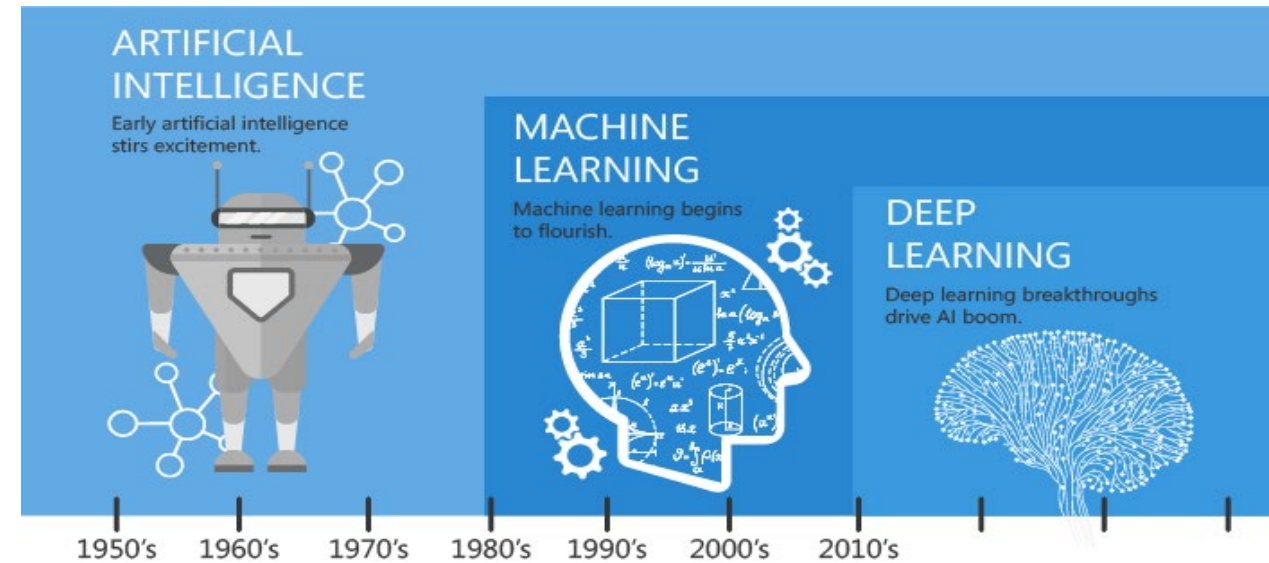
<https://engineering.osu.edu/people/alnaasan.1>

Outline

- **Introduction: The AI–HPC Convergence**
 - **Definitions: AI, Machine Learning, Deep Learning, and Generative AI**
 - **Overview: System-Aware AI Lifecycle and HPC Stack**
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

What is Machine Learning and Deep Learning?

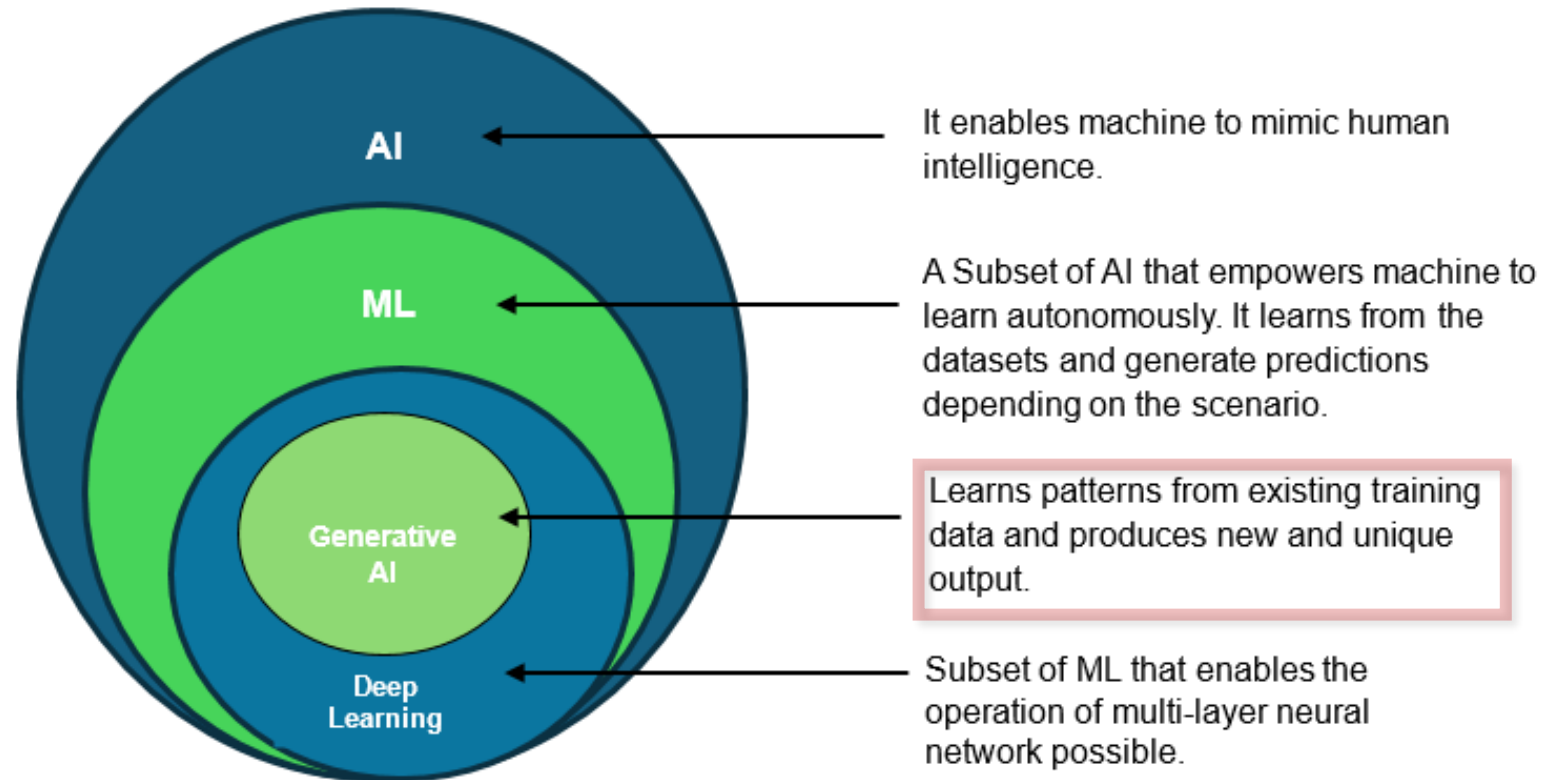
- Machine Learning (ML)
 - “the study of computer algorithms to improve automatically through experience and use of data”
- Deep Learning (DL) – a subset of ML
 - Uses Deep Neural Networks (DNNs)
 - **Perhaps, the most revolutionary subset!**
- Based on learning data representation
- DNN Examples: Convolutional Neural Networks, Recurrent Neural Networks, Hybrid Networks
- Data Scientist or Developer Perspective for using DNNs
 1. Identify DL as solution to a problem
 2. Determine Data Set
 3. Select Deep Learning Algorithm to Use
 4. Use a large data set to train an algorithm



Courtesy: <https://hackernoon.com/difference-between-artificial-intelligence-machine-learning-and-deep-learning-1pcv3zeg>, <https://blog.dataiku.com/ai-vs.-machine-learning-vs.-deep-learning>, https://en.wikipedia.org/wiki/Machine_learning

What is Generative AI?

- Generative AI is a subset of Deep Learning which creates new content like text, images, videos, or audio based on the data it was trained on.
- Examples:
 - Text: GPT, LLaMA, and DeepSeek.
 - Images: DALL-E and Stable Diffusion.
 - Videos: Runway and Sora.
 - Audio: AudioPaLM and VALL-E.
- What is not Generative AI?
 - Discriminative models that perform:
 - Classification
 - Regression
 - Object detection
 - Clustering
 - etc.

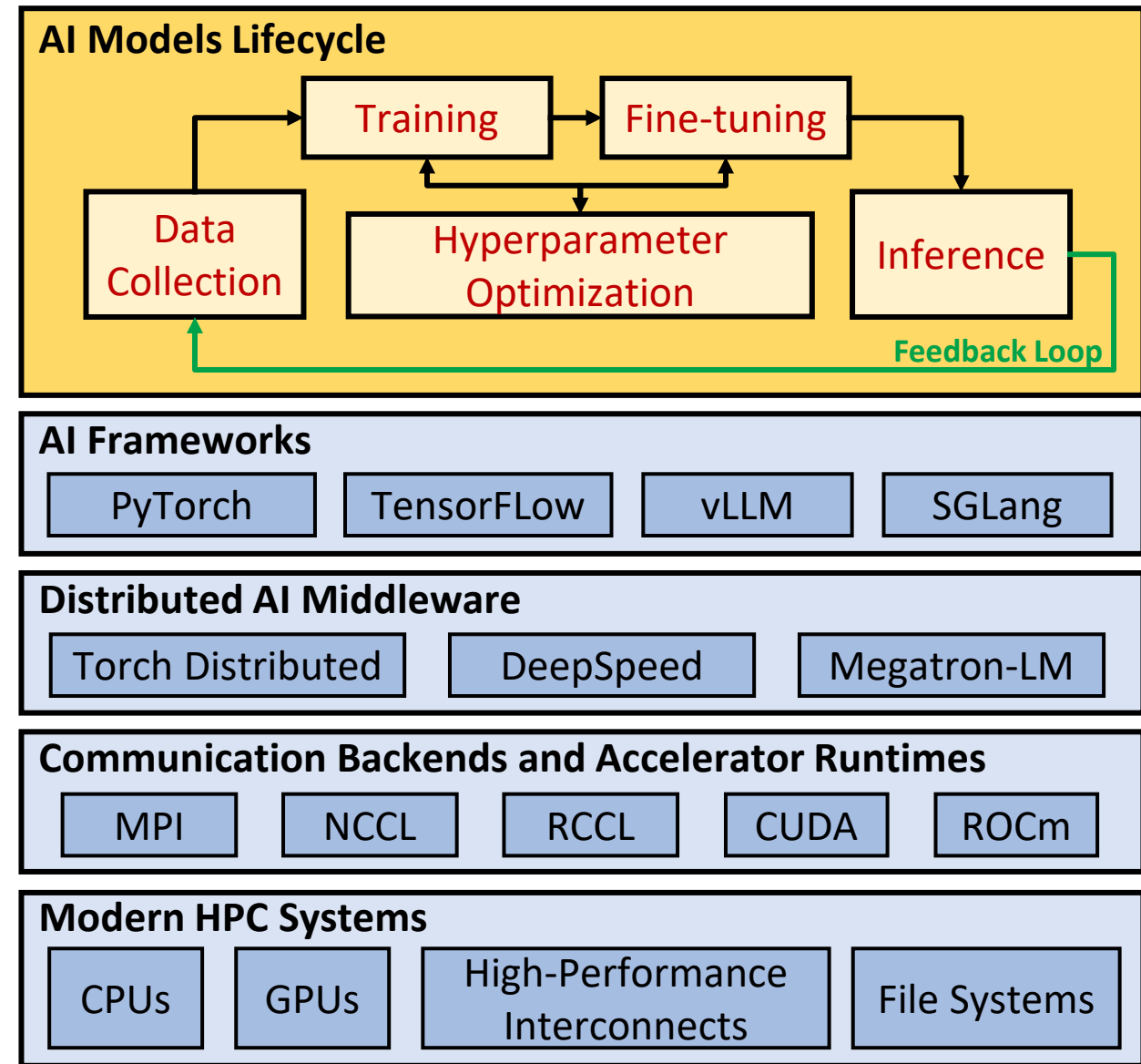


Courtesy: <https://www.tutorialspoint.com/gen-ai/ml-and-generative-ai.htm>

Overview: The Lifecycle of AI Models

AI model lifecycle is an iterative process involving:

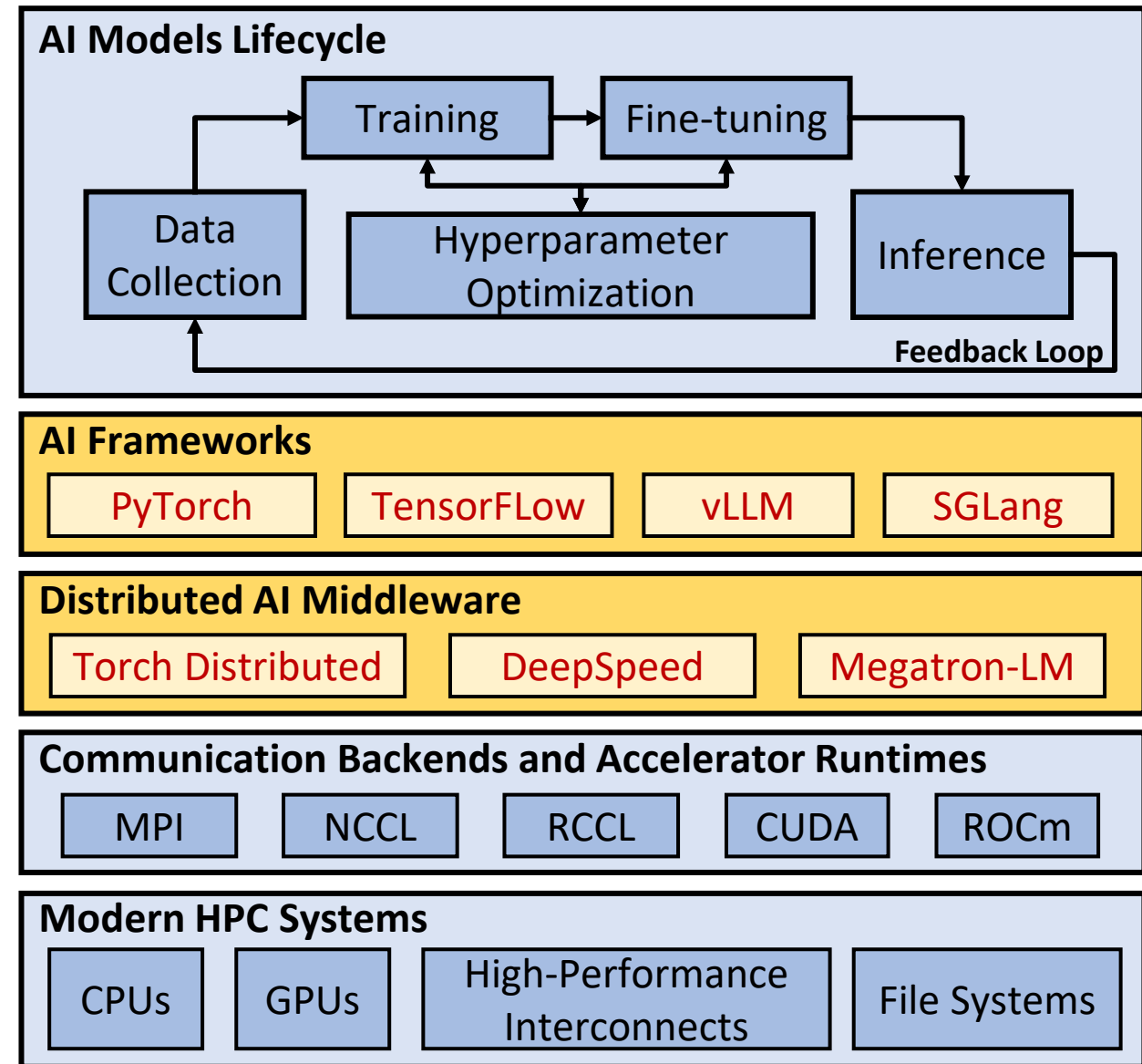
- **Data Collection:** Acquisition, cleaning, and preprocessing of large-scale datasets.
- **Training:** Learning model parameters using collected data to identify patterns and make predictions.
- **Fine-Tuning:** Adapting pretrained models to downstream tasks using a specialized datasets.
- **Hyperparameter Optimization (HPO):** Automated search over training configuration space to maximize model performance.
- **Inference:** Deploying trained models to serve predictions on real-world data.
- **Feedback Loop:** Feeding real-world inference data back into the pipeline to update datasets, retrain or fine-tune models.



System-Aware AI Lifecycle and HPC Stack

Overview: AI Frameworks and Distributed Middleware

- **Main objectives of AI frameworks:**
 - Hide complex mathematics
 - Allow users to focus on Developing AI models
- **Support for Parallelism:**
 - We have saturated the peak potential of current-generation architectures
 - A single GPU or a many-core CPU is not enough!
 - Distributed AI Middleware implements various parallelism strategies to support large scale model training.



System-Aware AI Lifecycle and HPC Stack

Overview: Communication Backends and Accelerator Runtimes

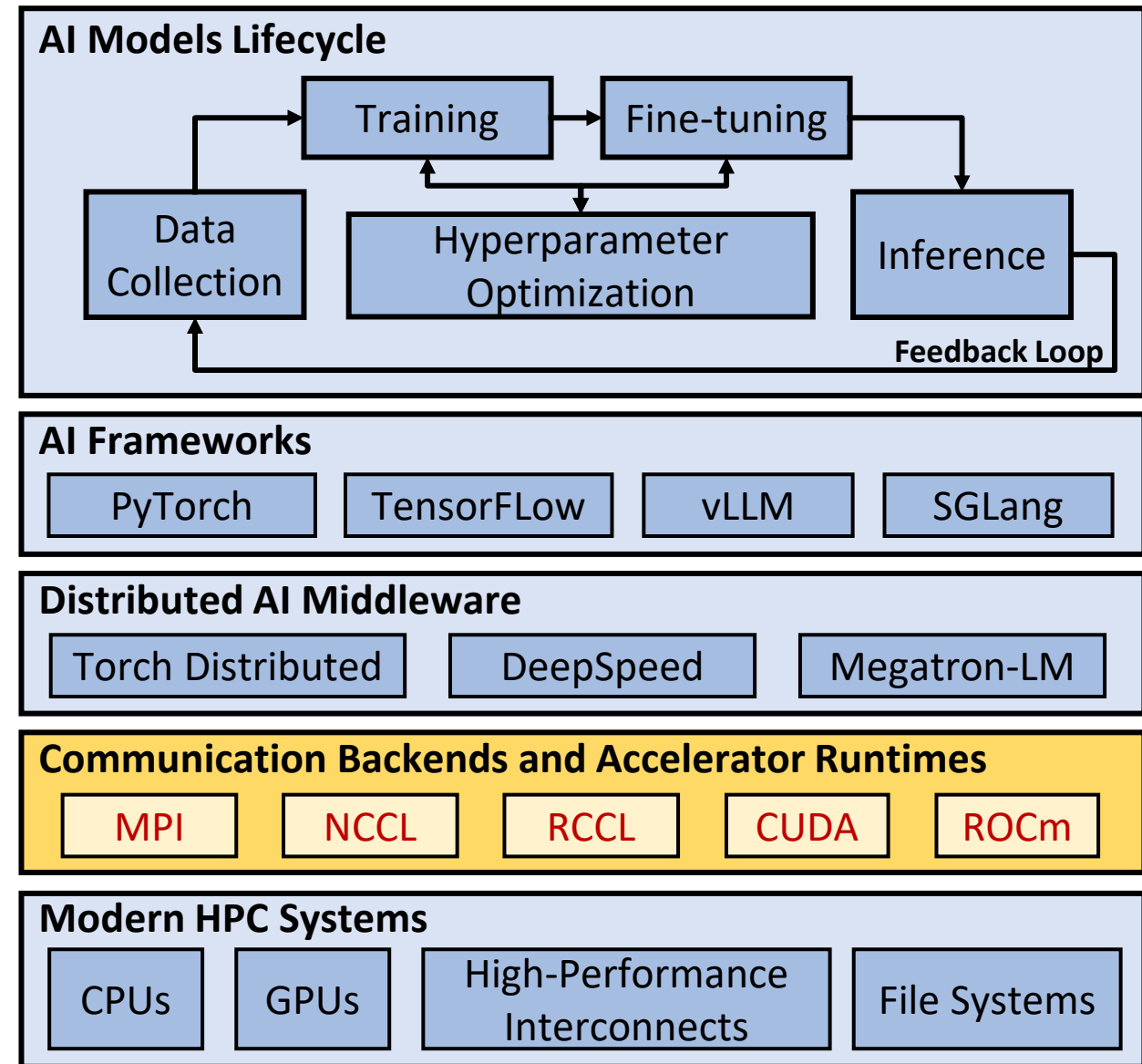
This layer provides the low-level communication primitives and control of accelerator hardware.

- **Communication Backends (MPI, NCCL, RCCL):**

High-performance communication libraries implement collective operations (e.g., AllReduce, Broadcast) and point-to-point messaging, optimized for HPC interconnects and multi-GPU scaling.

- **Accelerator Runtimes (CUDA, ROCm):**

Programming models and execution runtimes provide direct access to GPU hardware, including kernel execution, memory management, and device-level parallelism.

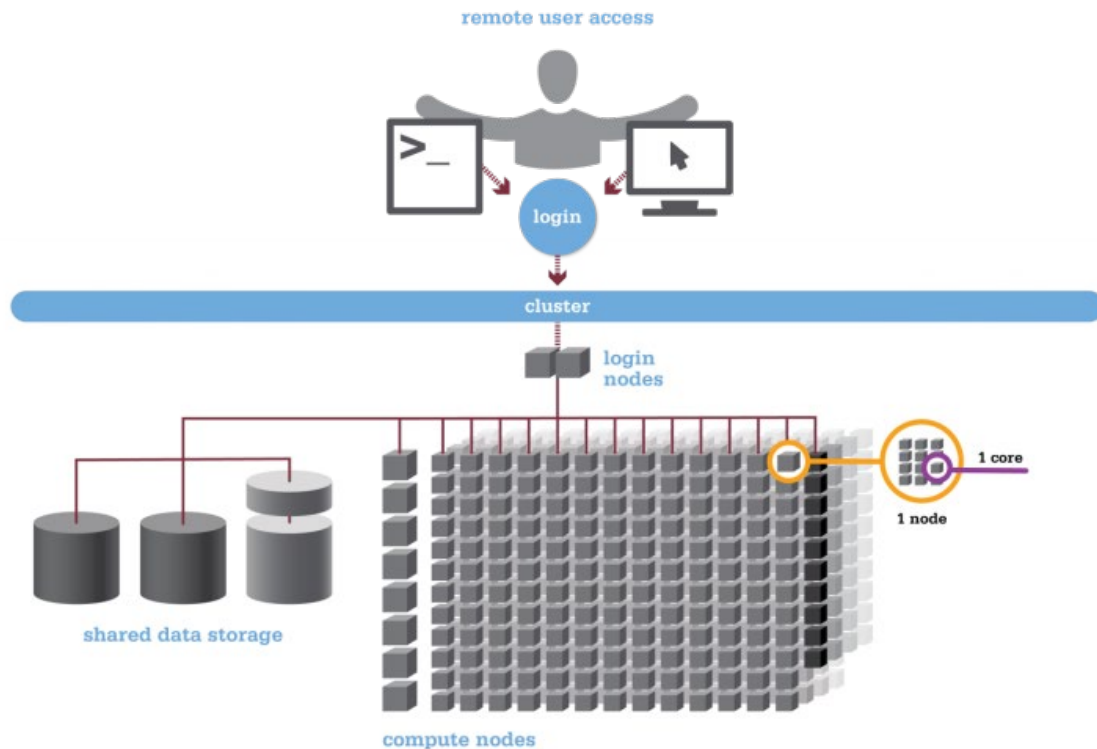


System-Aware AI Lifecycle and HPC Stack

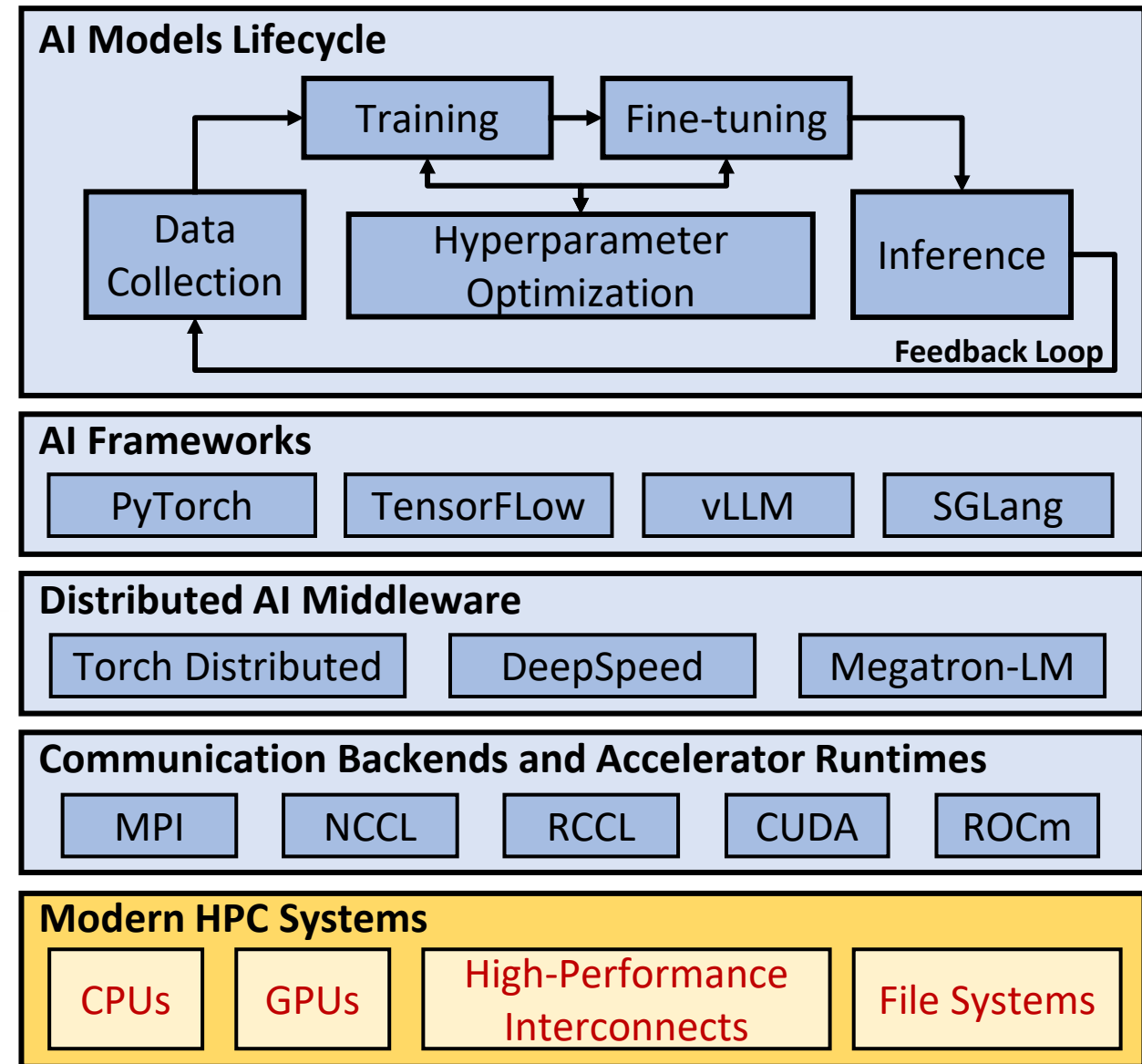
Overview: DL and High-Performance Architectures

HPC infrastructure consists of:

- Compute and Memory Architecture (CPUs, GPUs, HBM, DRAM)
- Storage and Interconnect (NVMe, Parallel File Systems, NVLink, InfiniBand)



Courtesy: https://www.osc.edu/resources/getting_started/hpc_basics

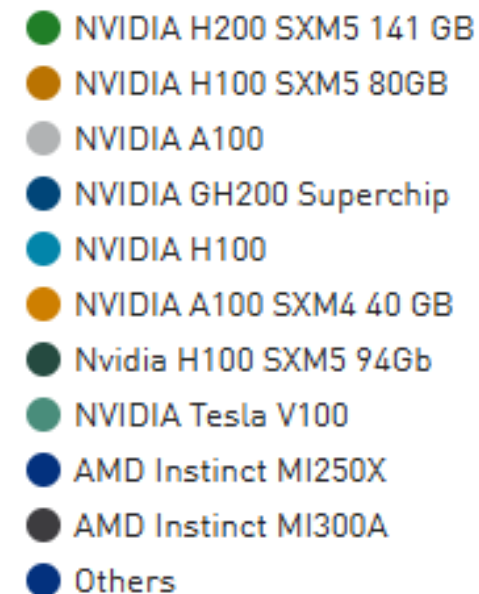
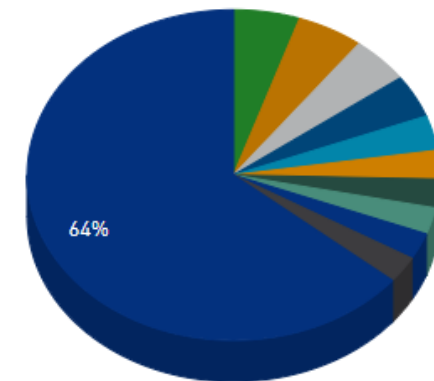


System-Aware AI Lifecycle and HPC Stack

DL and High-Performance Architectures

- NVIDIA GPUs are the main driving force for faster training of DL models
- However, High Performance Architectures for DL and HPC are evolving
 - 272/500 Top HPC systems are using accelerator/co-processor (June '26)
 - DGX-1 (Pascal), DGX-2 (Volta), DGX A100, DGX H100, HGX A100, HGX H100
 - Dedicated DL supercomputers
 - NVIDIA Eos – An Exaflop AI Supercomputer using DGX H100 (Announced)
 - AMD Instinct MI300A GPUs power El Capitan – the #2 Top500.
 - AMD EPYC (Rome/Milan) CPUs have 64 cores/socket (Frontier – #3 on Top500)
 - Sapphire Rapids Xeon CPUs have 52 cores/socket (Aurora – #4 on Top500)
 - Domain Specific Accelerators for DNNs are also emerging

Accelerator/Co-Processor System Share



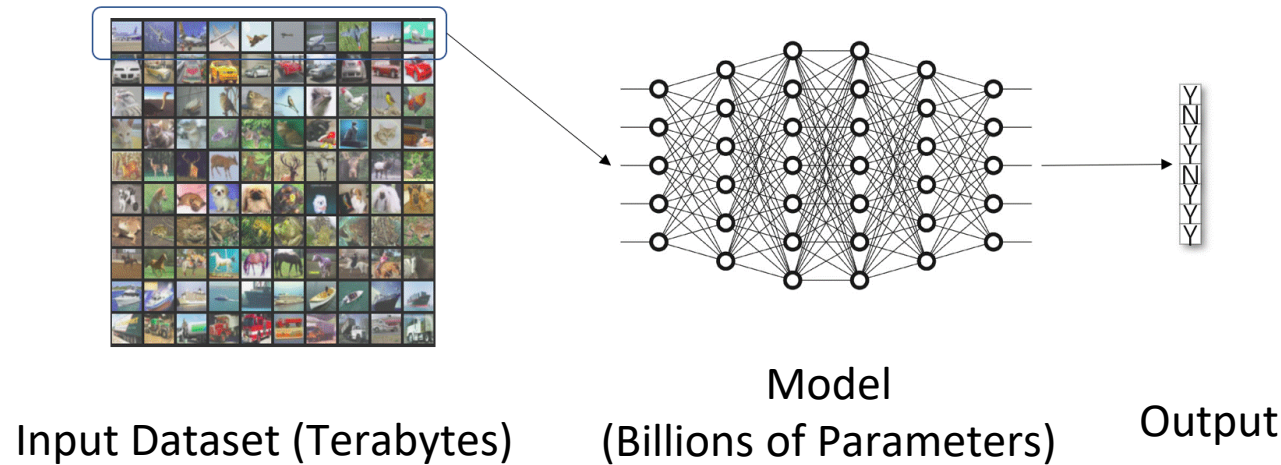
**Accelerator/CP
Performance Share**
www.top500.org

Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- **Challenges in Exploiting HPC Technologies for AI**
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

System Challenges: Training

Training involves **iterative** forward and backward passes over **large datasets** to optimize model parameters. At scale, training is dominated by **compute, memory, and communication overheads**.



Example: Training a 100B+ parameter model on a multi-terabyte dataset across a GPU cluster.

Limitations:

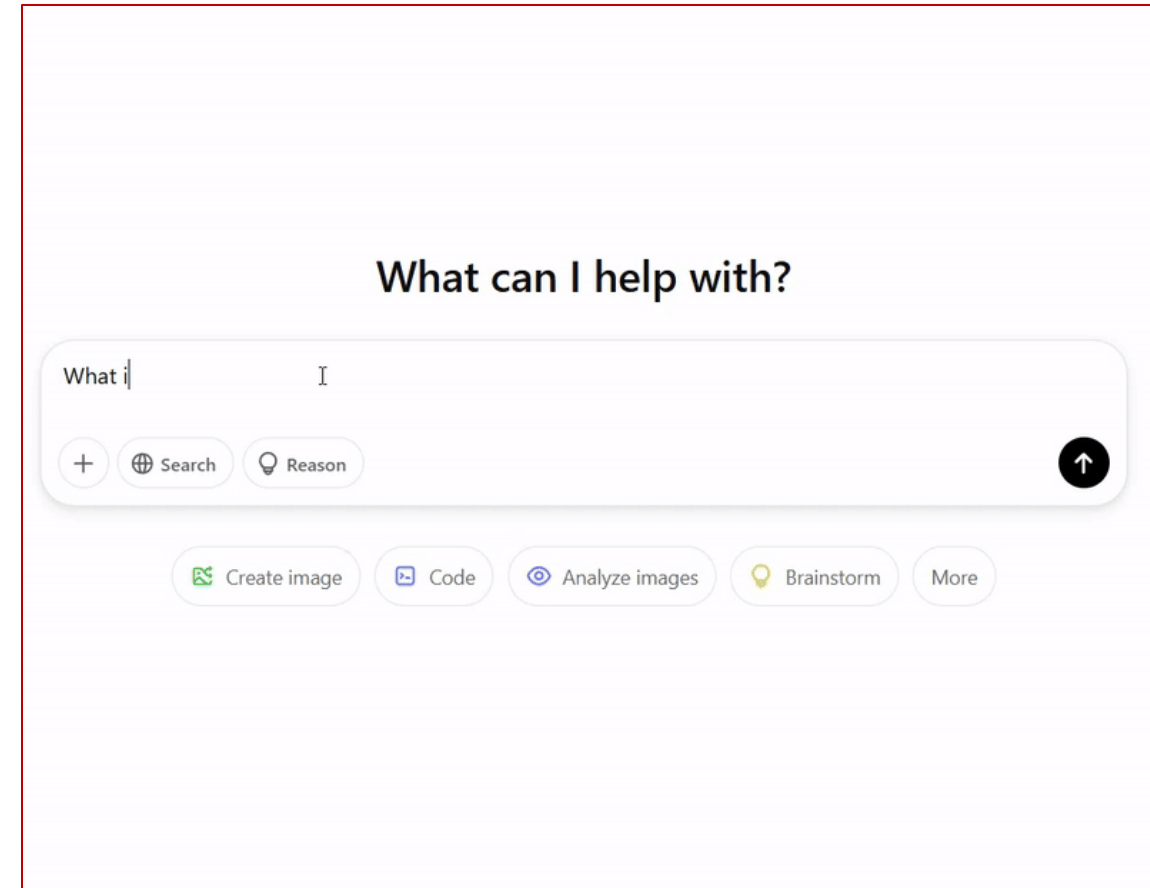
- Doesn't fit on a single node → requires splitting the model/data across devices
- Frequent communication operation → requires efficient utilization of interconnect
- Data pipeline pressure → storage and I/O must keep GPUs fed or utilization drops

System Challenges: Inference

In inference, the model generates outputs based on input prompts. For autoregressive models (most LLMs), inference follows an **iterative loop**, where each generated token (word) is **fed back** as input for the next step until completion.

LLM inference requires low-latency, high-throughput compute with the following key QoS (Quality of Service) requirements:

- **Low Latency** – Ensures fast response times, crucial for interactive applications.
- **Efficient Batch Processing** – Optimized for serving multiple queries in parallel to maximize throughput.
- **Mixed-Precision Support (FP16/BF16/INT8)** – Reduces compute overhead while maintaining accuracy.
- **High-Speed Interconnects (NVLink, InfiniBand)** – Required for multi-GPU inference to minimize communication bottlenecks.
- **High Memory Bandwidth** – To efficiently load large model weights and handle activation memory.



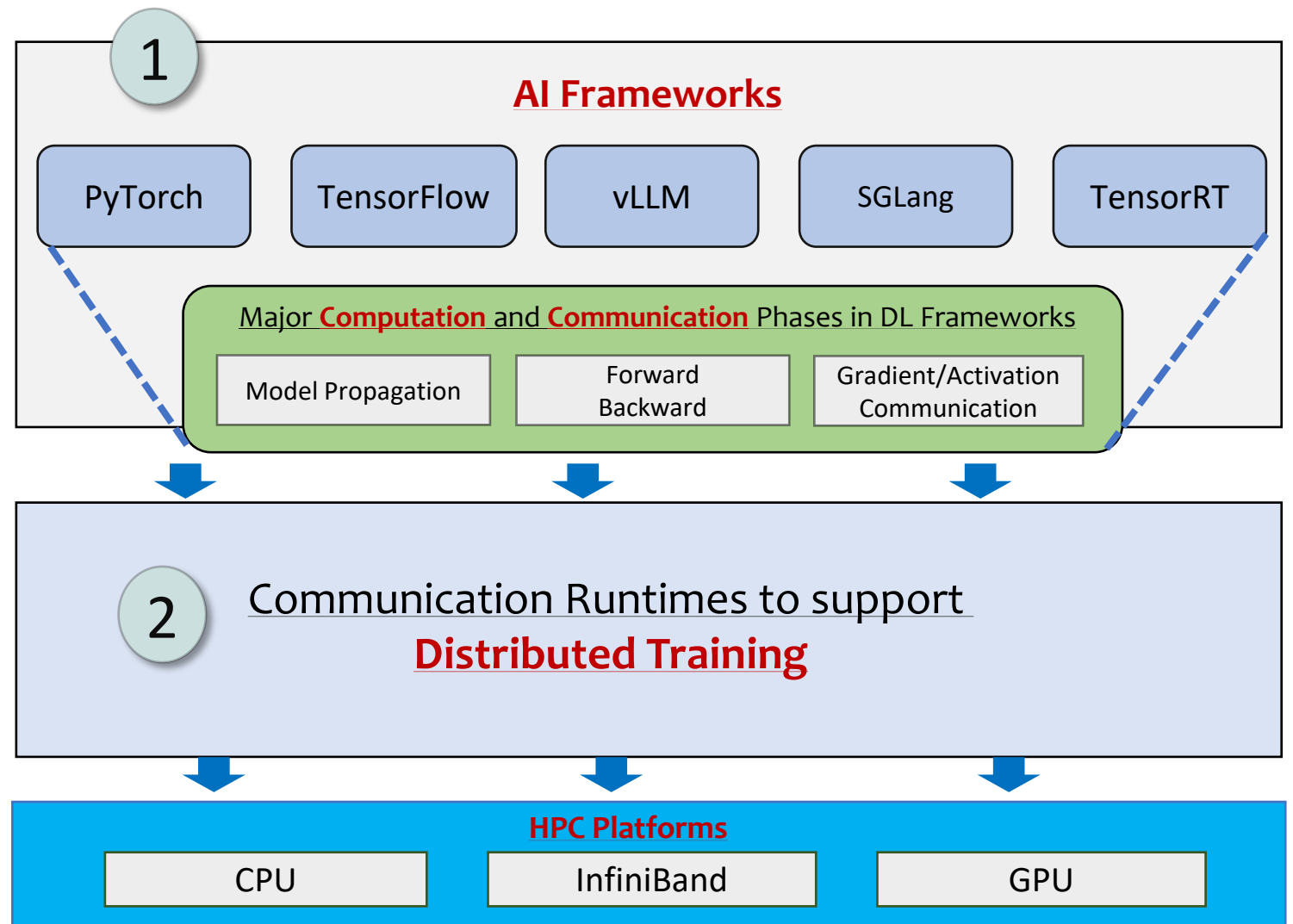
Online LLM Inference

Broad Challenge: Exploiting HPC for AI

*How to efficiently scale-out
AI frameworks and take
advantage of heterogeneous
High Performance Computing (HPC)
resources?*

Research Challenges to Exploit HPC Technologies

1. What are the fundamental issues in designing **DL frameworks**?
 - Memory Requirements
 - **Computation** Requirements
 - **Communication** Overhead
2. Why do we need to support **distributed training**?
 - To overcome the limits of single-node training
 - To better utilize hundreds of existing HPC Clusters



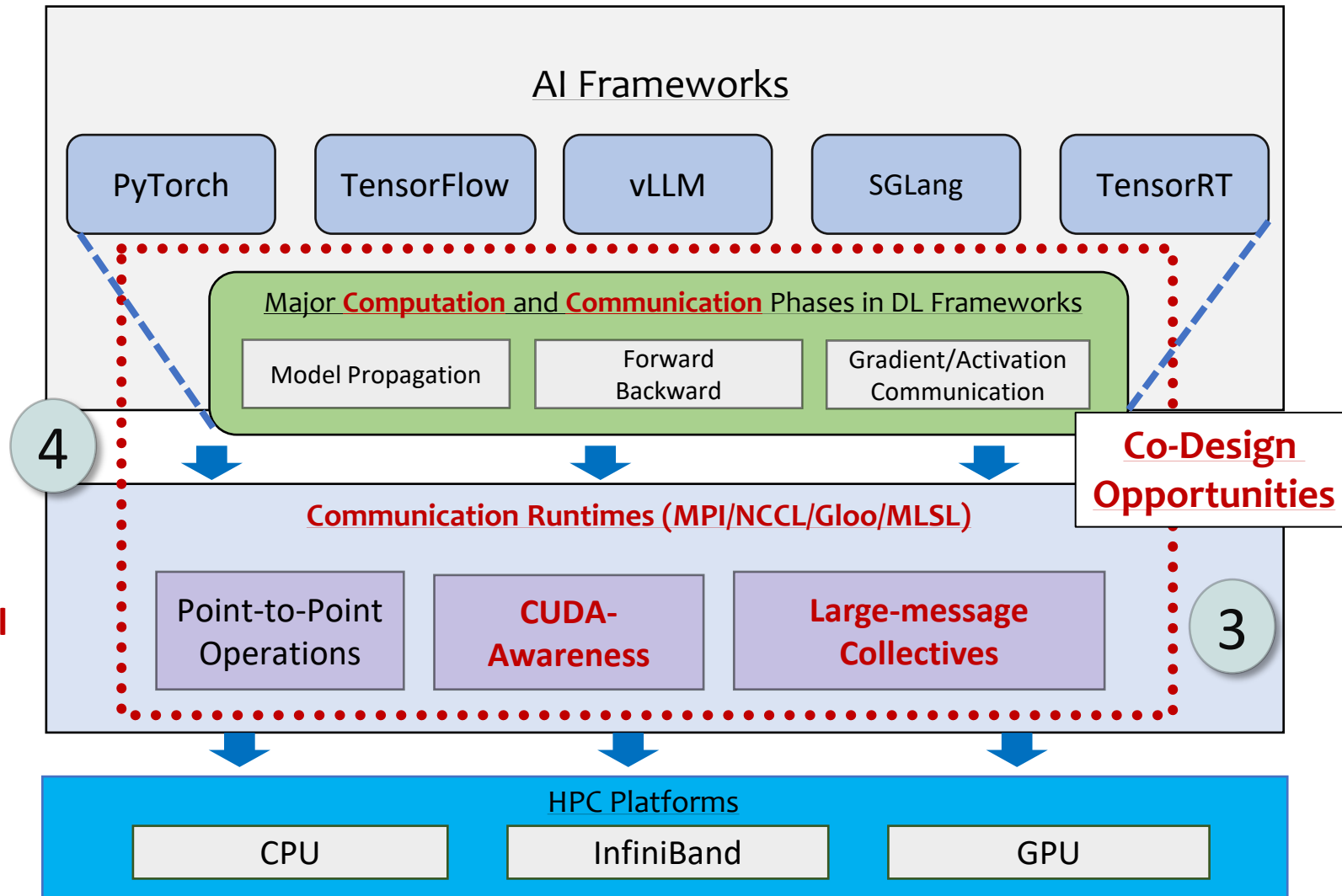
Research Challenges to Exploit HPC Technologies (Cont'd)

3. What are the **new design challenges** brought forward by DL frameworks for Communication runtimes?

- Large Message **Collective Communication** and Reductions
- GPU Buffers (**CUDA-Awareness**)

4. Can a **Co-design** approach help in achieving Scale-up and Scale-out efficiently?

- **Co-Design** the support at **Runtime level** and Exploit it at the **DL Framework level**
- What performance benefits can be observed?
- What needs to be fixed at the **communication runtime** layer?



Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- **Modern HPC Architectures for AI**
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

Drivers of Modern HPC Cluster Architectures



**Multi-/Many-core
Processors**



**High Performance Interconnects –
InfiniBand, Slingshot
<1usec latency, 200-400Gbps Bandwidth>**



**Accelerators
high compute density, high
performance/watt
>9.7 TFlop DP on a chip**



SSD, NVMe-SSD, NVRAM

- Multi-core/many-core technologies
- Remote Direct Memory Access (RDMA)-enabled networking (InfiniBand, RoCE, Slingshot)
- Solid State Drives (SSDs), Non-Volatile Random-Access Memory (NVRAM), NVMe-SSD
- Accelerators (NVIDIA GPGPUs)
- Available on HPC Clouds, e.g., Amazon EC2, NSF Chameleon, Microsoft Azure, etc.



Frontier



Fugaku



Summit



Lumi

High-Performance Architectures for Distributed DL

- **Hardware Architectures**

- **Interconnects**

- InfiniBand, RoCE, Omni-Path, Slingshot, etc.

- **Processors**

- GPUs, Multi-/Many-core CPUs, Tensor Processing Unit (TPU), FPGAs, etc.

- **Communication Middleware**

- Message Passing Interface (MPI)

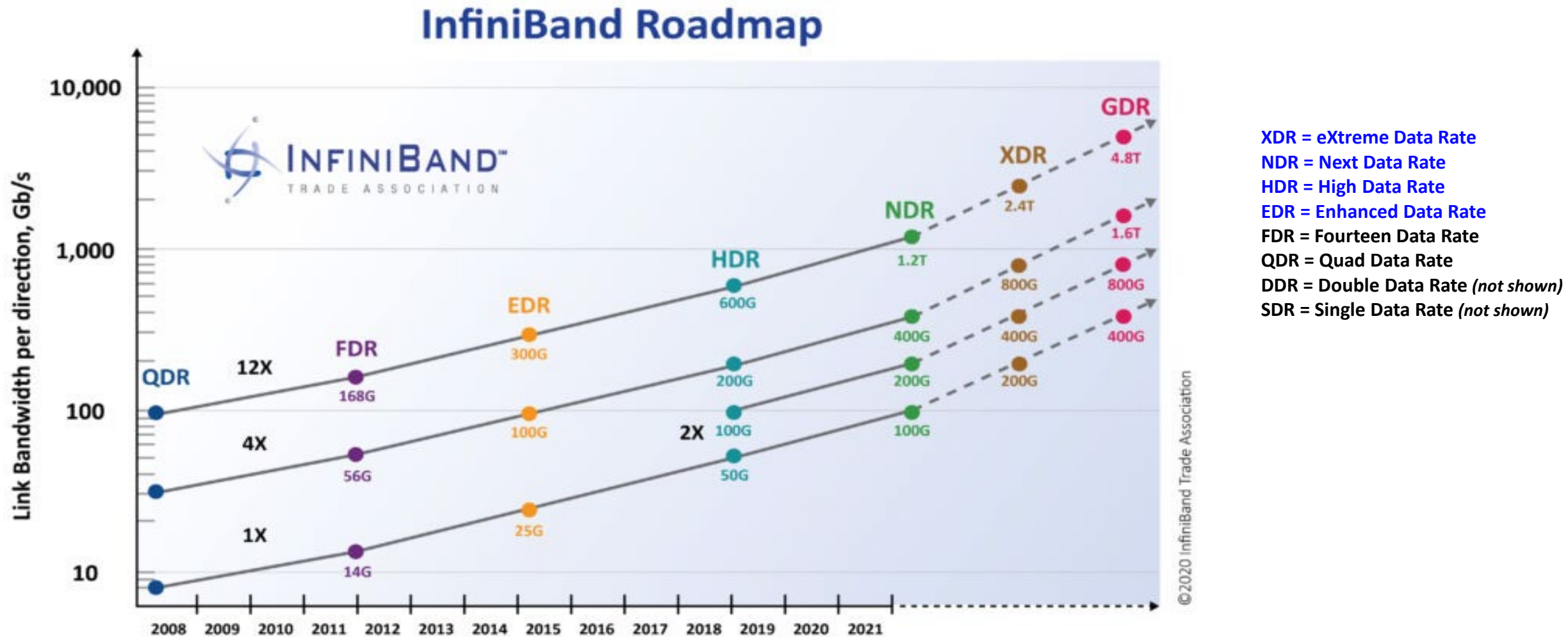
- CUDA-Aware MPI

- NVIDIA NCCL

Overview of High Performance Interconnects

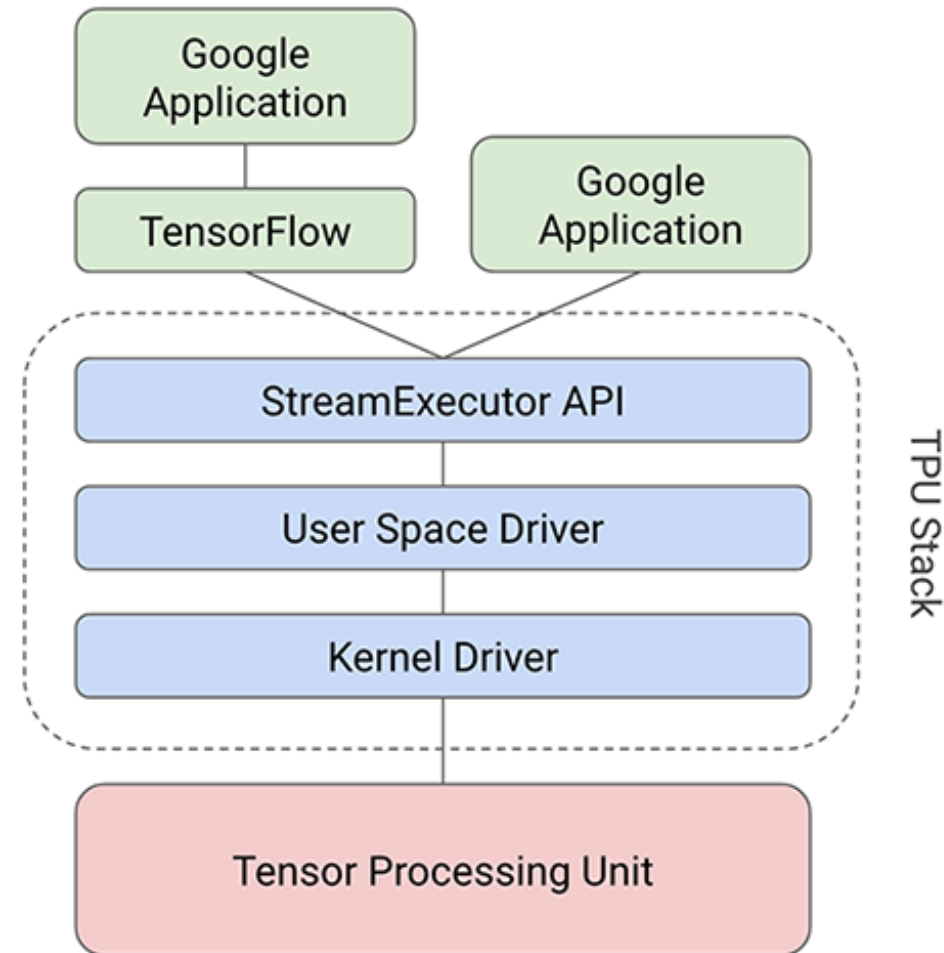
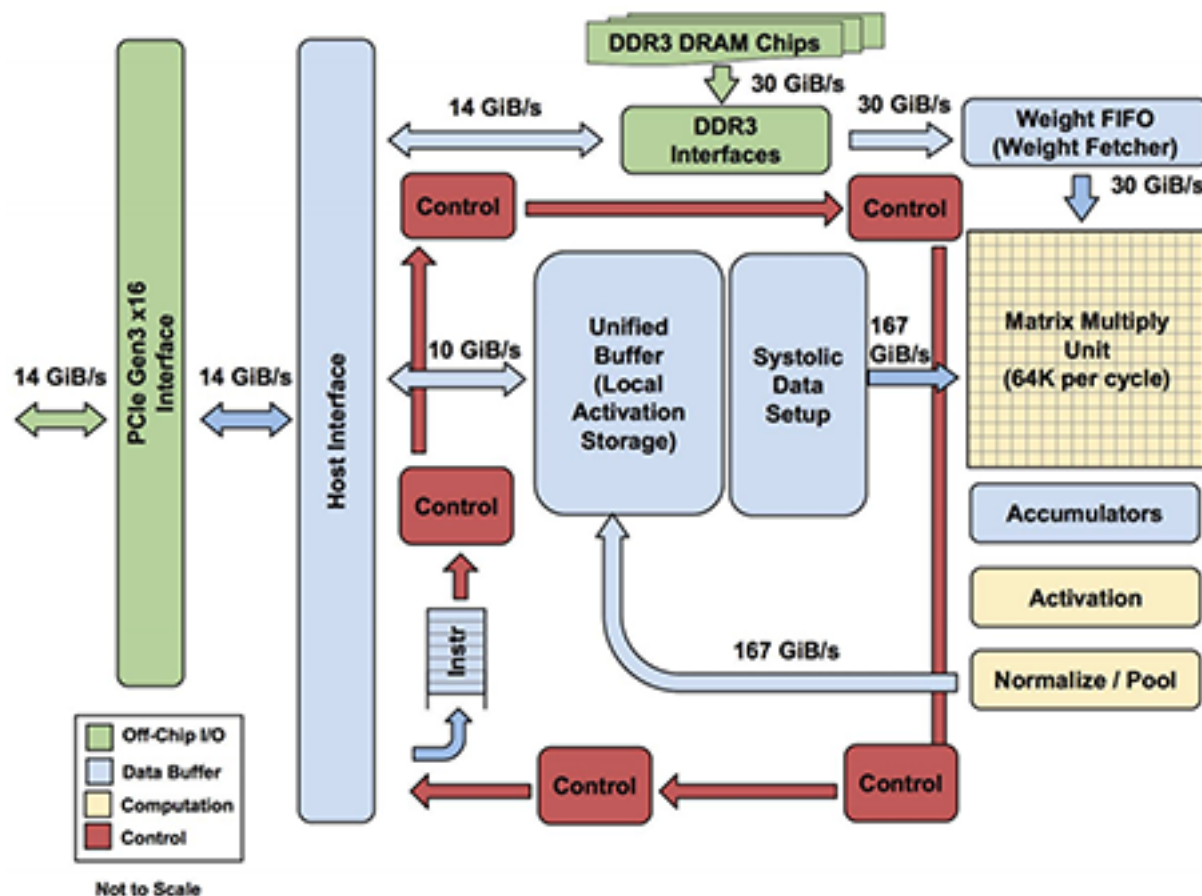
- High-Performance Computing (HPC) has adopted advanced interconnects and protocols
 - InfiniBand (IB)
 - Omni-Path
 - High Speed Ethernet 10/25/40/50/100/200/400 Gigabit Ethernet/iWARP
 - RDMA over Converged Enhanced Ethernet (RoCE)
- Very Good Performance
 - Low latency (few micro seconds)
 - High Bandwidth (400 Gb/s with NDR InfiniBand)
 - Low CPU overhead (5-10%)
- OpenFabrics software stack with IB, Omni-Path, iWARP and RoCE interfaces are driving HPC systems
- Many such systems in Top500 list

InfiniBand Link Speed Standardization Roadmap



Courtesy: InfiniBand Trade Association

Hardware for DNN Training and Inference: TPUs

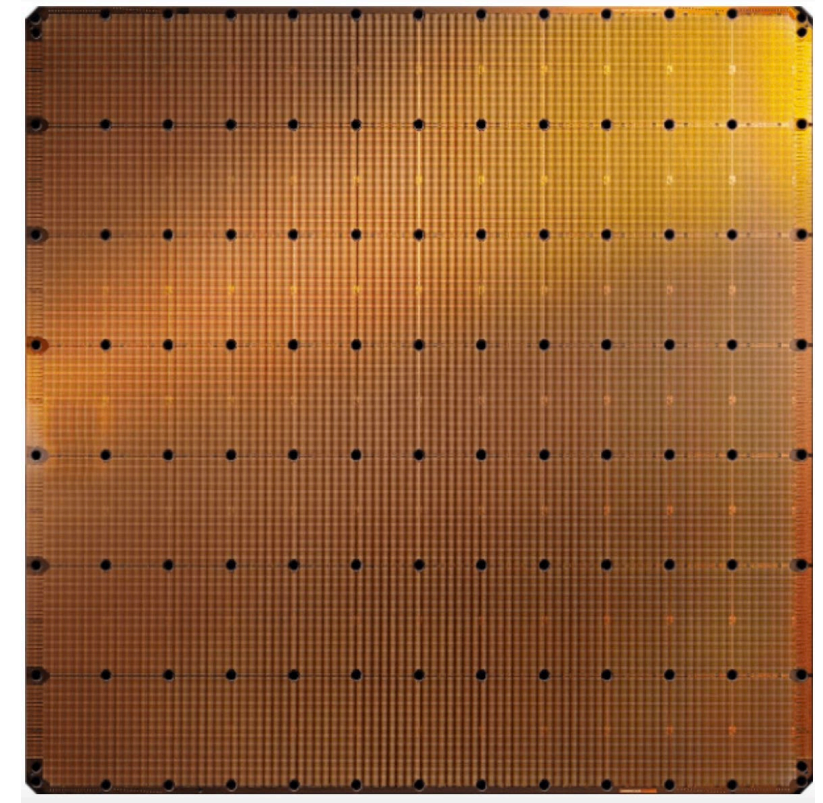


- CISC style instruction set
- Uses systolic arrays as the heart of multiply unit

Courtesy: <https://cloud.google.com/blog/big-data/2017/05/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>
: <https://www.nextplatform.com/2017/04/05/first-depth-look-googles-tpu-architecture/>

Cerebras Introduction

- Cerebras CS1
 - Released in 2019
 - AI accelerator chip with 1.2 trillion transistors
- Cerebras CS2
 - Released in 2021
 - Multi-trillion transistor processor - Cerebras Wafer Scale Engine (2nd Gen)
 - Cerebras System
 - Cerebras software platform



Cerebras WSE-2
2.6 trillion transistors
46,225 nm²

Cerebras CS-3 Architecture

- Cerebras CS3
 - Released in March 2024
 - Over 4 trillion (!!) transistors
- Configured with up to 1.2 Petabytes of external memory
- 250 Exaflops of AI performance with 2048 CS-3 systems linked together
 - Train Llama2-70B in one day

Wafer Scale Engine-3 vs. H100

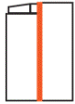
	Cerebras WSE-3	Nvidia H100	Cerebras Advantage
Chip size	46,225 mm ²	814 mm ²	57x
Cores	900,000	16,896 FP32 + 528 Tensor	52x
On-chip memory	44 Gigabytes	0.05 Gigabytes	880x
Memory bandwidth	21 Petabytes/sec	0.003 Petabytes/sec	7,000X
Fabric bandwidth	214 Petabits/sec	0.0576 Petabits/sec	3,715X

Spec	Cerebras CS-3	B200	DGX B200	GB200 NVL72	CS-3 / B200	CS-3 / DGX B200	CS-3 / NVL72
FP16 PFLOPs	125	4.4	36	360	28.4	3.5	0.3
Memory (GB)	1,200,000	192	1,536	13,500	6,250.0	781.3	88.9
NVLink Fabric Bandwidth (TB/s)	26,750	1.8	14.4	130	14,861	1,858	206
Power (Watts)	23,000	1,000	14,300	120,000	23.0	1.6	0.2
PFLOPs / W	0.005	0.004	0.003	0.003	1.2	2.2	1.8

LLM Training Scalability with Cerebras

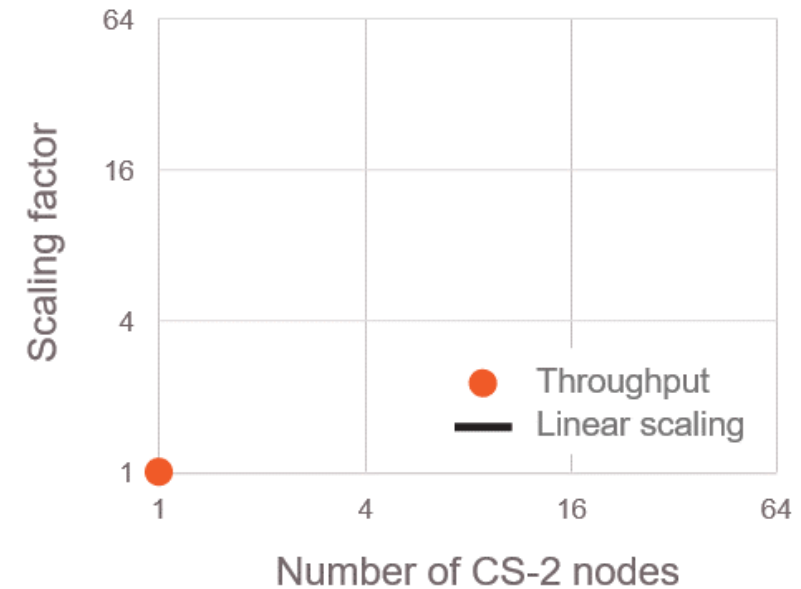
Ease-of-use

```
python run.py
--params params.yaml      ← Where's your dataset?
--num_csx = 1             ← How many nodes?
--model_dir = model_dir   ← Where to store weights?
--num_steps = 1000        ← How many training steps?
--mode=train              ← Train, evaluate or infer?
```



Linear scaling

Cerebras Cluster Performance



On GPT-3 XL model, Cerebras shows perfect linear scaling up to 16 CS-2s
— that's perfect scaling up to 13.6 million cores.

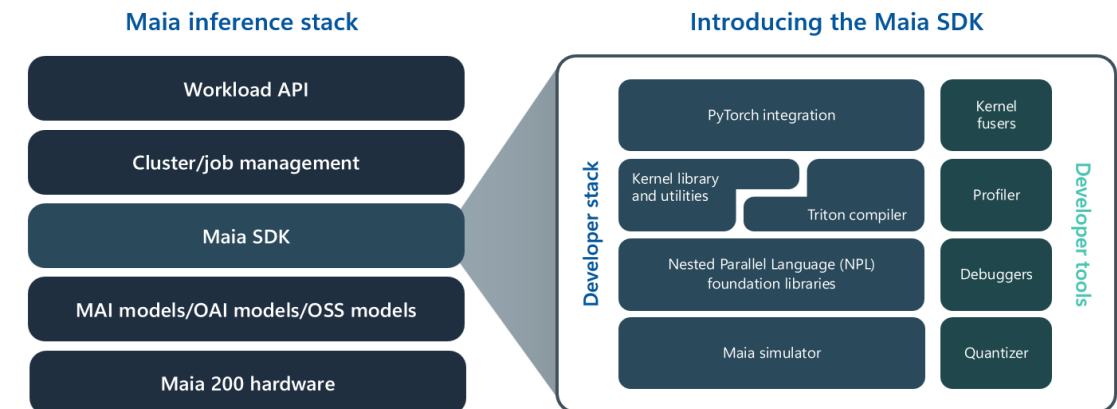
Microsoft Maia 200: “The AI Accelerator Built for Inference”

Industry-leading capability

- Microsoft Developing HW for AI inference in in-house Azure deployments
- Features:
 - 272MB on-die SRAM, partitioned to Cluster and Tile-level channels; low-latency, high-bandwidth data management
 - Custom on-chip interconnect for high-throughput transfers that uses the Microsoft Collective Communication Library (MCCL).
 - Two-Tier ScaleUp Interconnect with proprietary “AI Transport Layer” – scales to 6144 Maia accelerators

Peak specifications	Azure Maia 200	AWS Trainium3	Google TPU v7
Process Node	3nm	3nm	3nm
FP4 TFLOPS	10,145	2,517	n/a
FP8 TFLOPS	5,072	2,517	4,614
BF16 TFLOPS	1,268	671	2,307
HBM technology	HBM3E	HBM3E	HBM3E
HBM BW (TB/s)	7	4.9	7.4
HBM capacity (GB)	216	144	192
Scale-up BW: bidirectional (TB/s)	2.8 TB/s	2.2-2.56 TB/s	1.2 TB/s

Maia software stack



Courtesy: <https://techcommunity.microsoft.com/blog/azureinfrastructureblog/deep-dive-into-the-maia-200-architecture/4489312>, ;
<https://blogs.microsoft.com/blog/2026/01/26/maia-200-the-ai-accelerator-built-for-inference/>

Meta MTIA 3rd/4th/5th Generation Inference Chips

- Versions 1 and 2: 2023 and 2025 (See ISCA papers from those years)
- Made initially for Ranking and Recommendation (R&R) model training prior to GenAI
- MTIA 300/400/450/500: see increased scale-up network bandwidth
 - MTIA 400: 72-accelerator scale-up domain; on-path to deployment
 - MTIA 450: doubled HBM performance; to be deployed in early 2027
 - MTIA 500: 50% more HBM performance and mixed precision flops. To be deployed

Courtesy: <https://ai.meta.com/blog/meta-mtia-scale-ai-chips-for-billions/>

Metric	MTIA 300	MTIA 400	MTIA 450	MTIA 500
Workload Focus	R&R Training	General	GenAI Inference	GenAI Inference
Module TDP	800 W	1200 W	1400 W	1700 W
HBM Bandwidth	6.1 TB/s	9.2 TB/s	18.4 TB/s	27.6 TB/s
HBM Capacity	216 GB	288 GB	288 GB	384-512 GB
MX4 Performance	—	12 PFLOPs	21 PFLOPs	30 PFLOPs
FP8/MX8 Performance	1.2 PFLOPs	6 PFLOPs	7 PFLOPs	10 PFLOPs
BF16 Performance	0.6 PFLOPs	3 PFLOPs	3.5 PFLOPs	5 PFLOPs
Scale-up Domain Size	16	72	72	72
Scale-up Network (unidirectional bandwidth*)	1 TB/s	1.2 TB/s	1.2 TB/s	1.2 TB/s
Scale-out Network (unidirectional bandwidth*)	200 GB/s**	100 GB/s	100 GB/s	100 GB/s

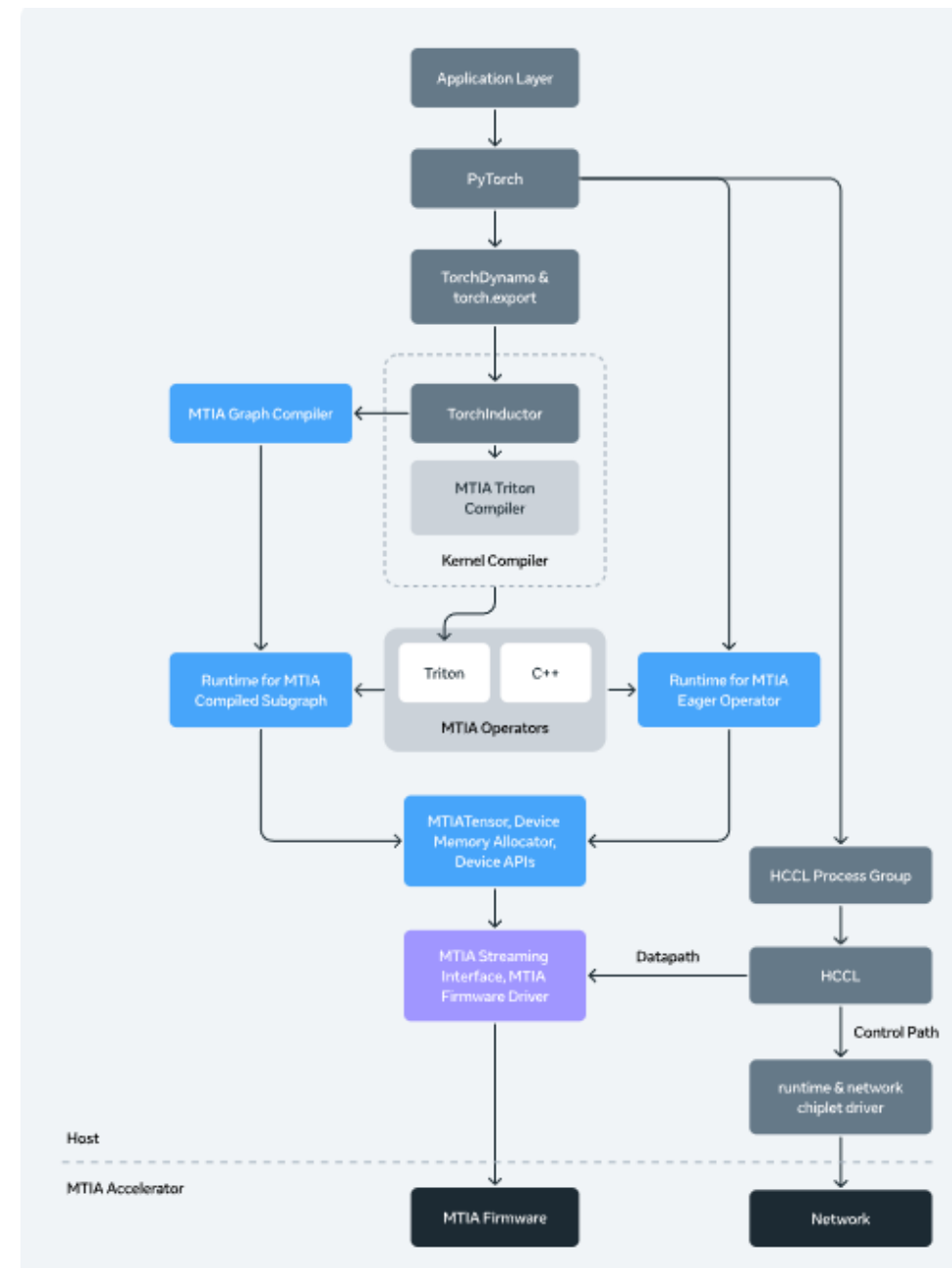
*Some vendors report bidirectional bandwidth. Multiply the value in the table by two to obtain the corresponding bidirectional bandwidth.

**MTIA 300 is configured with a scale-out network with higher bandwidth (200 GB/s) due to its relatively small scale-up domain size and the target R&R workloads.

Meta MTIA Diagrams and Software

- MTIA 500 diagram: Four trays of HBM (1 per PE Grid) with efficient device-to-device transfer
- System-on-Chip (SoC) attached for host inference and two, 6x800 Gbps RoCE interfaces
- MTIA Software Stack: PyTorch-native approach (Compilers, model onboarding, kernel authoring) on top of MTIA's "Hoot Collective Communications Library" (HCCL)
 - Provides a "GPU-like" experience with minimal overhead/changes

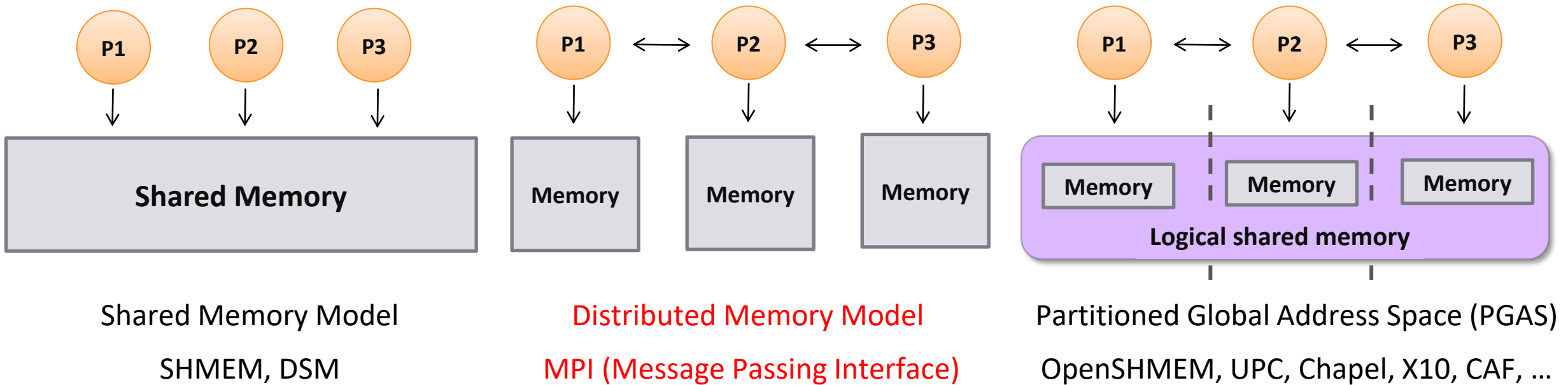
Courtesy: <https://ai.meta.com/blog/meta-mtia-scale-ai-chips-for-billions/>



High-Performance Architectures for Distributed DL

- Hardware Architectures
 - Interconnects
 - InfiniBand, RoCE, Omni-Path, etc.
 - Processors
 - GPUs, Multi-/Many-core CPUs, Tensor Processing Unit (TPU), FPGAs, etc.
- **Communication Middleware**
 - **Message Passing Interface (MPI)**
 - **CUDA-Aware MPI**
 - **NVIDIA NCCL**

Parallel Programming Models Overview



- Programming models provide abstract machine models
- Models can be mapped on different types of systems
 - e.g. Distributed Shared Memory (DSM), MPI within a node, etc.
- PGAS models and Hybrid MPI+PGAS models are gradually receiving importance

Allreduce Collective Communication Pattern

- Element-wise Sum data from all processes and sends to all processes

```
int MPI_Allreduce (const void *sendbuf, void * recvbuf, int count, MPI_Datatype datatype,  
                  MPI_Op operation, MPI_Comm comm)
```

Input-only Parameters

Parameter	Description
sendbuf	Starting address of send buffer
recvbuf	Starting address of recv buffer
type	Data type of buffer elements
count	Number of elements in the buffers
operation	Reduction operation to be performed (e.g. sum)
comm	Communicator handle

Input/Output Parameters

Parameter	Description
recvbuf	Starting address of receive buffer

Sendbuf (Before)

T1	T2	T3	T4
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

Recvbuf (After)

T1	T2	T3	T4
4	4	4	4
8	8	8	8
12	12	12	12
16	16	16	16

Overview of the MVAPICH Project

- High Performance open-source MPI Library
- Support for multiple interconnects
 - InfiniBand, Omni-Path, Ethernet/iWARP, RDMA over Converged Ethernet (RoCE), AWS EFA, OPX, Broadcom RoCE, Intel Ethernet, Rockport Networks, Slingshot 10/11
- Support for multiple platforms
 - x86, OpenPOWER, ARM, Xeon-Phi, GPGPUs (NVIDIA and AMD)
- Started in 2001, first open-source version demonstrated at SC '02
- Supports the latest MPI-5.0 standard
- <http://mvapich.cse.ohio-state.edu>
- Additional optimized versions for different systems/environments:
 - MVAPICH-Plus (Unification of MVAPICH2-X and MVAPICH2-GDR), since 2023
 - MVAPICH2-X (Advanced MPI + PGAS), since 2011
 - MVAPICH2-GDR with support for NVIDIA (since 2014) and AMD (since 2020) GPUs
 - MVAPICH2-MIC with support for Intel Xeon-Phi, since 2014
 - MVAPICH2-Virt with virtualization support, since 2015
 - MVAPICH2-EA with support for Energy-Awareness, since 2015
 - MVAPICH2-Azure for Azure HPC IB instances, since 2019
 - MVAPICH2-X-AWS for AWS HPC+EFA instances, since 2019
- Tools:
 - **OSU MPI Micro-Benchmarks (OMB), since 2003**
 - OSU InfiniBand Network Analysis and Monitoring (INAM), since 2015



- **Used by more than 3,500 organizations in 93 countries (listed under the Users Tab of the MVAPICH page)**
- **More than 2.08 Million downloads from the OSU site directly**
- Empowering many TOP500 clusters (Jun '26 ranking)
 - 27th, 10,649,600-core (Sunway TaihuLight) at NSC, Wuxi, China
 - 90th, 448, 448 cores (Frontera) at TACC
 - 298th, 42,336 cores (Cosmos) at SDSC
- Available with software stacks of many vendors and Linux Distros (RedHat, SuSE, OpenHPC, and Spack)
- Partner in the 90th ranked TACC Frontera system and upcoming Horizon LCCF System
- **Empowering Top500 systems for more than 20+ years**

GPU-Aware (CUDA-Aware) MPI Library: MVAPICH2-GDR

- Standard MPI interfaces used for unified data movement
- Takes advantage of Unified Virtual Addressing ($\geq$ CUDA 4.0)
- Overlaps data movement from GPU with RDMA transfers

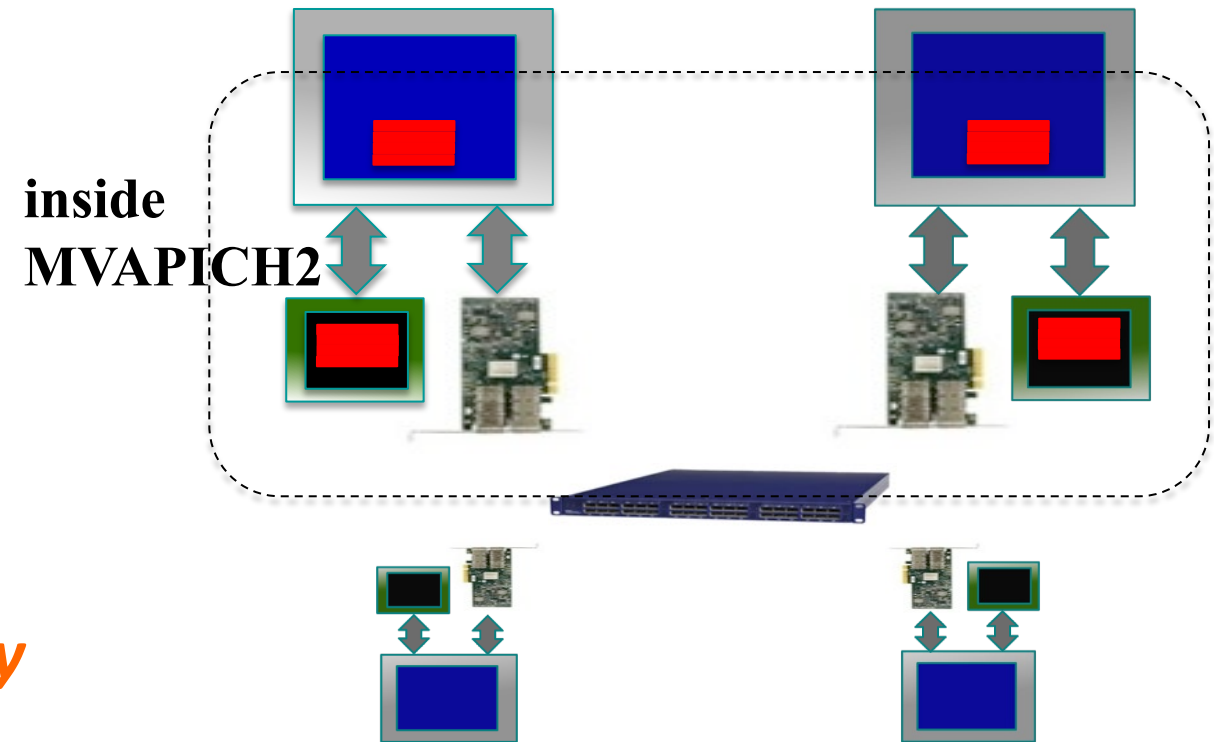
At Sender:

```
MPI_Send(s_devbuf, size, ...);
```

At Receiver:

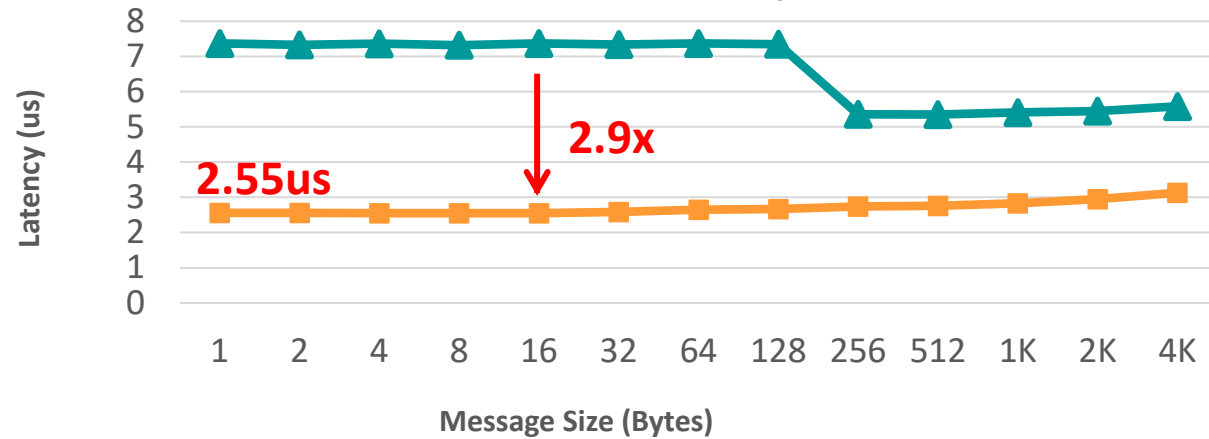
```
MPI_Recv(r_devbuf, size, ...);
```

High Performance and High Productivity

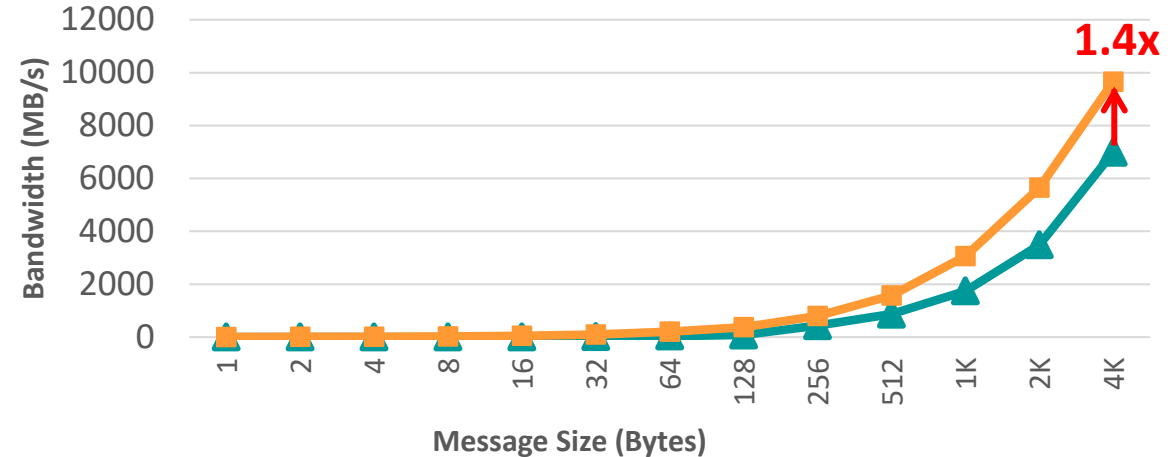


Optimized MVAPICH-Plus Design

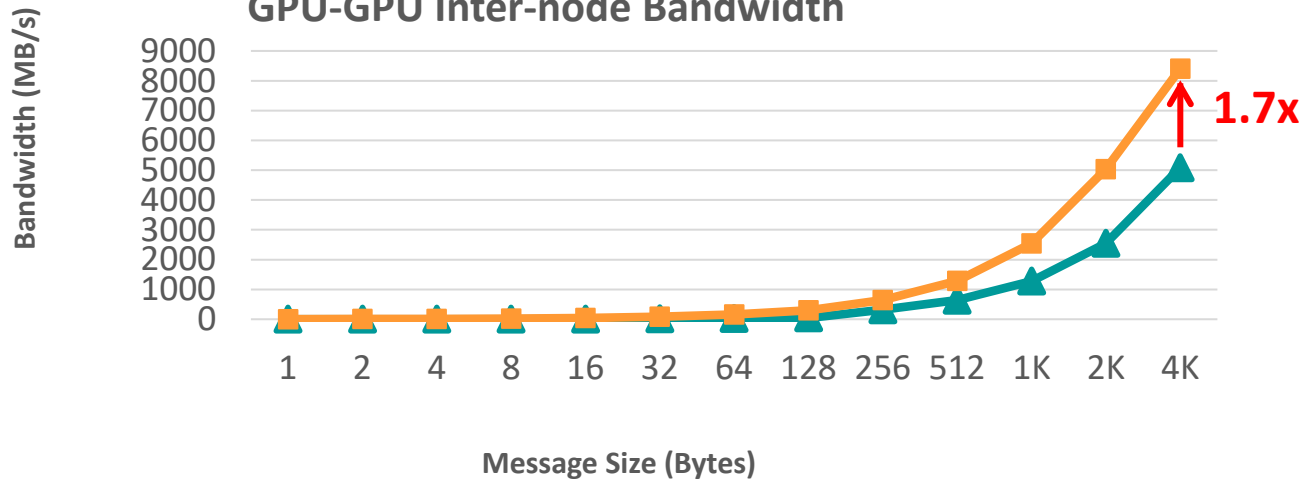
GPU-GPU Inter-node Latency



GPU-GPU Inter-node Bi-Bandwidth



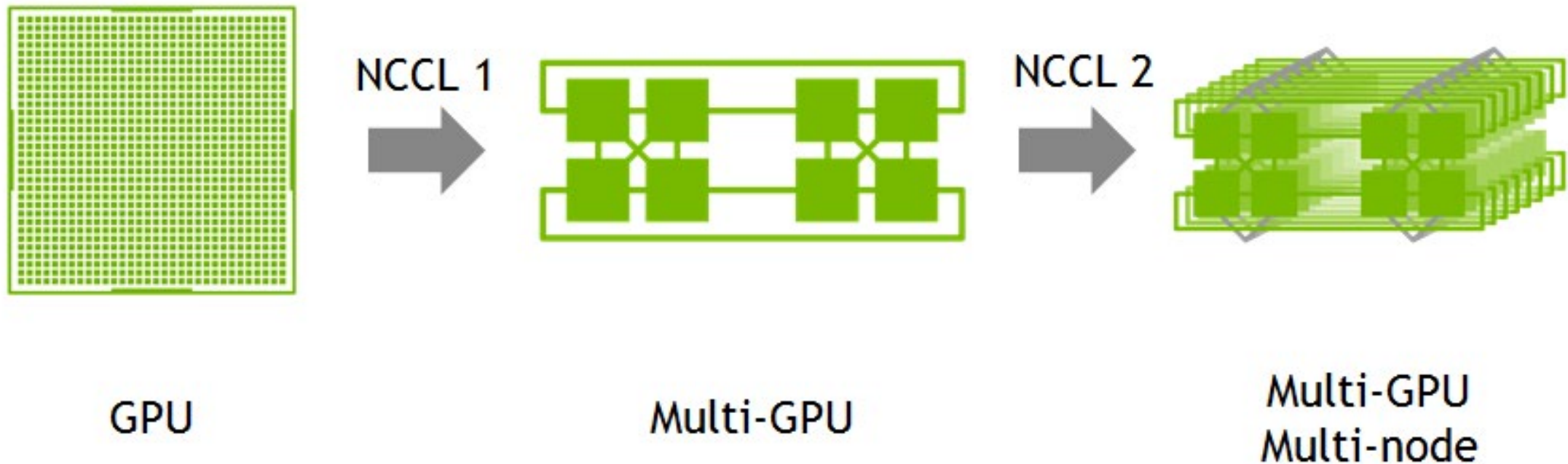
GPU-GPU Inter-node Bandwidth



MVAPICH-Plus 4.1
GH200 Superchip (Grace @ 3.1 GHz) node - 72 cores
NVIDIA H100 GPU
Mellanox ConnectX-7 NDR400 HCA
CUDA 12.4
Mellanox OFED 25.01 with GPU-Direct-RDMA

NCCL Communication Library

- NVIDIA Collective Communication Library (NCCL)
- Main Motivation: Deep Learning workloads
- NCCL1– efficient dense-GPU communication within the node
- NCCL2– multiple DGX systems connected to each other with InfiniBand systems



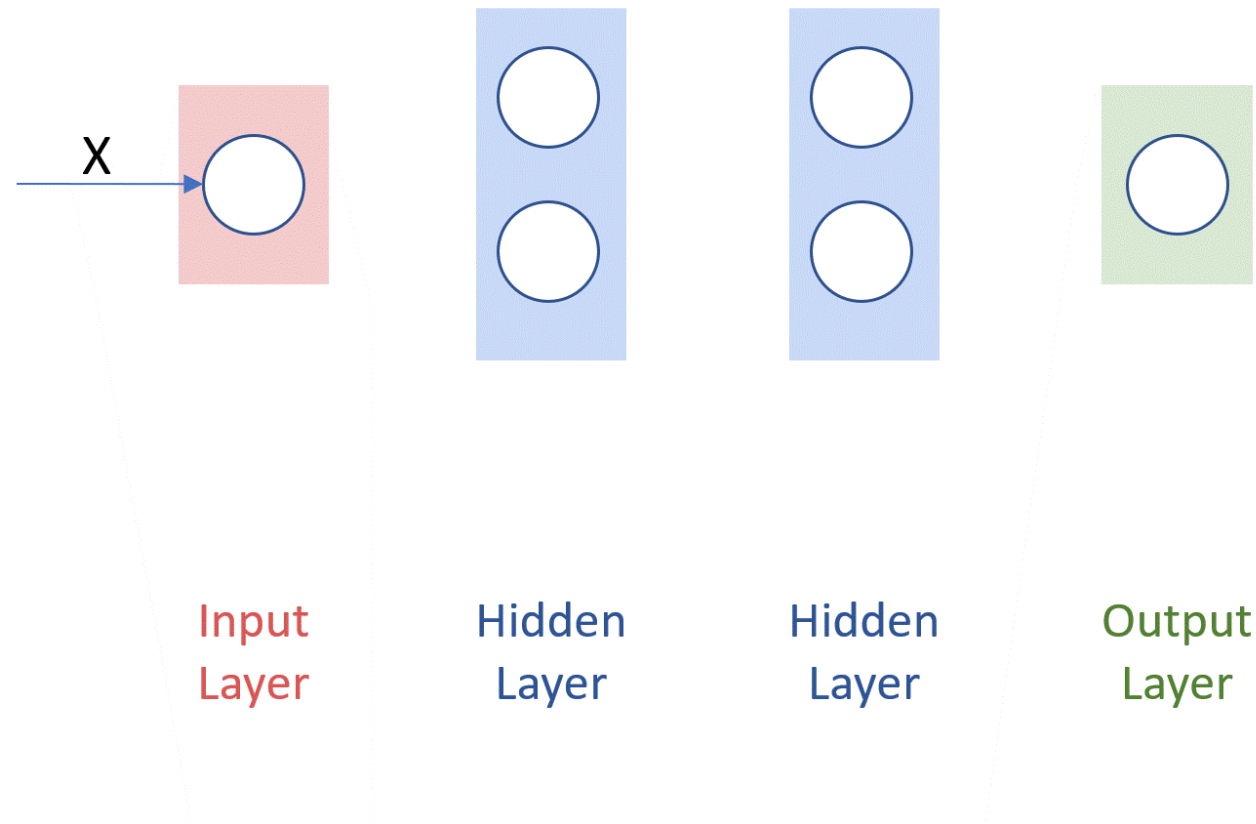
Courtesy: <https://developer.nvidia.com/nccl>

Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- **Foundations of Deep Neural Networks**
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

Understanding the Deep Neural Network Concepts

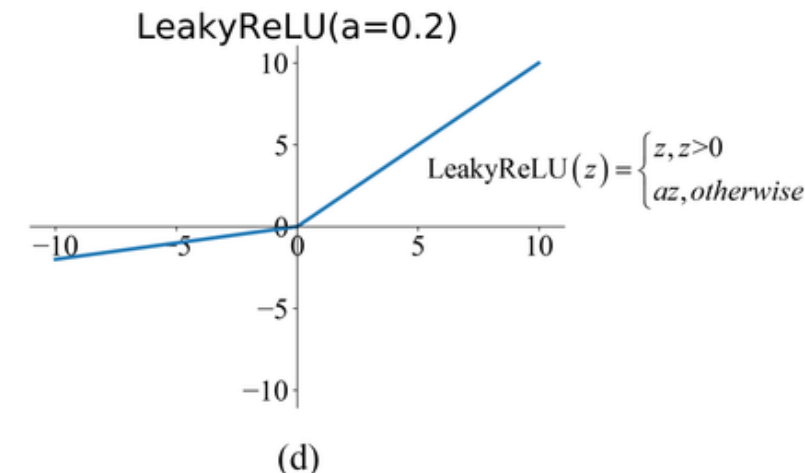
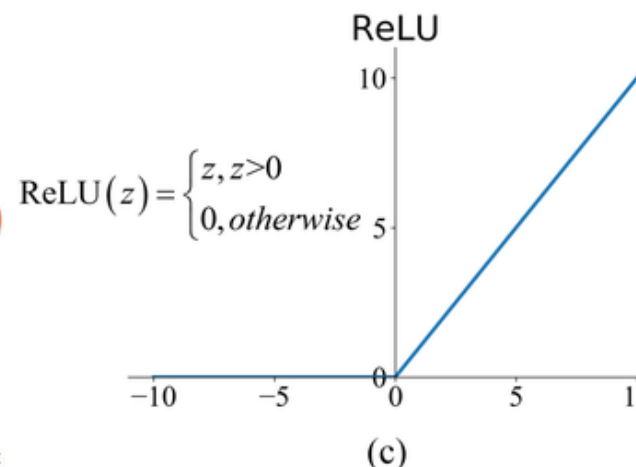
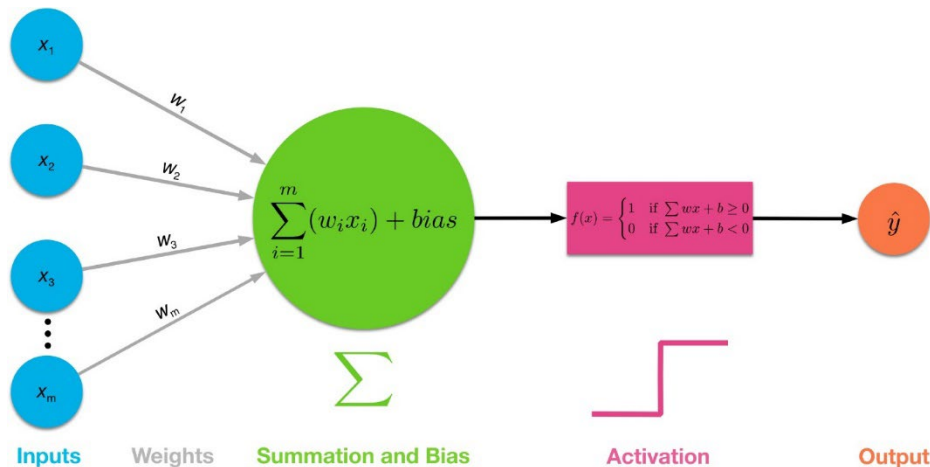
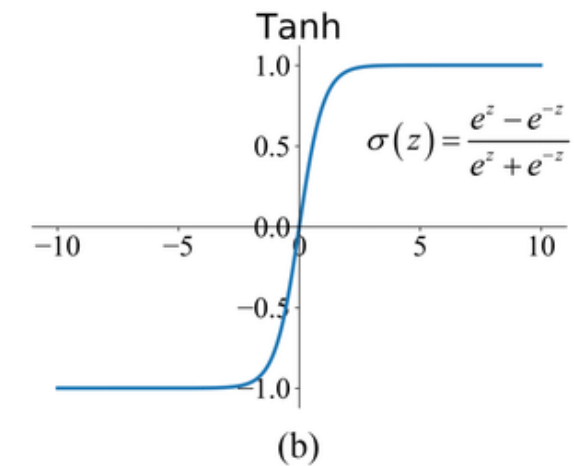
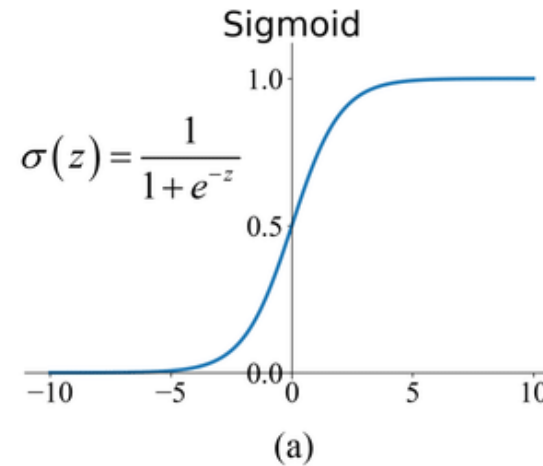
- Example of a 3-layer Deep Neural Network (DNN) – (input layer is not counted)



Courtesy: <http://cs231n.github.io/neural-networks-1/>

Essential Concepts: Activation Functions

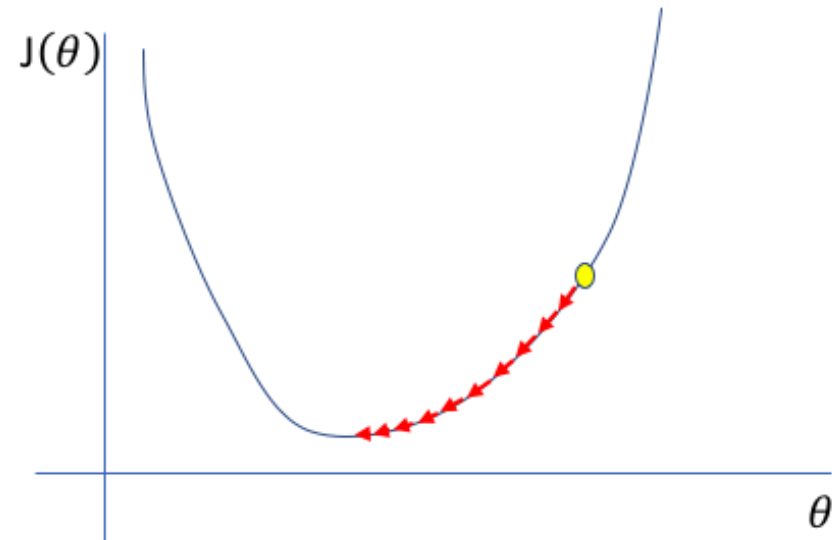
- Sigmoid
- Tanh
- ReLU
- Leaky ReLU



Courtesy: <https://journals.aps.org/pre/abstract/10.1103/PhysRevE.100.033308>

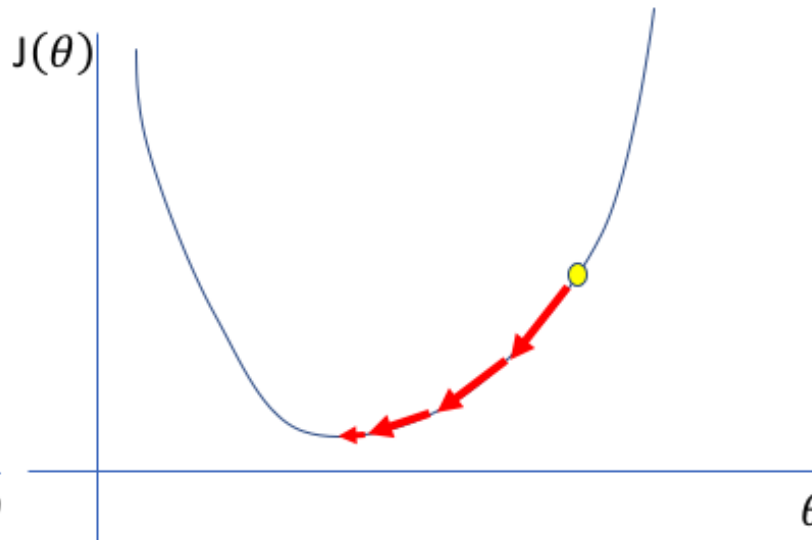
Essential Concepts: Learning Rate (α)

Too low



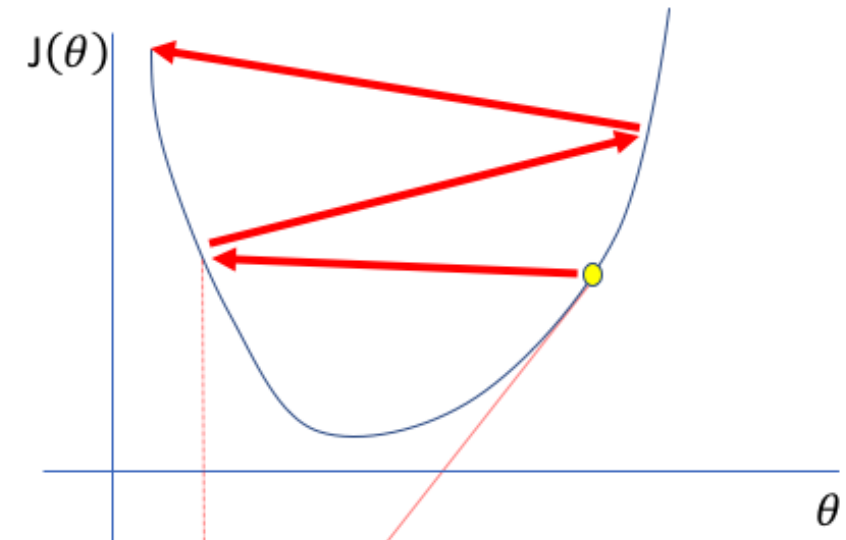
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high

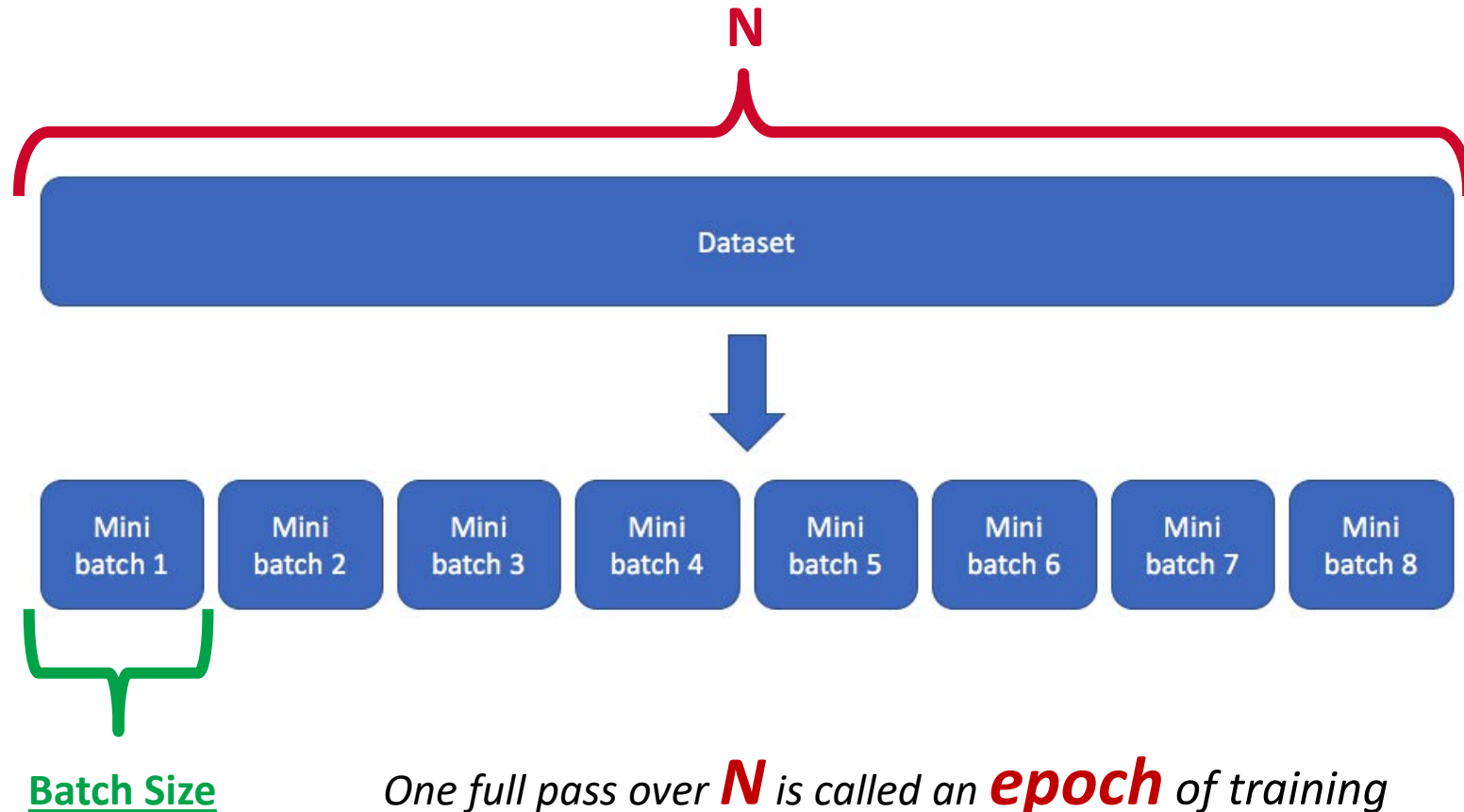


Too large of a learning rate causes drastic updates which lead to divergent behaviors

Courtesy: <https://www.jeremyjordan.me/nn-learning-rate/>

Essential Concepts: Batch Size

- Batched Gradient Descent
 - Batch Size = **N**
- Stochastic Gradient Descent
 - Batch Size = **1**
- Mini-batch Gradient Descent
 - Somewhere in the middle
 - Common:
 - Batch Size = 64, 128, 256, etc.
- Finding the optimal batch size will yield the fastest learning.

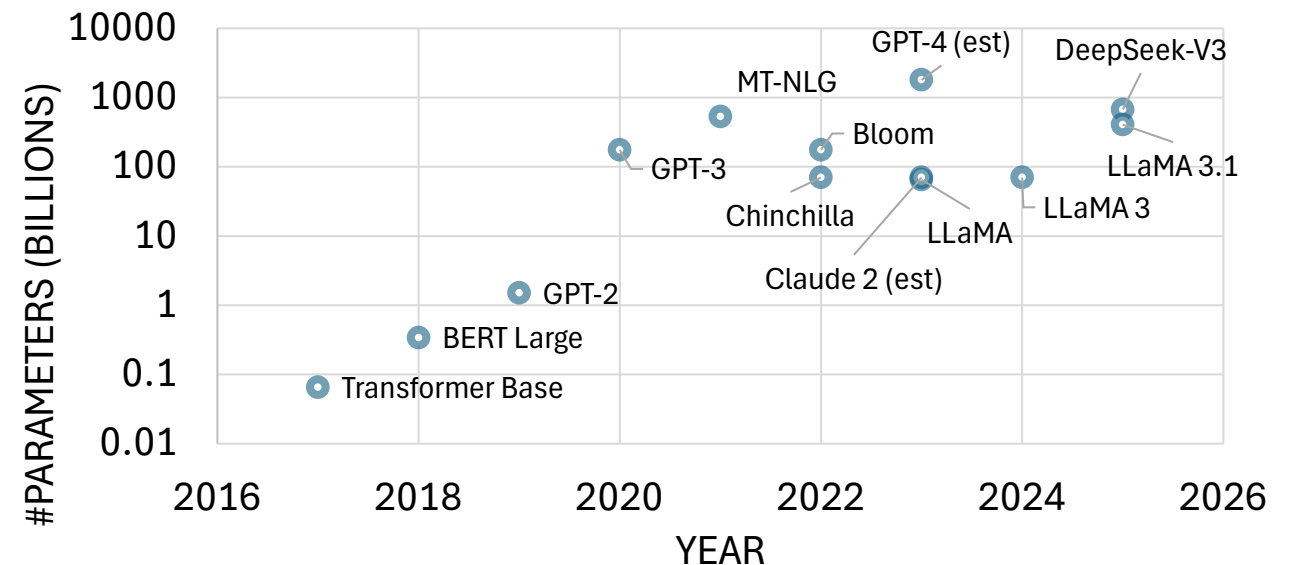
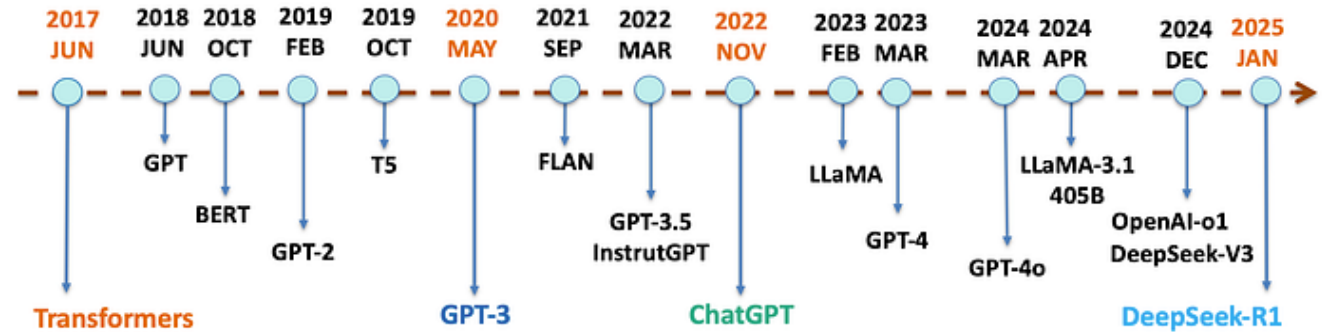


Courtesy: <https://www.jeremyjordan.me/gradient-descent/>

Evolution of Language Models

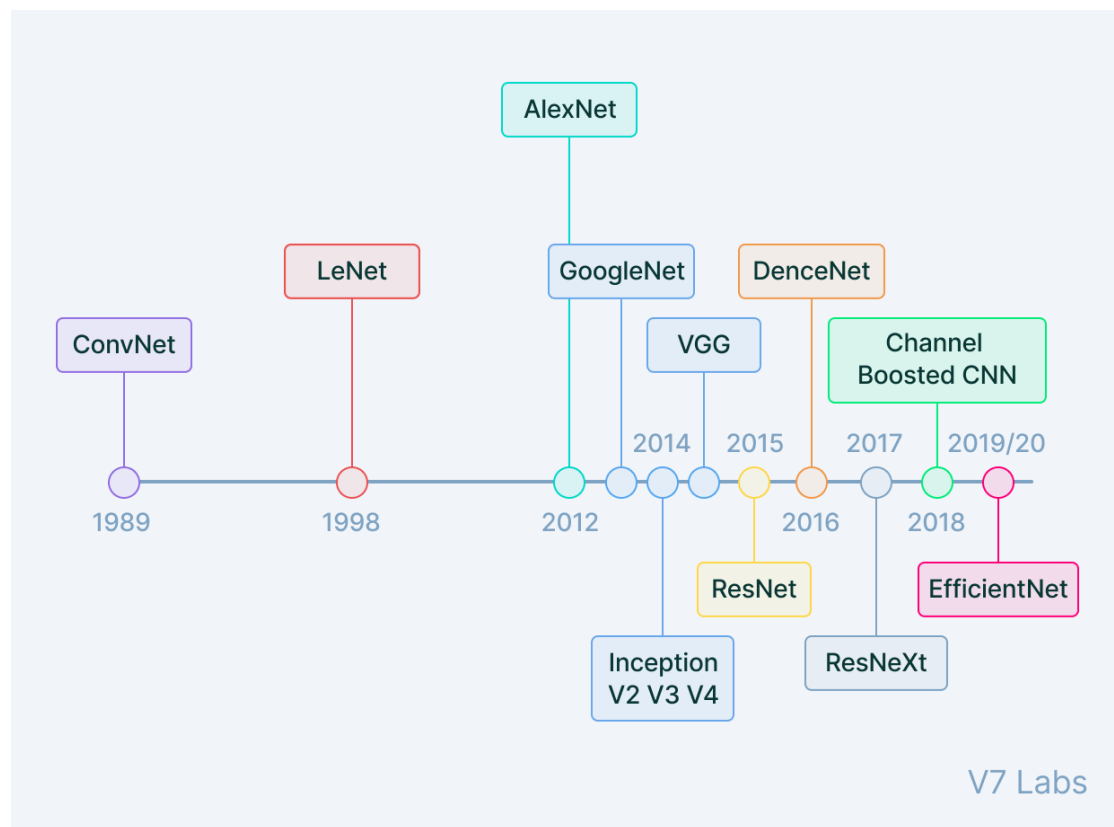
- LLM Performance improves predictably with scale (# of parameters and dataset size).
- Bigger $\neq$ always better: optimal performance requires balancing model size and data
- Diminishing returns at extreme scale $\rightarrow$ smaller gains per added parameter

A Brief History of LLMs

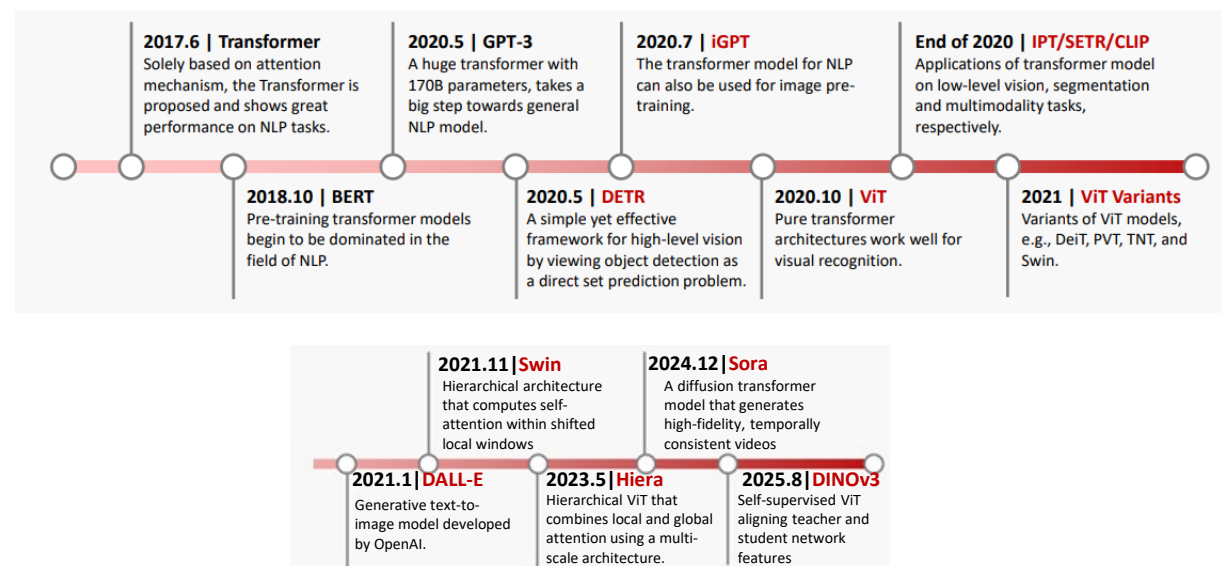


Evolution of Computer Vision Models

CNN Architectures



Vision Transformer Architectures



Courtesy: <https://www.v7labs.com/blog/convolutional-neural-networks-guide>

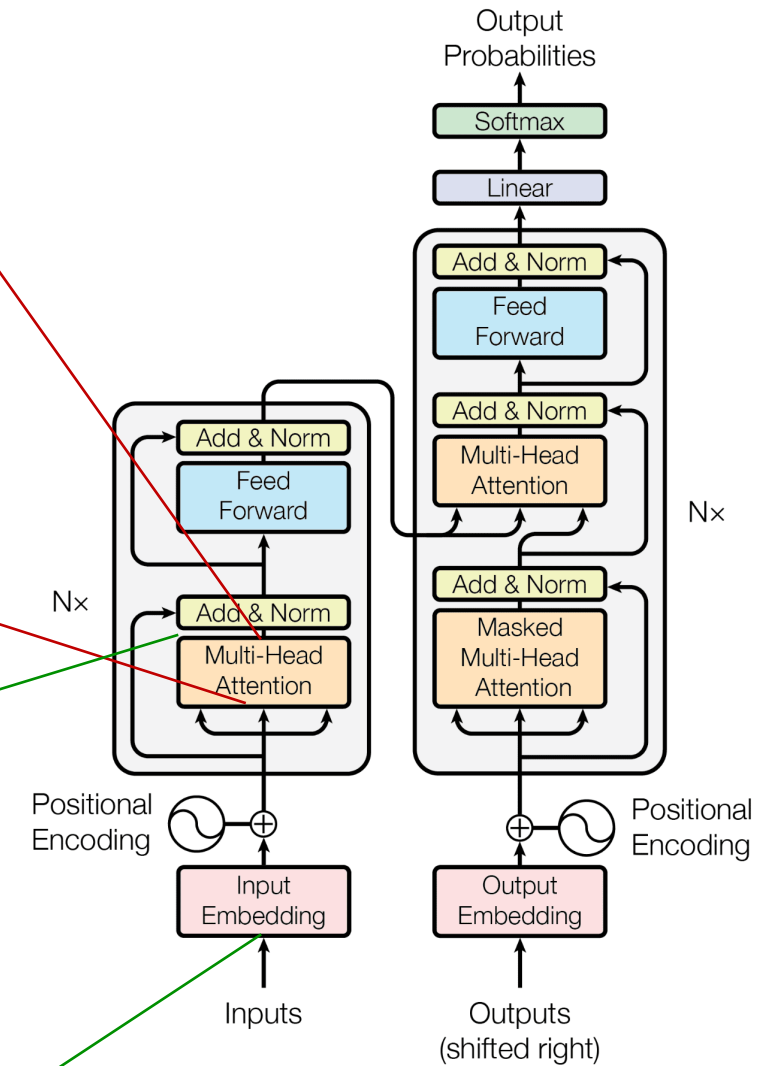
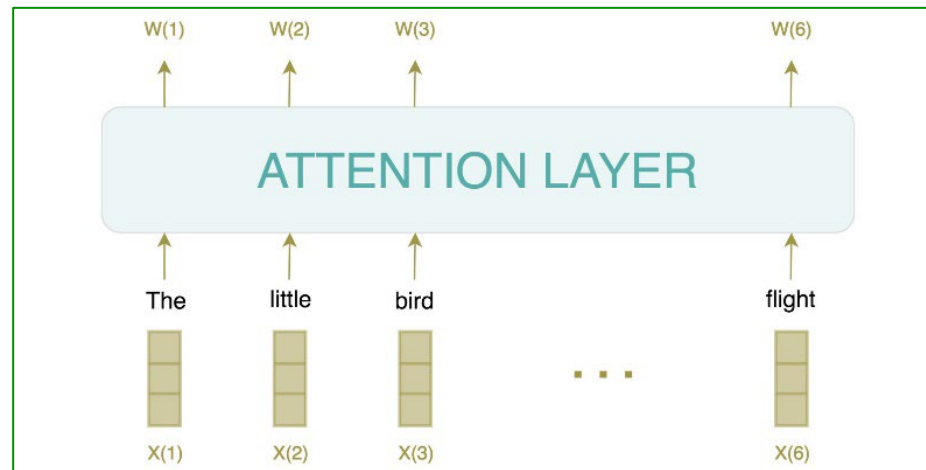
A Survey on Vision Transformer (Kai Han et. Al 2022) <https://arxiv.org/abs/2012.12556>

What are Transformer models?

Focus		Attention Vectors	
The	→ The big red dog	[0.71	0.04 0.07 0.18] ^T
big	→ The big red dog	[0.01	0.84 0.02 0.13] ^T
red	→ The big red dog	[0.09	0.05 0.62 0.24] ^T
dog	→ The big red dog	[0.03	0.03 0.03 0.91] ^T



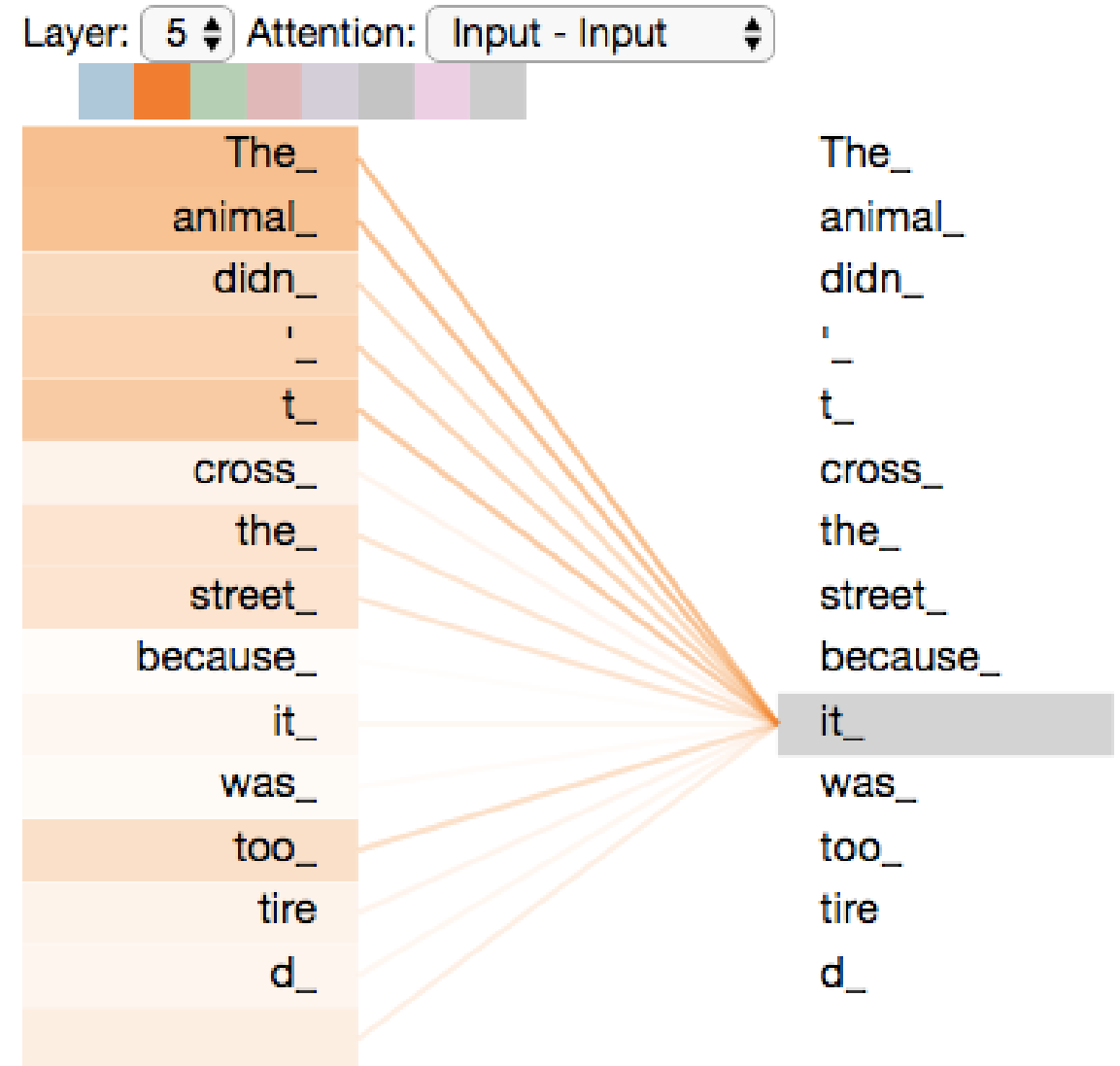
Attention please!



Attention is all you need (Vaswani et al. 2017)
<https://arxiv.org/abs/1706.03762>

Self-Attention: The Backbone of Transformers

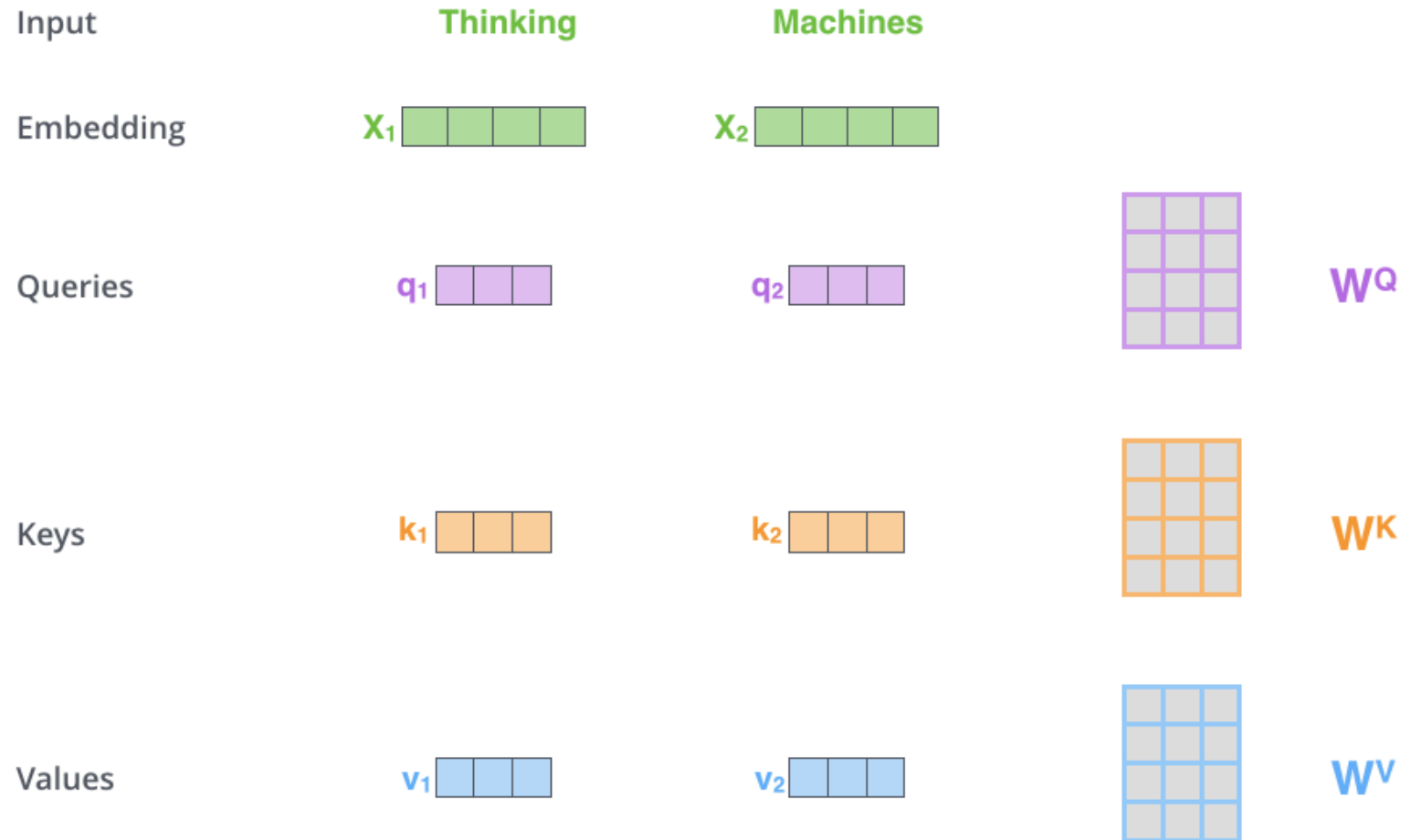
- **Goal:** It relates different positions of a single sequence
 - **Intuition:** For each output in the sequence, predict which input tokens to focus on and how much
- **Consider:** *"The animal didn't cross the street because it was too tired"*
 - What does **it** refer to?
 - Weigh the associations of **it** with every other token in the sequence
- How does a transformer compute this attention?



Courtesy: <https://jalammar.github.io/illustrated-transformer/>

Self-Attention: Query, Key, and Value Decomposition

- Query, Key, and Value vectors are created for each token
- Input or embedding is multiplied with trainable weights to get Query, Key, and Value
 - $Q_1 = X_1 * W^Q$
 - $K_1 = X_1 * W^K$
 - $V_1 = X_1 * W^V$
- What are these mysterious QKV vectors? An aside...



Courtesy: <https://jalammar.github.io/illustrated-transformer/>

Attention Intuitively

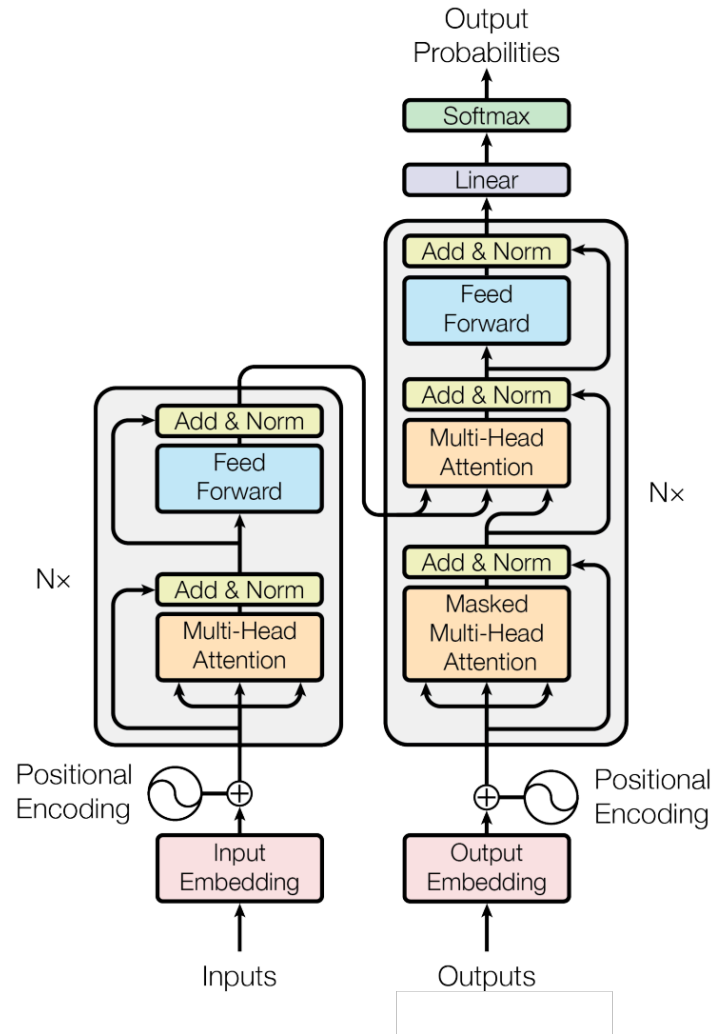
- How should we think of Q, K, V, and their associated weight matrices?
- Attention is just fancy retrieval!
 - What is retrieval?
 - E.g. your Google search maps a **query** (search text) against a set of **keys** (webpage content) associated with candidate webpages in the Google database, then presents the best matched webpages (**values**)
- Attention mimics the retrieval of a **value** v_i for a **query** q based on a **key** k_i in a database
- $attention(q, \mathbf{k}, \mathbf{v}) = \sum_i similarity(q, k_i) \times v_i$

Courtesy: <https://jalammar.github.io/illustrated-transformer/>

What are Transformer models?

BERT

Encoder



GPT

Decoder

Outline

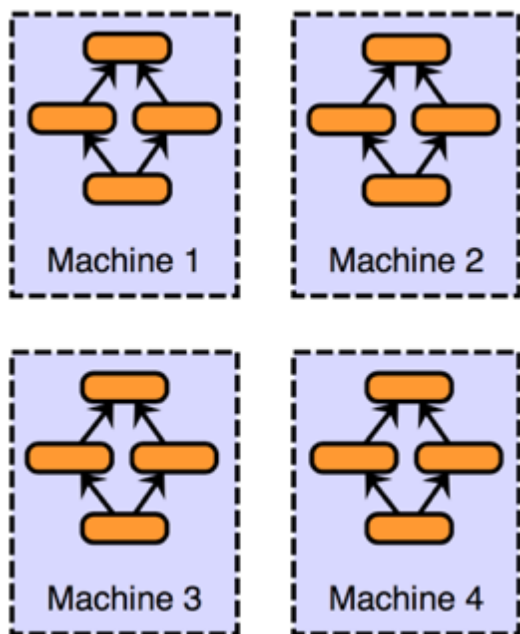
- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- **Distributed Data Parallel Deep Learning**
 - **Basic Principles and Parallelism Strategies**
 - **Lab 1: Hands-on Exercises (Distributed Data Parallelism)**
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

The Need for Parallel and Distributed Training

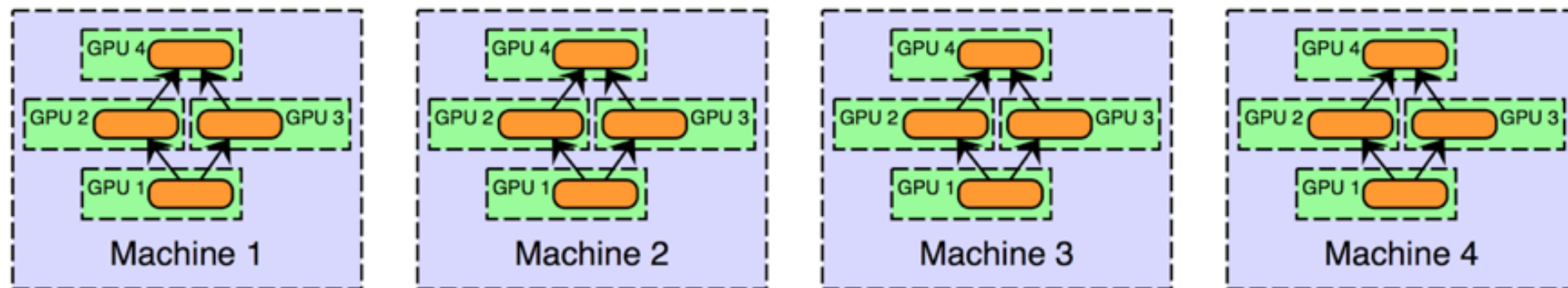
- Why do we need Parallel Training?
- Larger and Deeper models are being proposed
 - **Language Models: RNNs -> Transformers -> BERT – GPT-(1,2,3,4)**
 - **Vision Models: AlexNet -> ResNet -> NASNet – AmoebaNet → Vision Transformers**
 - DNNs require a lot of memory and a lot of computation
 - Larger models cannot fit a GPU's memory
- Single GPU training cannot keep up with ever-larger models
- Community has moved to multi-GPU training
- Multi-GPU in one node is good but there is a limit to Scale-up (8-16 GPUs)
- **Multi-node (Distributed or Parallel) Training is necessary!!**

Parallelization Strategies

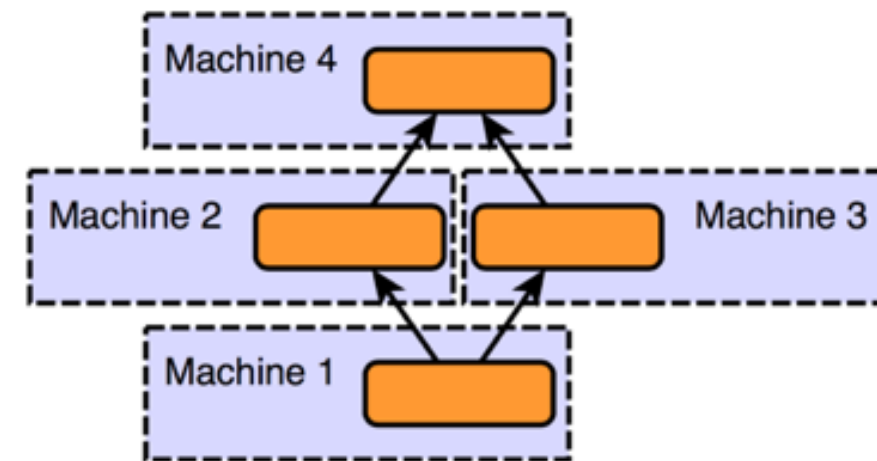
- Some parallelization strategies..
 - Data Parallelism or Model Parallelism
 - Hybrid Parallelism



Data Parallelism



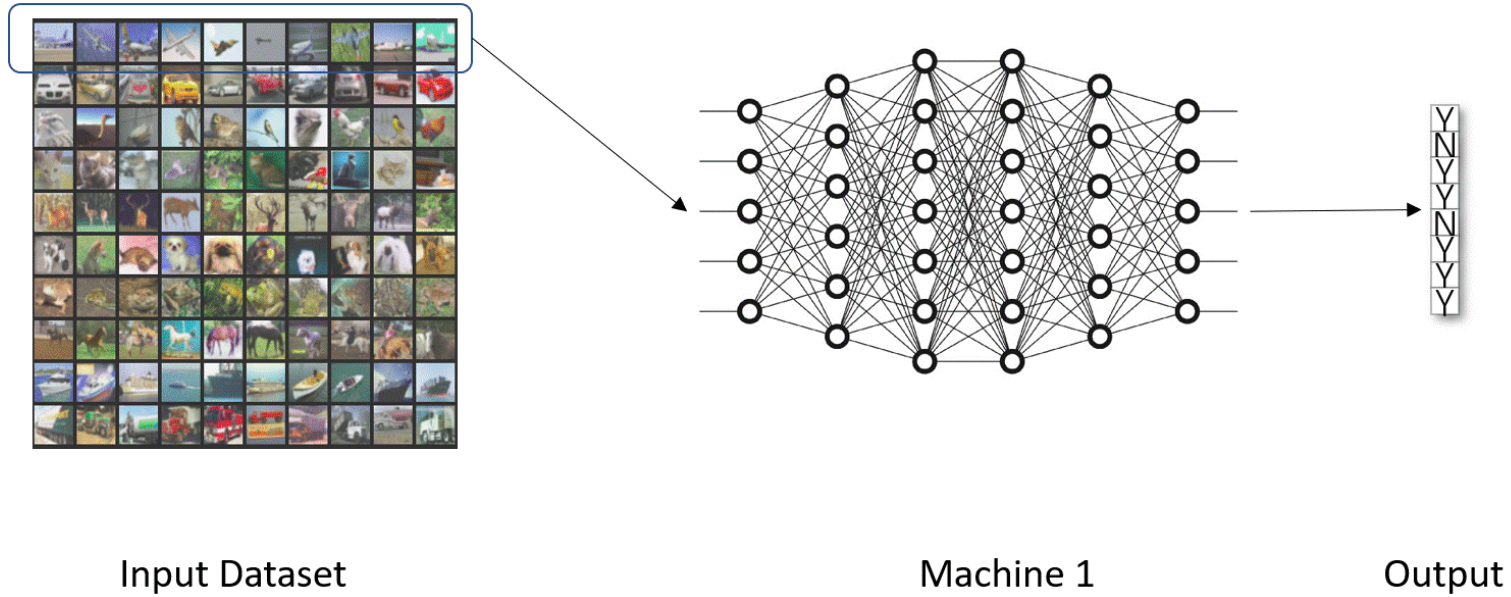
Hybrid (Model and Data) Parallelism



Model Parallelism

Need for Data Parallelism

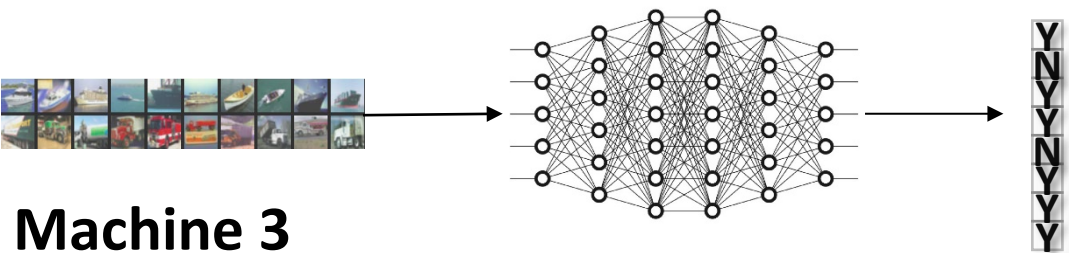
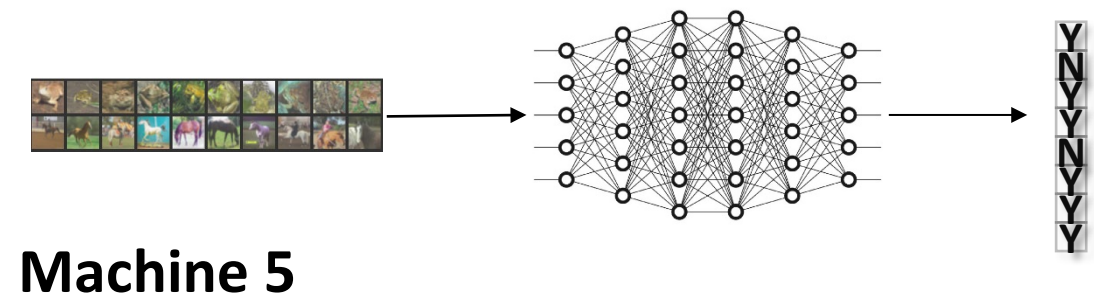
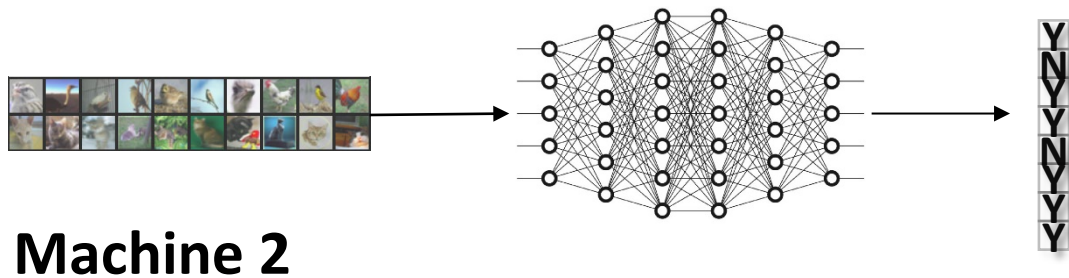
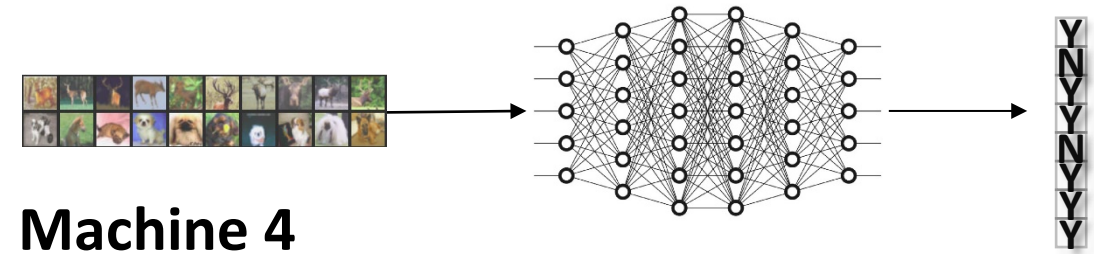
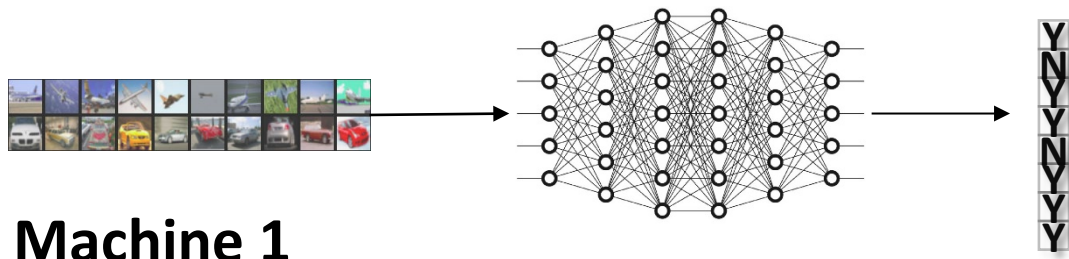
Let's revisit Mini-Batch Gradient Descent



Drawback: If the dataset has 1 million images, then it will take forever to train the model on such a big dataset

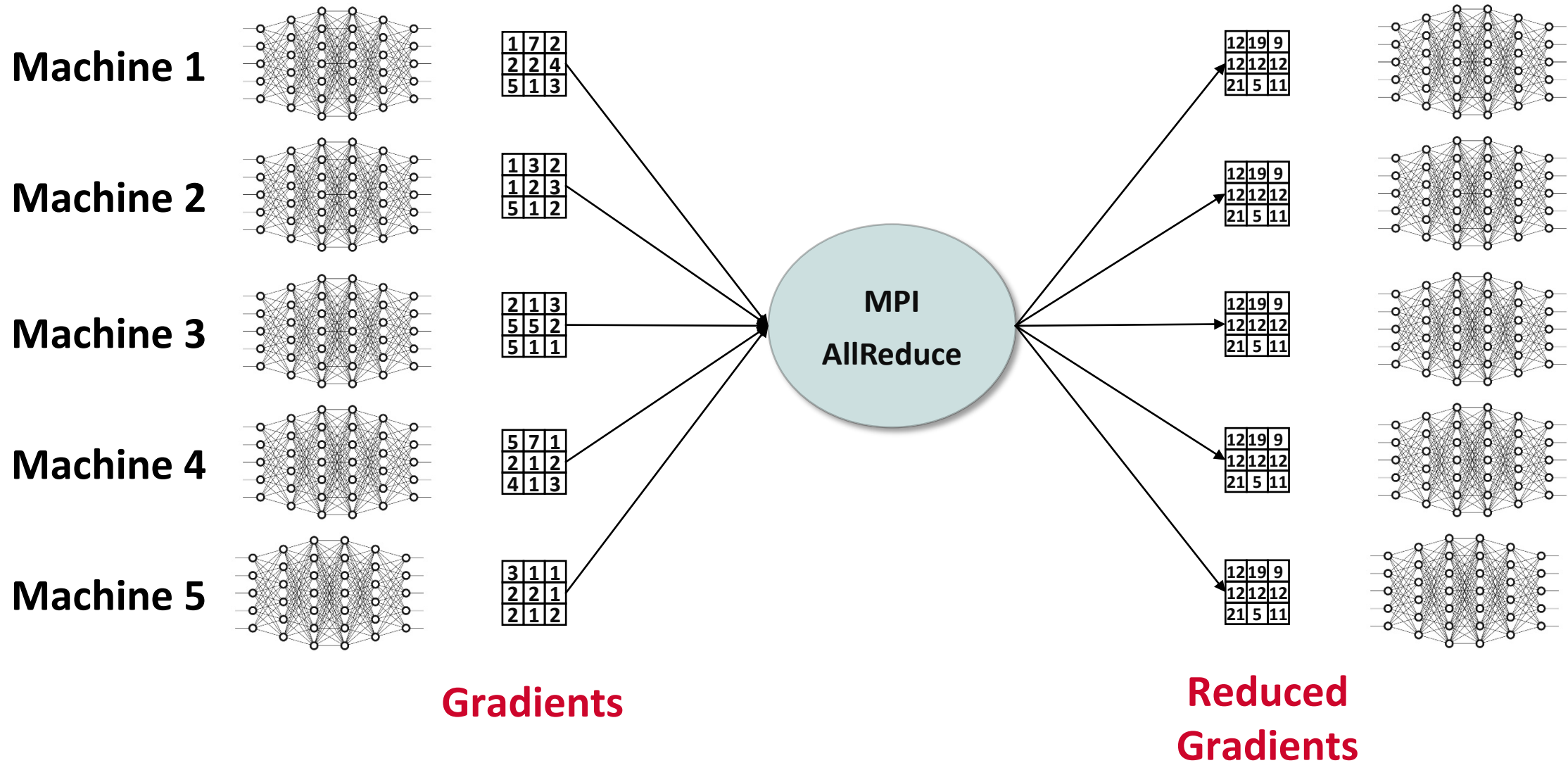
Solution: Can we use multiple machines to speedup the training of Deep learning models? (i.e. Utilize Supercomputers to Parallelize)

Need for Communication in Data Parallelism



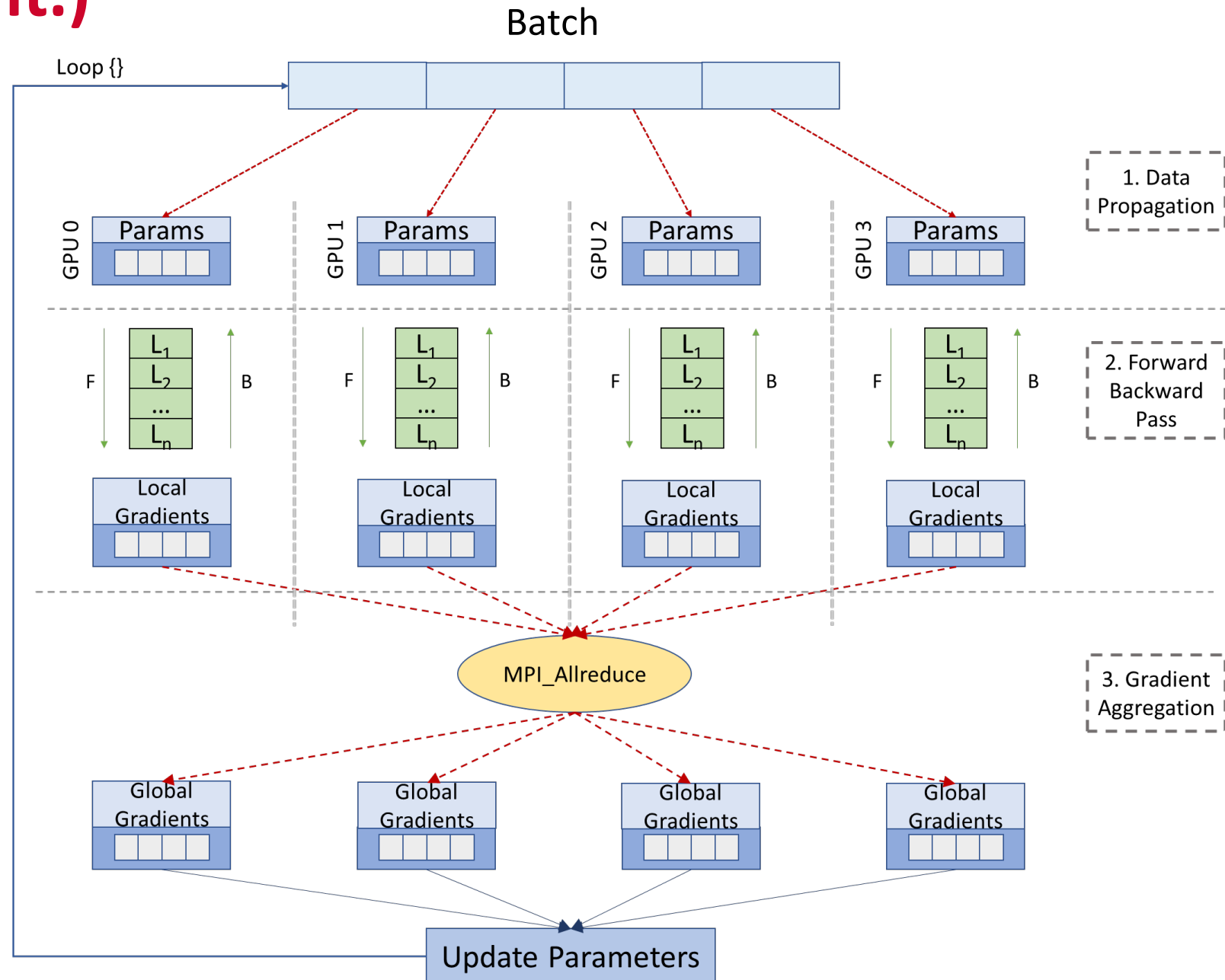
Problem: Train a single model on whole dataset,
not 5 models on different sets of dataset

Data Parallelism



Data Parallelism (Cont.)

- **Step1: Data Propagation**
 - Distribute the Data among GPUs
- **Step2: Forward Backward Pass**
 - Perform forward pass and calculate the prediction
 - Calculate Error by comparing prediction with actual output
 - Perform backward pass and calculate gradients
- **Step3: Gradient Aggregation**
 - Call MPI_Allreduce to reduce the local gradients
 - Update parameters locally using global gradients



Getting Set-up for the Hands-on Exercises

- You will run the experiments on the OSU RI2 cluster
- **Please use the account name and password from <http://go.osu.edu/dltutorial>**
- E.g. ssh ri2tut01@ri2.cse.ohio-state.edu and *tutorial01* as password
- Once on the shell, go to /opt/tutorials/dl-tutorial (copy/paste the following line)
cd /opt/tutorials/hoti-hpcai-tutorial/labs
- There is a folder for each lab (labs 1-2)
- Take a look at the README.md file for all scripts
 - **copy/paste the run commands from README.md and not the slide deck**

Lab 1 - DNN Training using PyTorch

- **Objectives**

- How to train a GPT-like model with PyTorch on a single GPU?
- How to perform distributed data-parallel training across multiple GPUs?
- How to use NCCL and MPI communication backends over InfiniBand?

- **Tasks**

- Task 1: PyTorch Single-GPU Training
- Task 2: PyTorch Distributed Training with MPI
- Task 3: PyTorch Distributed Training with NCCL

Distributed Training with PyTorch using NCCL and MPI

- **Model Summary**

- Architecture: GPT-2 124M configuration
- Transformer layers: 12
- Attention heads: 12
- Embedding dimension: 768
- Sequence length: 1,024 tokens
- Precision: FP16
- Optimizer: Fused AdamW
- Dataset: Synthetic token sequences

- **Environment Summary**

- Single-GPU run: 1 node, 1 NVIDIA V100 GPU
- Distributed runs: 2 nodes
- CUDA version: 12.6
- HBM Memory: 16 GB
- Framework: PyTorch DistributedDataParallel
- Job launcher: Slurm srun

Lab 1 – Task 1: GPT training on a single GPU

```
$ cd /opt/tutorials/hoti-hpcai-tutorial/lab1
$ srun -N 1 --reservation=dltutorial run_gpt_training_single.sh
```

V100

Warming up...

Benchmarking...

Iteration 1/5: 1,282.92 ms

Iteration 2/5: 1,286.13 ms

Iteration 3/5: 1,283.89 ms

Iteration 4/5: 1,285.03 ms

Iteration 5/5: 1,286.67 ms

RESULTS

```
GPUs : 1
Timing scope : rank 0
Measured iteration times : 1282.92 ms, 1286.13 ms, 1283.89 ms, 1285.03 ms, 1286.67 ms
Average iteration time : 1,284.93 ms
Throughput / GPU : 9,563.18 tokens/s
Aggregate throughput (1 GPU(s)) : 9,563.18 tokens/s
```

Lab 1 – Task 2: Two nodes with 1 GPU/node (MVAPICH-Plus)

```
$ srun -N 1 --reservation=dltutorial run_gpt_training_multi_MVP.sh
```

Warming up...

Benchmarking...

Iteration 1/5: 1,333.11 ms

Iteration 2/5: 1,331.28 ms

Iteration 3/5: 1,328.67 ms

Iteration 4/5: 1,331.46 ms

Iteration 5/5: 1,329.66 ms

V100

=====

RESULTS

=====

Communication backend : MPI

GPUs : 2

Timing scope : rank 0

Measured iteration times : 1333.11 ms, 1331.28 ms, 1328.67 ms, 1331.46 ms, 1329.66 ms

Average iteration time : 1,330.83 ms

Throughput / GPU : 9,233.31 tokens/s

Aggregate throughput (2 GPU(s)) : 18,466.62 tokens/s

=====

~1.9X on
2 GPUs

Lab 1 – Task 3: Two nodes with 1 GPU/node (NCCL)

```
$ srun -N 1 --reservation=dltutorial run_gpt_training_multi_NCCL.sh
```

Warming up...

Benchmarking...

Iteration 1/5: 1,330.97 ms

Iteration 2/5: 1,331.88 ms

Iteration 3/5: 1,331.40 ms

Iteration 4/5: 1,333.65 ms

Iteration 5/5: 1,331.10 ms

V100

=====

RESULTS

=====

Communication backend : NCCL

GPUs : 2

Timing scope : rank 0

Measured iteration times : 1330.97 ms, 1331.88 ms, 1331.40 ms, 1333.65 ms, 1331.10 ms

Average iteration time : 1,331.80 ms

Throughput / GPU : 9,226.61 tokens/s

Aggregate throughput (2 GPU(s)) : 18,453.22 tokens/s

=====

~1.9X on
2 GPUs

Hands-on Exercises: Key Takeaways from DL labs

- PyTorch DistributedDataParallel replicates the model across workers and synchronizes gradients during training.
- Single/Multiple GPU jobs -- similar
- Torch.distributed can be configured MPI, GLOO, NCCL, xCCL, etc.
- The experiments demonstrate that MVAPICH-Plus and NCCL can both deliver near-linear multi-node scaling for this workload.
- User guide to install the full HPC-AI stack:
<https://hpc-ai.engineering.osu.edu/userguide/>

More on the HPC-AI stack later!

Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- **Advanced Parallelism Strategies**
 - **Hybrid Parallelism Techniques**
 - **Case studies and solutions**
 - **Lab 2: Hands-on Exercises (Hybrid Parallelism)**
- Distributed Inference Systems
- Open Issues and Challenges
- Conclusion

Solutions and Case Studies: Exploiting HPC for DL

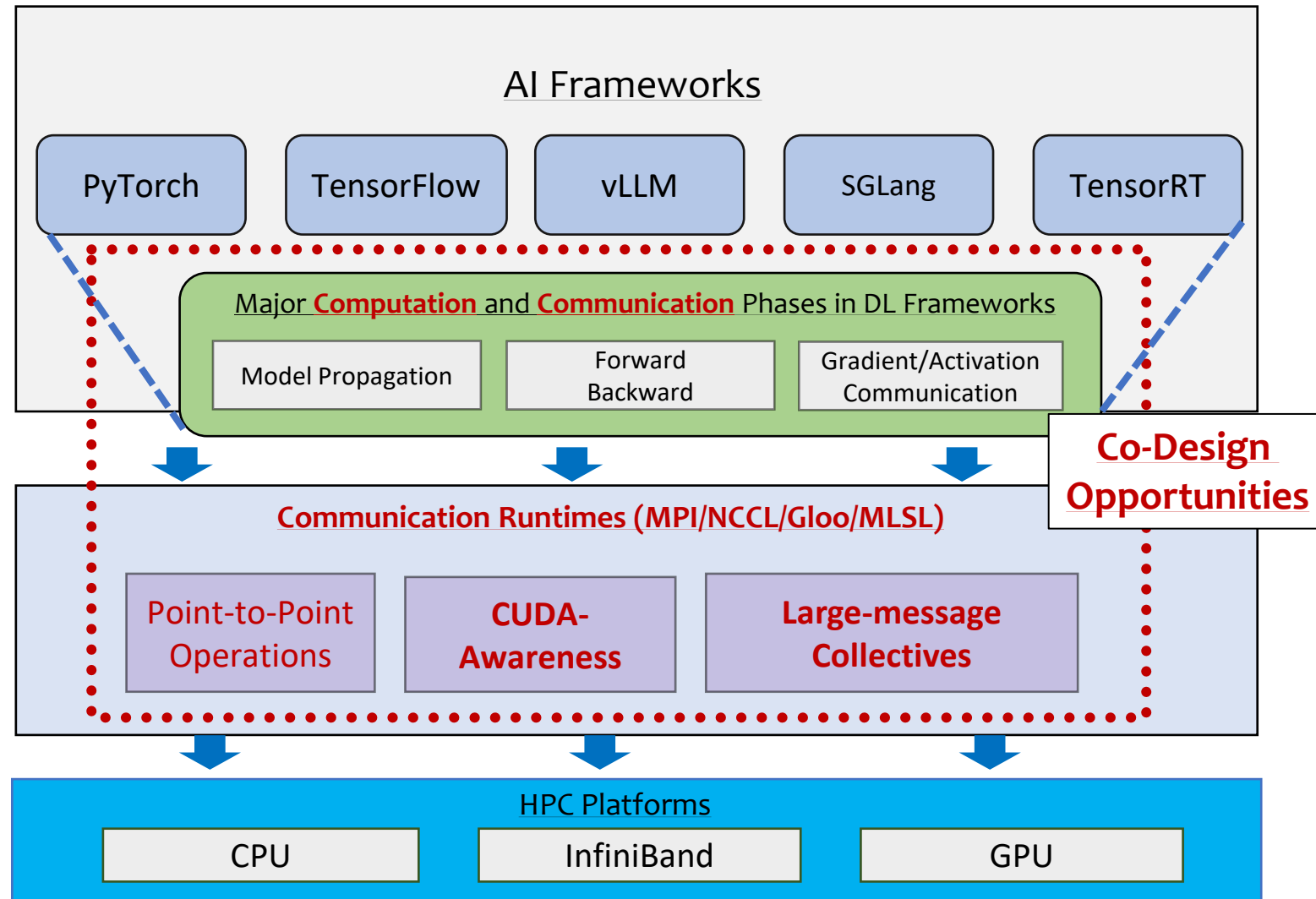
- Hybrid Parallelism

Techniques:

- ZeRO/FSDP
- Tensor Parallelism
- Pipeline Parallelism
- 3D Parallelism
- Sequence Parallelism
- Context Parallelism
- Expert Parallelism

- Case studies and solutions:

- HPC-AI Stack



DeepSpeed ZeRO

ZeRO 4-way data parallel training

Using:

- P_{os} (Optimizer state)
- P_g (Gradient)
- P_p (Parameters)

Courtesy: <https://www.microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/>

Memory Anatomy of a DNN (for ZeRO/FSDP)

Key question: What is the GPU memory M required to fit a model during training:

$$M_{Tot} = M_m + M_o + M_g + M_a$$

Where M_m is model memory, M_o optimizer memory, M_g gradient memory, and M_a activation memory

- p : Num model parameters
- k_l : Low precision B/param
- k_h : High precision B/param
- d : GPU Devices
- s : Sequence length
- b : Batch size
- h : Hidden size
- L : Transformer layers
- a : Num attention heads
- z : ZeRO stage

$$M_m = \begin{cases} \frac{k_l \cdot p}{d}, & z = 3 \\ k_l \cdot p, & \text{else} \end{cases}$$

$$M_o = \begin{cases} \frac{(3k_h) \cdot p}{d}, & z \geq 1 \\ (3k_h) \cdot p, & \text{else} \end{cases}$$

$$M_g \leq \begin{cases} \frac{(k_l \text{ or } k_h) \cdot p}{d}, & z \geq 2 \\ (k_l \text{ or } k_h) \cdot p, & \text{else} \end{cases}$$

$$M_a \approx sbhL([16k_l + 2] + [2k_l + 1] \frac{a \cdot s}{h})$$

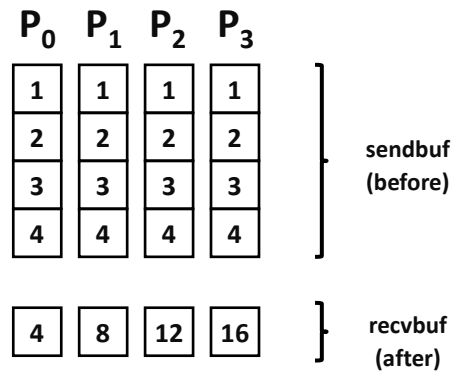
DeepSpeed ZeRO

- Instead of being limited by the **device** memory, we are now limited by the **aggregate** memory
- E.g. You want to train a trillion-parameter model on 1024 GPUs with 16 GB memory each
 - With 16-bit precision, model+optimizer = ~16 TB of memory
 - **We can fit this into 1024 GPUs with ZeRO:** $\frac{16 \text{ TB}}{1024 \text{ GPUs}} = 16 \frac{\text{GB}}{\text{GPU}}$
- ZeRO-Infinity introduces offload to CPU memory or NVMe disk for the truly desperate
- Since ZeRO removes the DP memory limit, do we still need MP?
 - There are still models and data samples (e.g. pathology, astronomy, etc) that don't fit inside GPU memory ***even with ZeRO***
 - We can use pipeline + tensor parallelism along with ZeRO for these cases (called 3D-parallel, more on this later!)

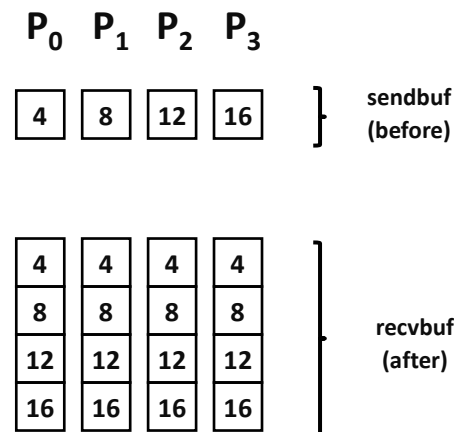
DeepSpeed ZeRO Communication

- Remember that ZeRO-1/2 have the same communication volume as distributed data parallelism (DDP), and that ZeRO-3 requires 50% more communication volume? Time to understand that!
- Communication volume of DDP on P processes on a tensor with N elements is that of AllReduce: $2N * (P-1)/P$
 - Assumes that AllReduce is implemented as a bandwidth-optimal ring
 - For practical purposes, P is usually large enough so that the communication volume is $2N$
 - This doesn't mean that AllReduce runtime is the same regardless of P !
 - The bandwidth to send this communication volume $2N$ will be much lower for large P
 - AllReduce is itself composed of a ReduceScatter + AllGather operations:
 - ReduceScatter and AllGather individual communication volume: N

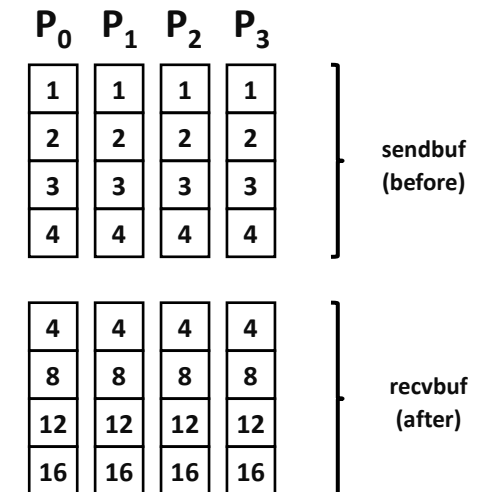
ReduceScatter



AllGather

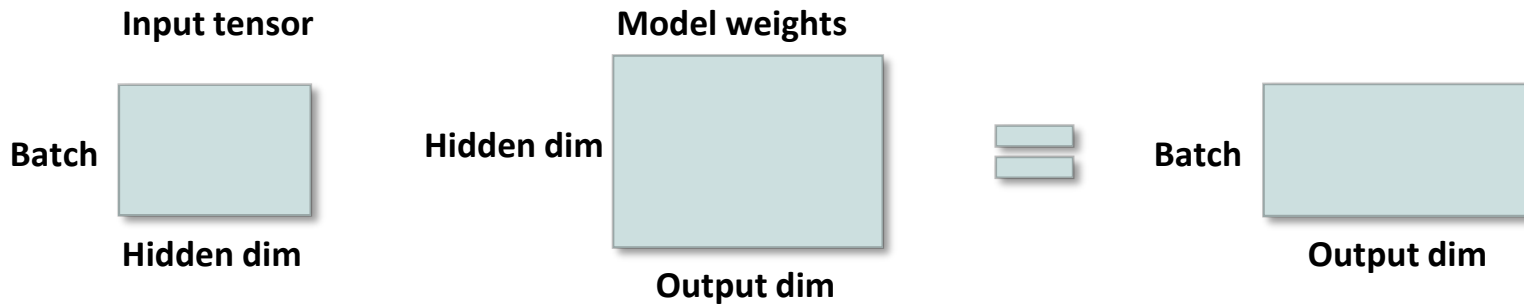


AllReduce

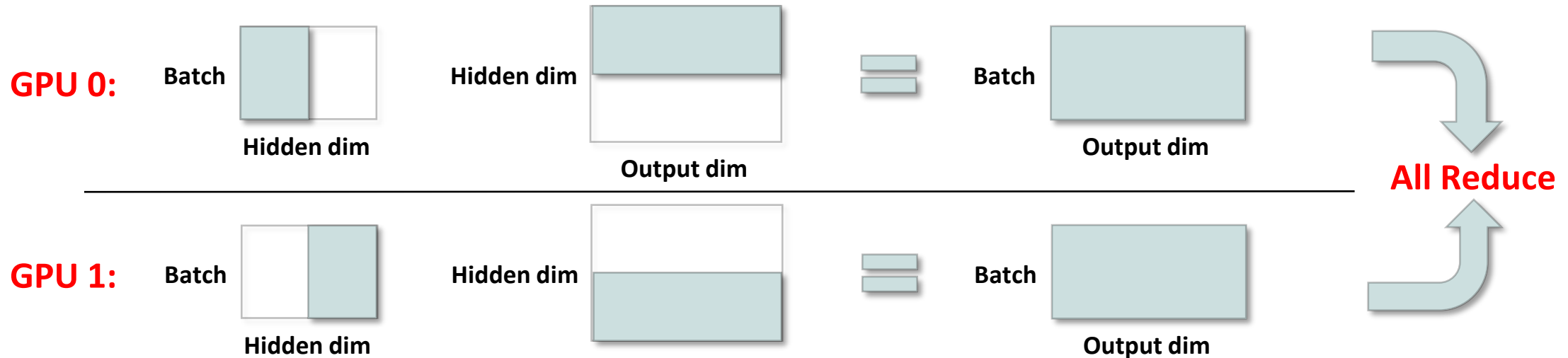


Tensor Parallelism

- LLM models consist of matrix multiplications.

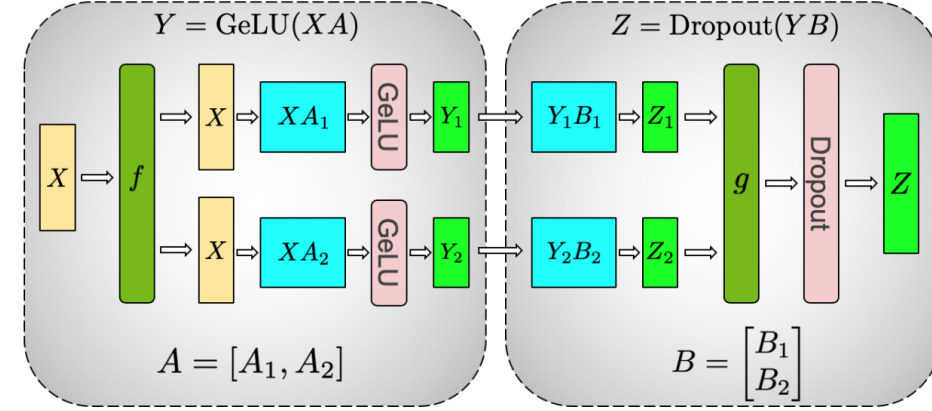


- Tensor Parallelism splits along hidden dim, and distributes the computation to multiple GPUs.

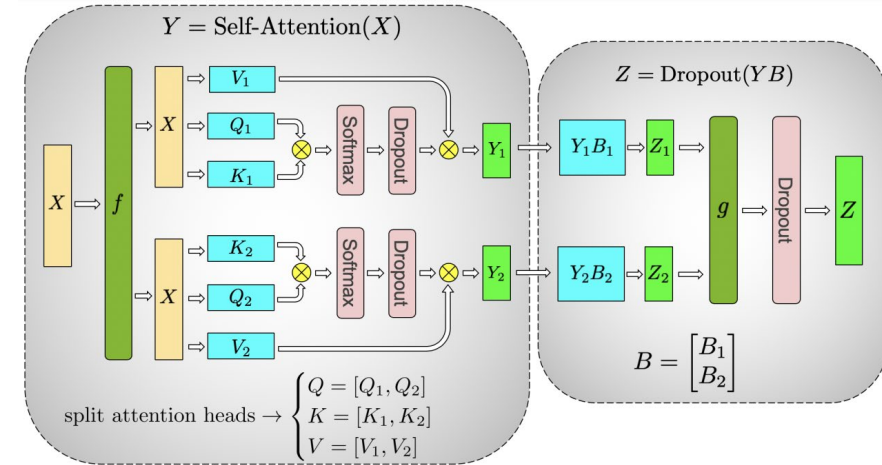


Tensor Parallelism for Transformers

- A transformer layer consists of a self attention block followed by a two-layer, multi-layer perceptron (MLP)
- MLP TP:** Split the initial GEMM+GeLU column-wise, then split the following GEMM+Dropout row-wise
- Self-attention TP:** Split the QKV column-wise, then split the GEMM+Dropout row-wise



(a) MLP



(b) Self-Attention

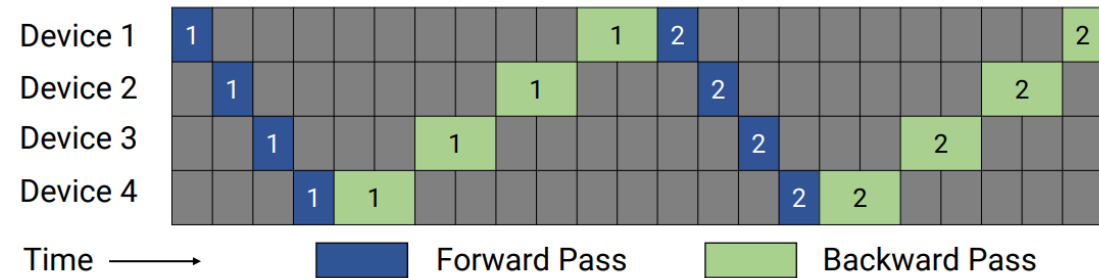
$$f = \begin{cases} I_{FP} \\ Allreduce_{BP} \end{cases} \quad g = \begin{cases} Allreduce_{FP} \\ I_{BP} \end{cases}$$

Courtesy: <https://arxiv.org/pdf/1909.08053.pdf>

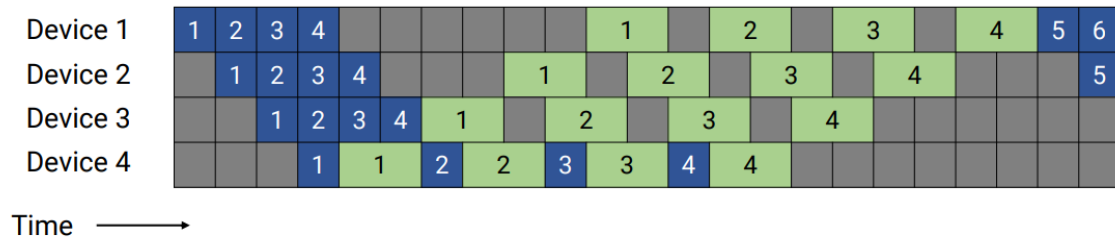
Pipeline Parallelism

- Split layers across devices and stream microbatches → overlap compute across stages (improves utilization vs. naive pipeline).
- Trade-offs (by schedule):
 - Naive: large pipeline “bubbles” → low utilization
 - PipeDream: better overlap but weight staleness
 - Megatron (1F1B): near-steady utilization with controlled memory
- Costs & constraints:
 - Inter-stage communication (point-to-point) + load imbalance can bottleneck
 - Requires microbatching → higher activation memory / tuning complexity

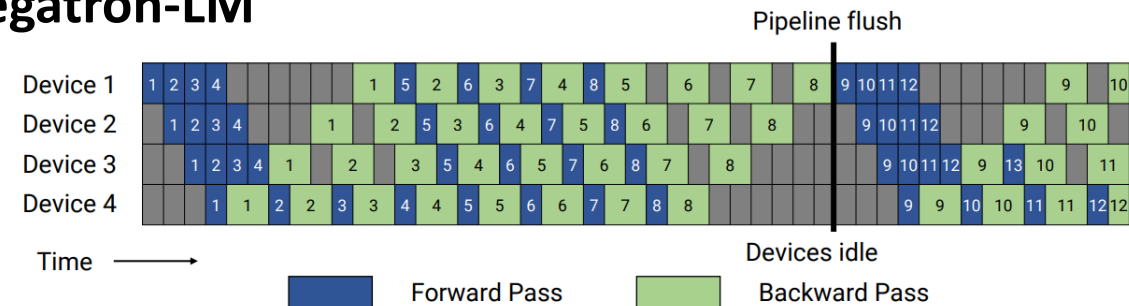
Naïve Implementation of inter-layer (pipeline) parallelism



Pipedream

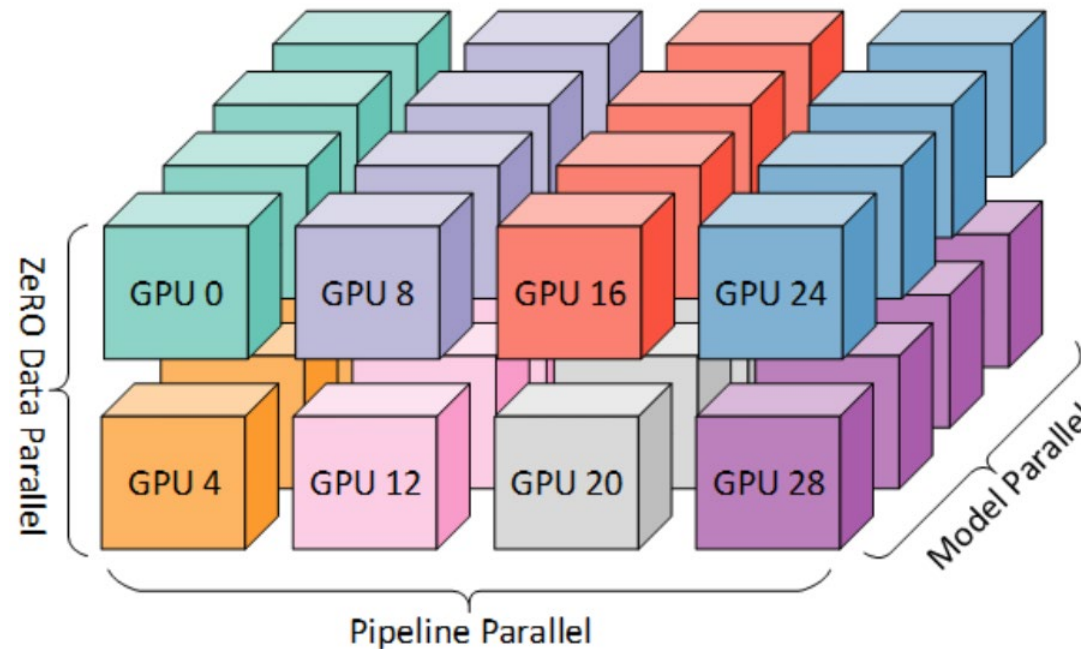


Megatron-LM



3D Parallelism

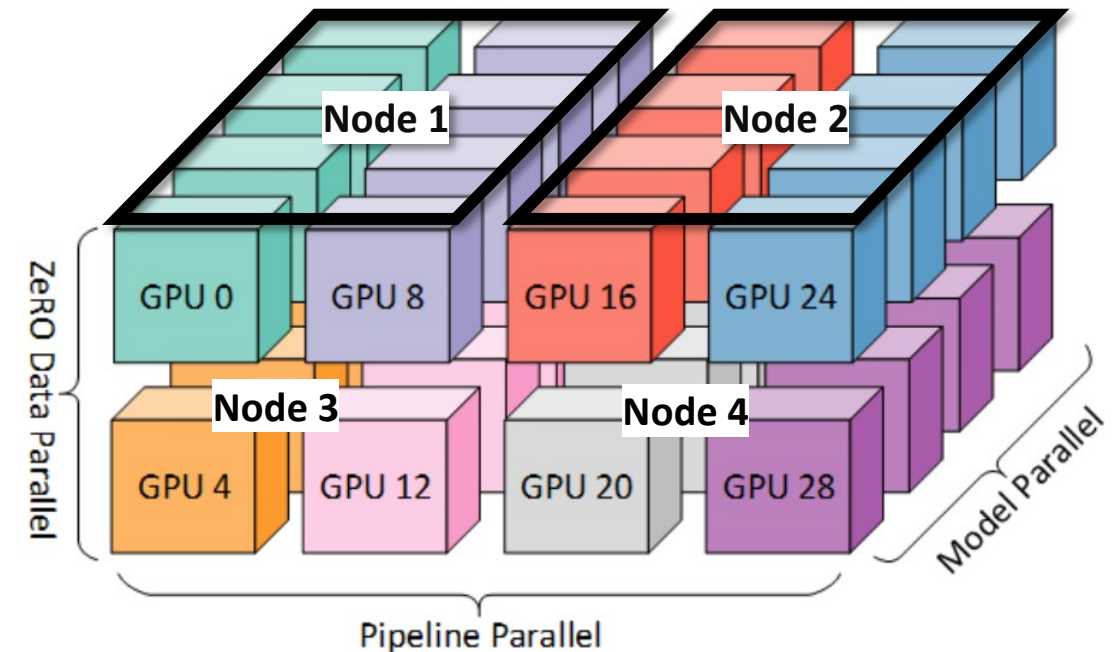
- Combine PP with TP and DP for 3D parallelism. For example:
 - Split given layer(s) via TP across 4 GPUs
 - Split the model into 4 pipeline stages
 - The above TP+PP combination compose a single DP unit
 - Use 2 DP units with the above configuration for 32-GPU parallelism
- **Question:** Given that each node contains 8 GPUs, where should you place the node boundaries?



Credit: <https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/>

3D Parallelism

- Combine PP with TP and DP for 3D parallelism. For example:
 - Split given layer(s) via TP across 4 GPUs
 - Split the model into 4 pipeline stages
 - The above TP+PP combination compose a single DP unit
 - Use 2 DP units with the above configuration for 32-GPU parallelism
- **Question:** Given 4 nodes with 8 GPUs each, where should you place the node boundaries?
- **Answer:** Keep as many TP partitions as possible within a node
 - Each model replica requires $TP \cdot PP = 16$ GPUs
 - Two pipeline stages per node
 - Pipeline-parallel pt2pt comms across nodes, no inter-node TP
 - Between pipeline stages 2 and 3 out of the total 4
 - ZeRO-1 across nodes as well, but same comms volume as DP and easy to overlap with compute



Sequence Parallelism

- Tensor parallelism left the LayerNorm and Dropout blocks intact and as a result, they are replicated across the tensor parallel group.
- These elements do not require a lot of compute but demand a considerable amount of activation memory
- **Key Idea:** Since LayerNorm and Dropout are independent along the input sequence s , partition them across GPUs
- Reduces the memory overhead, but introduces more communication overhead

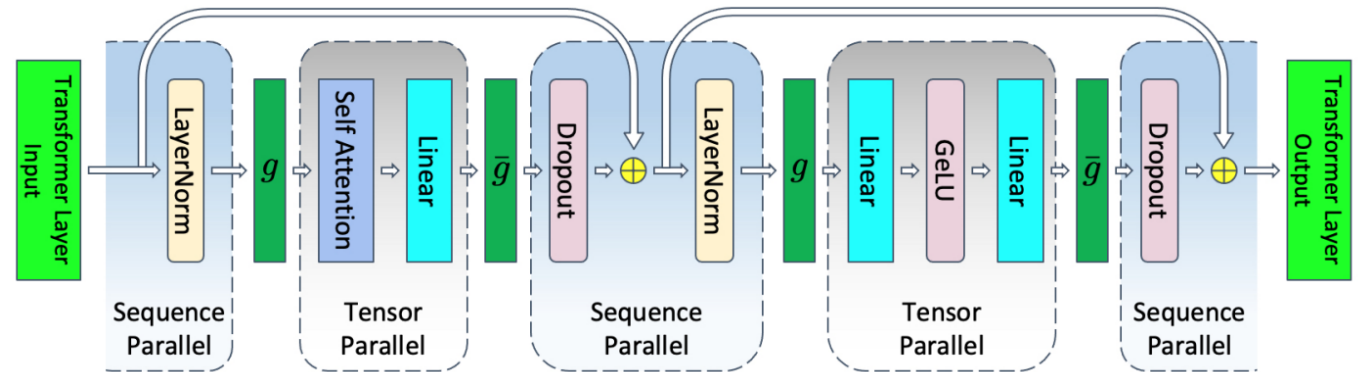
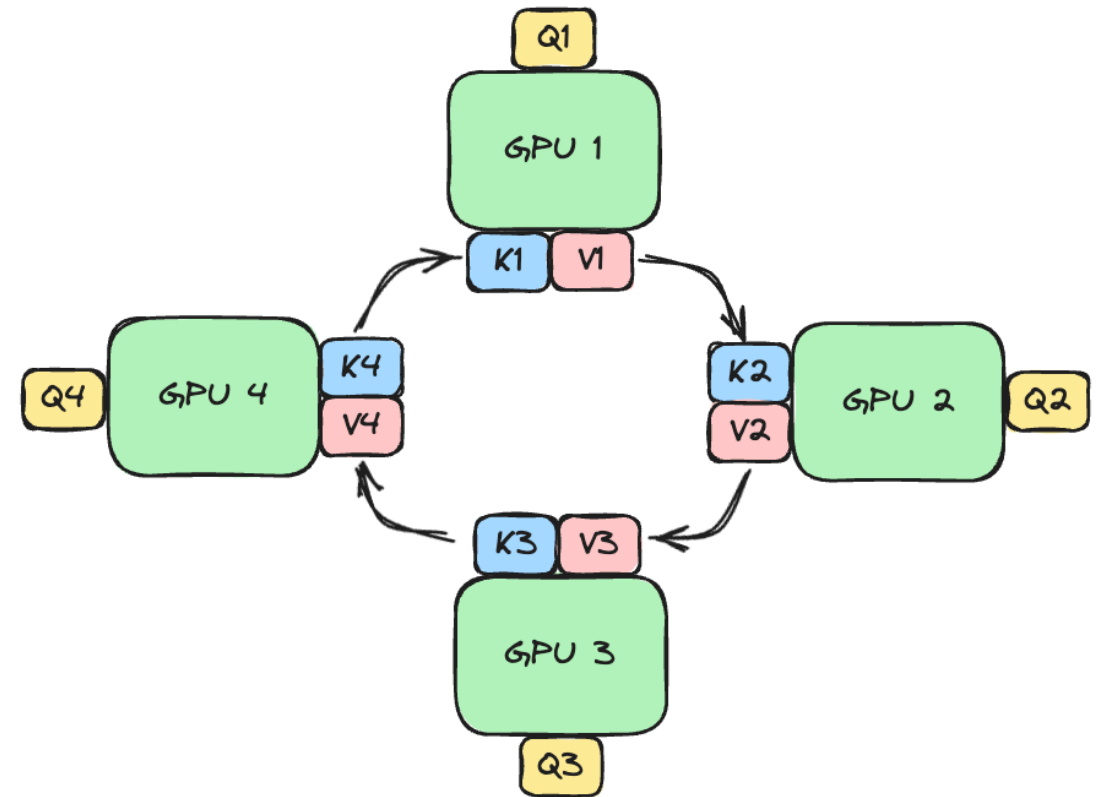


Figure 5: Transformer layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in the forward pass and reduce-scatter in the backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

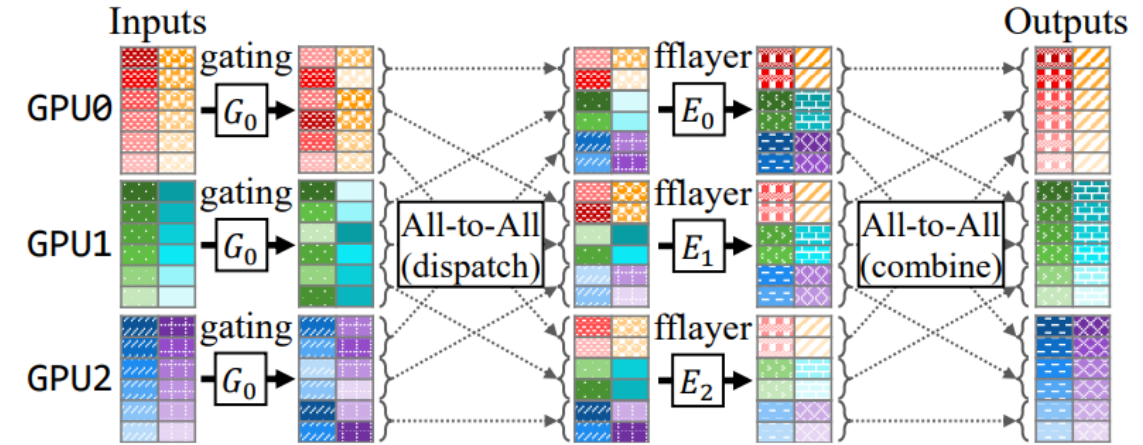
Context Parallelism with Ring Attention

- Key idea for Ring Attention:
 - Split the Q, K, V matrices equally across GPUs
 - Rotate the K and V blocks between GPUs in a ring
 - Keep the Q blocks persistent on the same GPU
 - Overlap the K, V block communication with computation
 - Keep rotating K and V until every GPU has seen every block



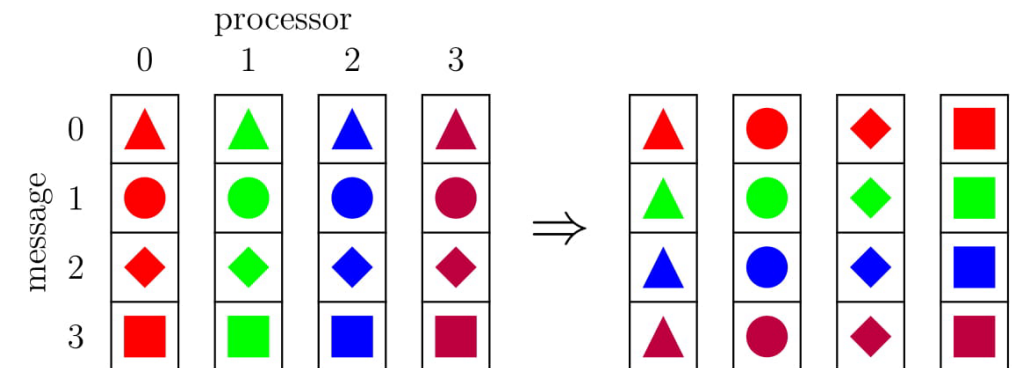
Expert Parallelism

- Assume top- k along the *experts* axis
- Experts are distributed across GPUs
 - Causes load-imbalance issues because some GPUs are doing more/less work
 - Two expensive AlltoAll communication operations are required in the forward pass (one to communicate tokens to experts, one to communicate expert outputs), and two in the backward pass



Credit: <https://arxiv.org/pdf/2206.03382.pdf>

- AlltoAll is an expensive communication operation!
 - For p messages of size m each, communication volume of at least $m(n - 1)$
 - Essentially a matrix transpose across $(\text{tensor size}) \times (\# \text{processes})$
 - Hard to overlap
- Most MoE optimization is along optimizing these AlltoAll operations!



Credit: [https://en.wikipedia.org/wiki/All-to-all_\(parallel_pattern\)](https://en.wikipedia.org/wiki/All-to-all_(parallel_pattern))

Summary of Communication in AI Parallelism Strategies

Parallelism Strategy	Description	Main Communication Operation(s)
Data Parallelism (DP)	Replicates the full model and splits the batch across devices.	AllReduce
FSDP / ZeRO	Shards parameters, gradients, and optimizer states.	AllGather + ReduceScatter
Tensor Parallelism (TP)	Splits individual layer/tensor computations across devices.	AllReduce
Pipeline Parallelism (PP)	Splits consecutive model layers into stages across devices.	Point-to-Point Send/Recv
3D Parallelism	Combines data, tensor, and pipeline parallelism simultaneously.	AllReduce/AllGather/ReduceScatter/P2P
Sequence Parallelism (SP)	Splits sequence-dependent activations/computation.	AllGather and ReduceScatter
Context Parallelism (CP)	Splits long-context tokens across devices.	P2P/Ring or AllGather
Expert Parallelism (EP)	Distributes MoE experts across devices and routes tokens.	All-to-All or All-to-Allv

Summary of major parallelism strategies and their associated communication operations

Lab 2 – Out-of-core DNN Training using DeepSpeed

- Objectives
 - Test an out-of-core DNN on a single node
 - Train the out-of-core DNN on two node using DeepSpeed
- Tasks
 - Task 1: Single GPU
 - Task 2: Multi-GPU

Lab 2 – Task 1: Test a 2.5B Bert DNN on a single GPU

```
$ cd /opt/tutorials/hoti-tutorial/labs/lab3
```

```
$ srun -N 1 --reservation=dltutorial test_bert.sh
```

```
+ /opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed//bin/python  
/home/opt/tutorials/DeepSpeed/DeepSpeedExamples/HelloDeepSpeed/train_bert.py --checkpoint_dir /home/jain.575/checks --  
num_layers 192 --ff_dim 4096 --h_dim 1024 --local_rank 0 --batch_size 1 --num_iterations 10
```

```
Traceback (most recent call last):
```

```
File "/home/opt/tutorials/DeepSpeed/DeepSpeedExamples/HelloDeepSpeed/train_bert.py", line 789, in <module>
```

```
    fire.Fire(train)
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/fire/core.py", line 141, in Fire
```

```
    component_trace = _Fire(component, args, parsed_flag_args, context, name)
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/fire/core.py", line 466, in _Fire
```

```
    component, remaining_args = _CallAndUpdateTrace(
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/fire/core.py", line 681, in _CallAndUpdateTrace
```

```
    component = fn(*varargs, **kwargs)
```

```
File "/home/opt/tutorials/DeepSpeed/DeepSpeedExamples/HelloDeepSpeed/train_bert.py", line 757, in train
```

```
    optimizer.step()
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/torch/optim/optimizer.py", line 140, in wrapper
```

```
    out = func(*args, **kwargs)
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/torch/optim/optimizer.py", line 23, in _use_grad
```

```
    ret = func(self, *args, **kwargs)
```

```
File "/opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed/lib/python3.8/site-packages/torch/optim/adam.py", line 220, in step
```

```
    state['exp_avg_sq'] = torch.zeros_like(p, memory_format=torch.preserve_format)
```

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 16.00 MiB (GPU 0; 31.75 GiB total capacity; 30.68 GiB already allocated; 3.50 MiB  
free; 30.82 GiB reserved in total by PyTorch) If reserved memory is >> allocated memory try setting max_split_size_mb to avoid fragmentation. See  
documentation for Memory Management and PYTORCH_CUDA_ALLOC_CONF
```

Lab 2 – Task 2: Run 2.5B Bert DNN on two GPUs

```
$ cd /opt/tutorials/hoti-tutorial/labs/lab3
```

```
$ srun -N 2 --reservation=dltutorial run_bert.sh
```

```
+ /opt/tutorials/DeepSpeed/miniconda3/envs/deepspeed//bin/deepspeed -H /home/jain.575/hosts17108  
/home/opt/tutorials/DeepSpeed/DeepSpeedExamples/HelloDeepSpeed/train_bert_ds.py --checkpoint_dir /home/jain.575/checks --  
num_layers 192 --ff_dim 4096 --h_dim 1024 --batch_size 1 --num_iterations 10
```

```
gpu01: [2022-12-31 22:50:20,415] [INFO] [timer.py:197:stop] 0/4, RunningAvgSamplesPerSec=1.989045629615473, CurrSamplesPerSec=1.9197348998894654,  
MemAllocated=18.42GB, MaxMemAllocated=25.43GB  
gpu01: [2022-12-31 22:50:21,470] [INFO] [stage_1_and_2.py:1765:step] [deepspeed] OVERFLOW! Rank 0 Skipping step. Attempted loss scale: 268435456.0,  
reducing to 134217728.0  
gpu01: [2022-12-31 22:50:21,472] [INFO] [timer.py:197:stop] 0/5, RunningAvgSamplesPerSec=1.9581760061302556, CurrSamplesPerSec=1.8992247658347254,  
MemAllocated=18.42GB, MaxMemAllocated=25.44GB  
gpu01: [2022-12-31 22:50:22,518] [INFO] [stage_1_and_2.py:1765:step] [deepspeed] OVERFLOW! Rank 0 Skipping step. Attempted loss scale: 134217728.0,  
reducing to 67108864.0  
gpu01: [2022-12-31 22:50:22,520] [INFO] [timer.py:197:stop] 0/6, RunningAvgSamplesPerSec=1.9472242517233995, CurrSamplesPerSec=1.9150918757408228,  
MemAllocated=18.42GB, MaxMemAllocated=25.44GB  
gpu01: [2022-12-31 22:50:23,557] [INFO] [stage_1_and_2.py:1765:step] [deepspeed] OVERFLOW! Rank 0 Skipping step. Attempted loss scale: 67108864.0,  
reducing to 33554432.0  
gpu01: [2022-12-31 22:50:23,558] [INFO] [timer.py:197:stop] 0/7, RunningAvgSamplesPerSec=1.9440531488041186, CurrSamplesPerSec=1.9314713530693848,  
MemAllocated=18.42GB, MaxMemAllocated=25.44GB  
gpu01: [2022-12-31 22:50:24,577] [INFO] [stage_1_and_2.py:1765:step] [deepspeed] OVERFLOW! Rank 0 Skipping step. Attempted loss scale: 33554432.0,  
reducing to 16777216.0  
gpu01: [2022-12-31 22:50:24,579] [INFO] [timer.py:197:stop] 0/8, RunningAvgSamplesPerSec=1.9473977612894298, CurrSamplesPerSec=1.9642949469669437,  
MemAllocated=18.42GB, MaxMemAllocated=25.44GB  
gpu01: [2022-12-31 22:50:25,644] [INFO] [stage_1_and_2.py:1765:step] [deepspeed] OVERFLOW! Rank 0 Skipping step. Attempted loss scale: 16777216.0,  
reducing to 8388608.0  
gpu01: [2022-12-31 22:50:25,645] [INFO] [timer.py:197:stop] 0/9, RunningAvgSamplesPerSec=1.9380716205436017, CurrSamplesPerSec=1.883938232505326,  
MemAllocated=18.42GB, MaxMemAllocated=25.44GB
```

Solutions and Case Studies: Exploiting HPC for DL

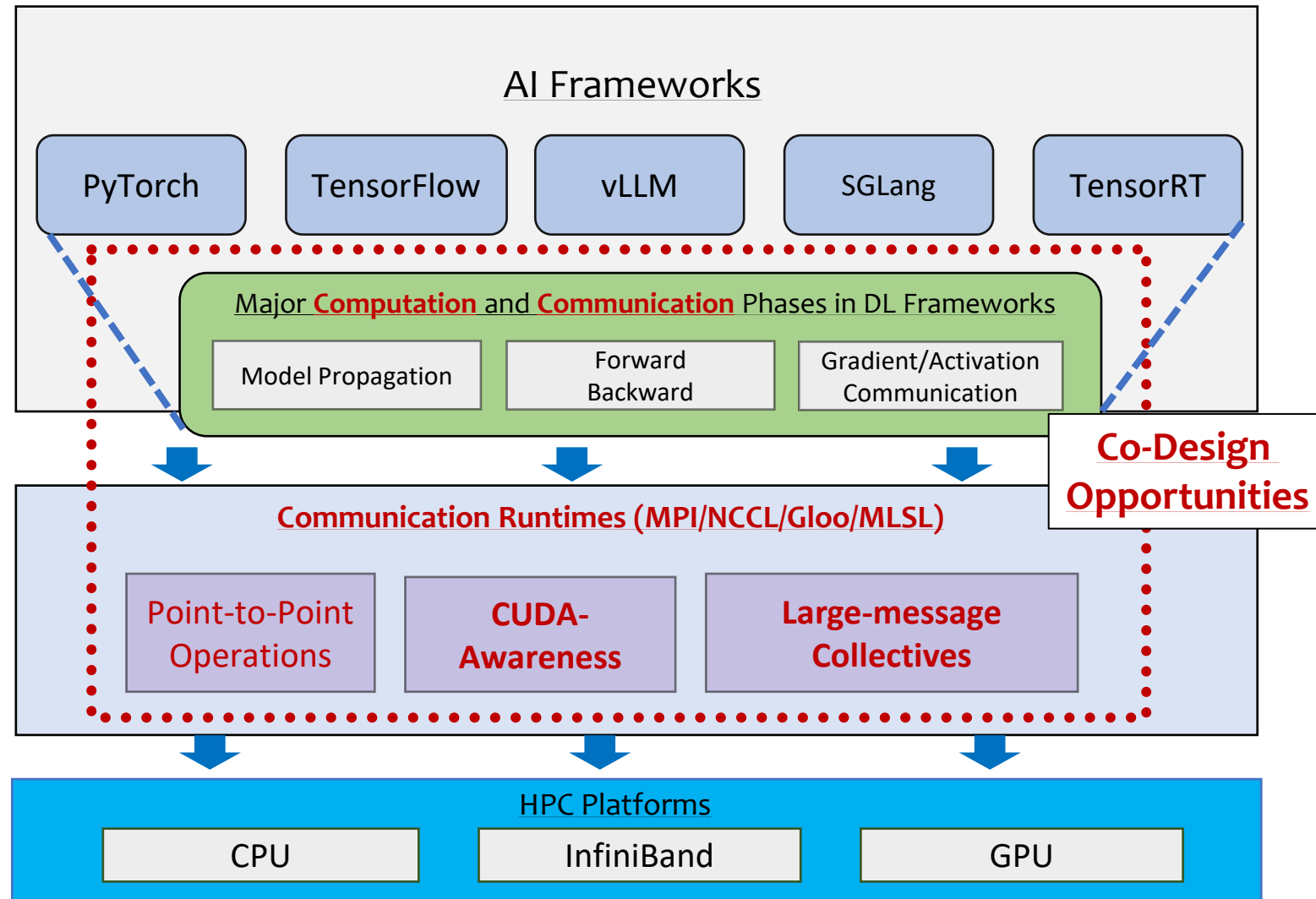
- Hybrid Parallelism

Techniques:

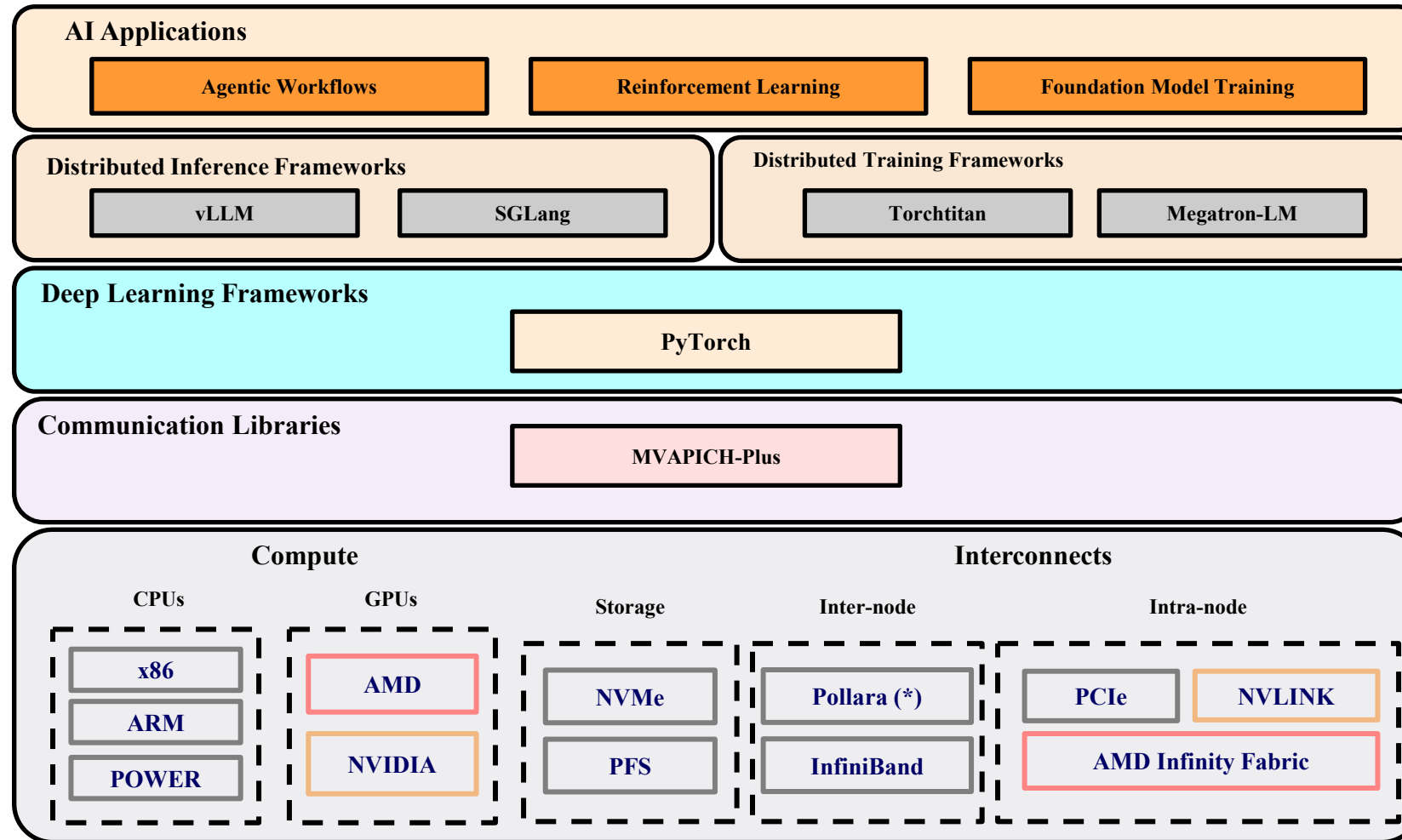
- ZeRO/FSDP
- Tensor Parallelism
- Pipeline Parallelism
- 3D Parallelism
- Sequence Parallelism
- Context Parallelism
- Expert Parallelism

- Case studies and solutions:

- HPC-AI Stack



HPC-Accelerated Vendor Neutral AI Stack with MPI Backend



V1.0 is available from: <https://github.com/OSU-Nowlab/osu-hidl> ^{* Upcoming}

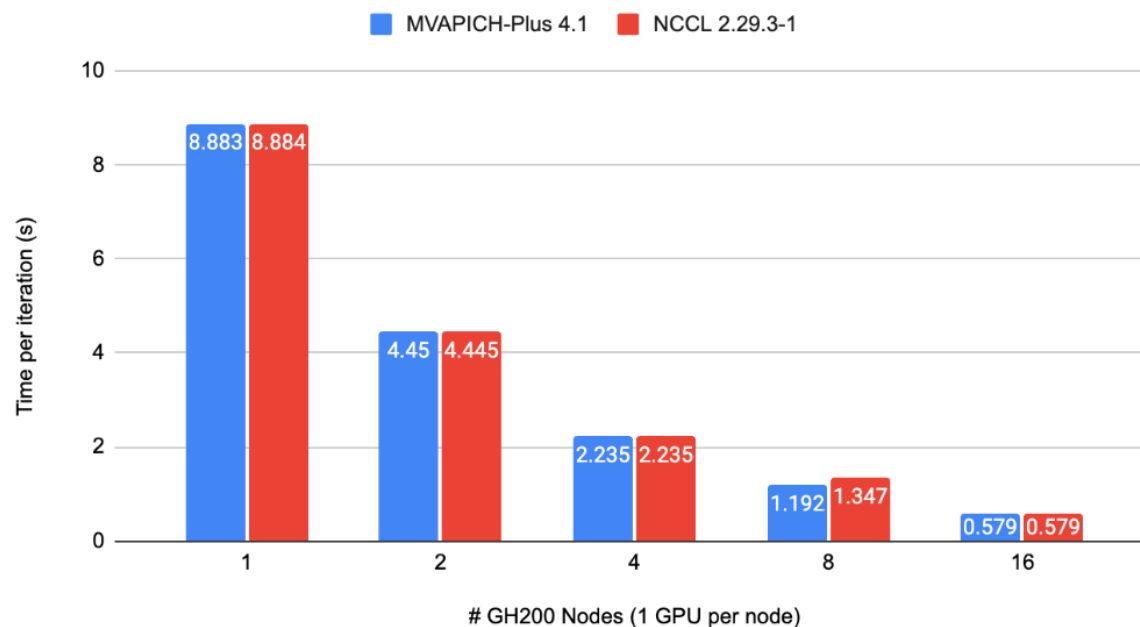
HPC-AI V1.0 Release

- Full-stack integration of Training and Inference Frameworks: PyTorch, DeepSpeed, vLLM, and SGLang with MVAPICH-Plus
- Native PyTorch Distributed Data Parallel (DDP) training with MPI backend
- Advanced decoding method (MAC-Attention) and communication runtime (MCR-DL)
- Efficient large-message collectives (e.g., Allreduce) on various CPUs and GPUs
- GPU-Direct Ring and Two-level multi-leader algorithms for Allreduce operations
- Support for fork safety in distributed training and inference environments
- Exploits efficient large message collectives in MVAPICH-Plus 4.1 and later Open-source framework builds with advanced MPI backend support
- Vendor-neutral stack with competitive performance to GPU-based collective libraries (e.g., NCCL, RCCL)
- Battle tested on modern HPC clusters (e.g., OLCF Frontier, TACC Vista, SDSC Cosmos) with up-to-date accelerator generations (e.g., AMD, NVIDIA)
- Compatible with
 - InfiniBand Networks: Mellanox InfiniBand adapters (EDR, FDR, HDR, NDR)
 - Slingshot Networks: HPE Slingshot
 - GPU&CPU Support:
 - NVIDIA GPU A100, H100, GH200
 - AMD MI250X, MI300A GPUs
 - Software Stack:
 - CUDA [12.x] and Latest CuDNN
 - (NEW)ROCm [7.x]
 - (NEW)PyTorch [2.10.0]
 - (NEW)Training & Inference: DeepSpeed, vLLM, SGLang
 - (NEW)Advanced: MAC-Attention, MCR-DL
 - (NEW)Python [3.x]

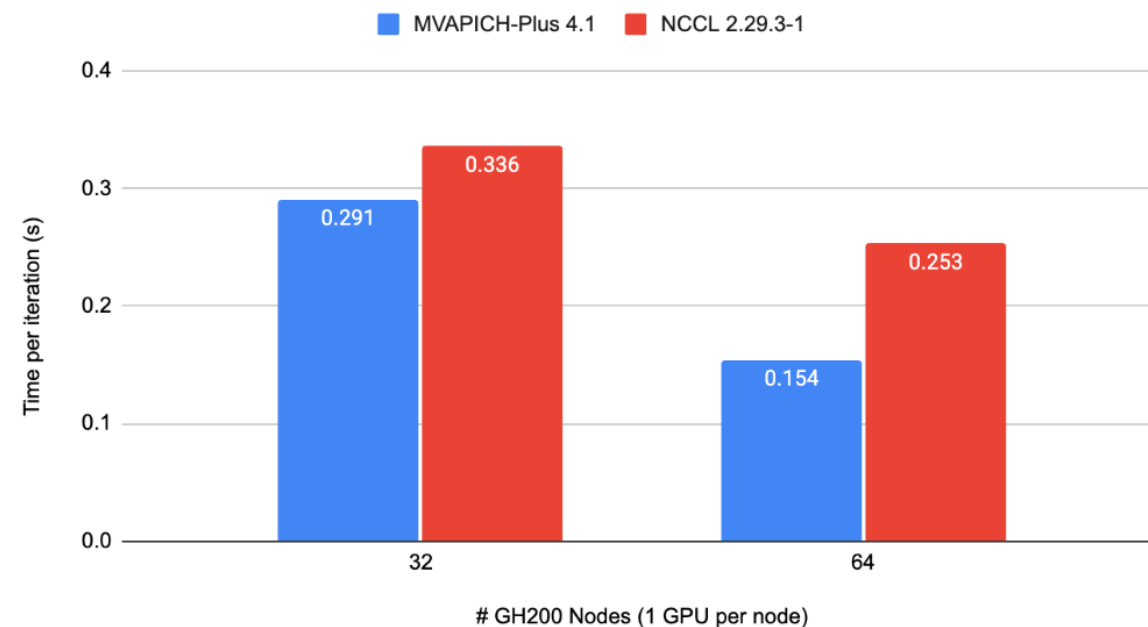
For more details: <https://hpc-ai.engineering.osu.edu/>

Distributed Data Parallel Training on H200 (Vista)

MVAPICH-Plus 4.1 vs NCCL 2.29.3-1; Batch Size: 12; Block Size: 512



MVAPICH-Plus 4.1 vs NCCL 2.29.3-1; Batch Size: 12; Block Size: 512



- **1.64x speedup**(compared to NCCL 2.29.3-1) on 64 GPUs

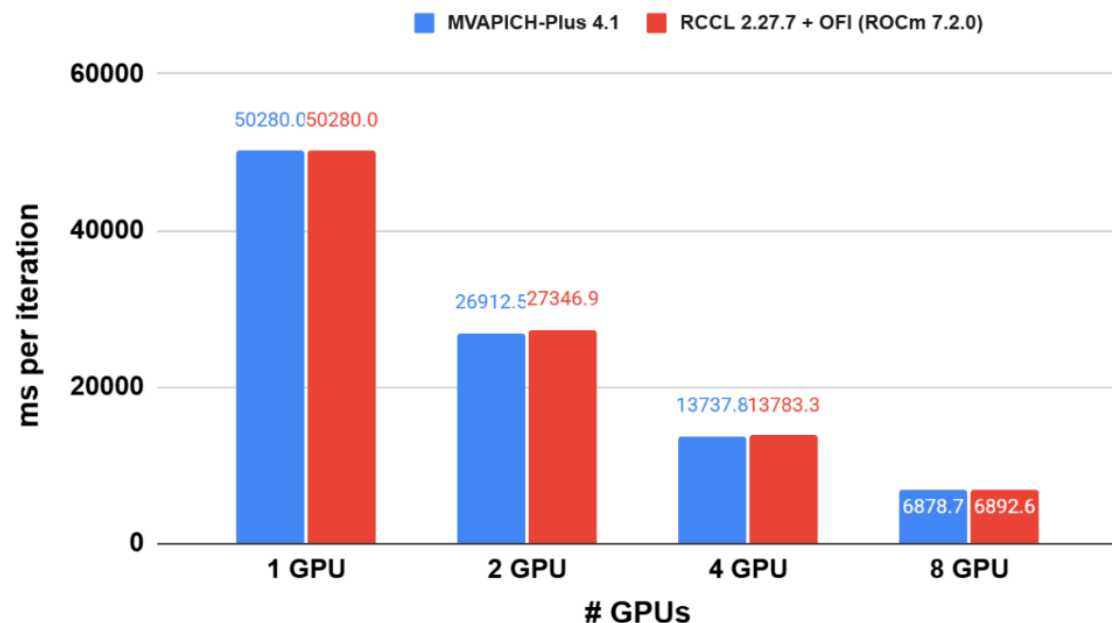
Machine Specifications: TACC Vista

CPU Model	CPU Core Info	Memory	GPU Model	GPU Memory	Interconnects
NVIDIA Grace CPU	1x72@3.1 GHz	116 GB DDR5	NVIDIA H200 GPU (1/Node)	96 GB HBM 3	Mellanox NDR (400 Gb/s)

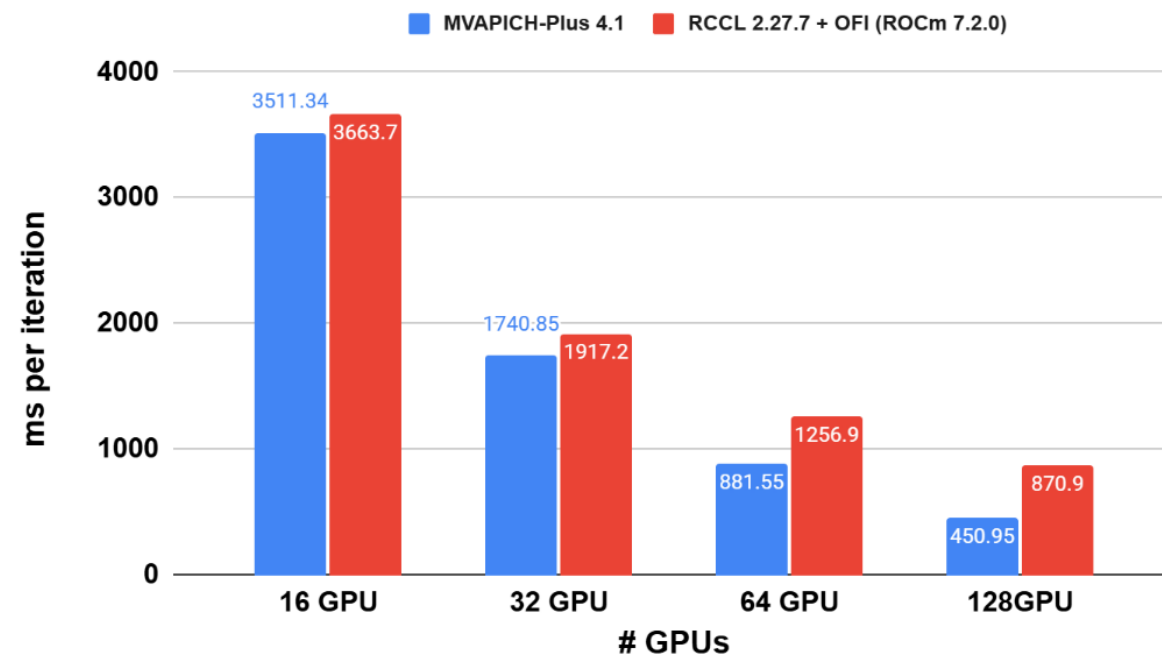
Model	Batch Size	Block Size	Gradient Accumulation Steps	Benchmark	Dataset	DL Framework
GPT-2	12	512	256	NanoGPT	OpenWebText	PyTorch 2.10.0

Distributed Data Parallel Training on MI250X (Frontier)

GPT-2 DDP 1.5 Million Token per Iter



GPT-2 DDP 1.5 Million Token per Iter



- **1.9x speedup**(compared to RCCL 2.27.7 + OFI) on 128 GPUs

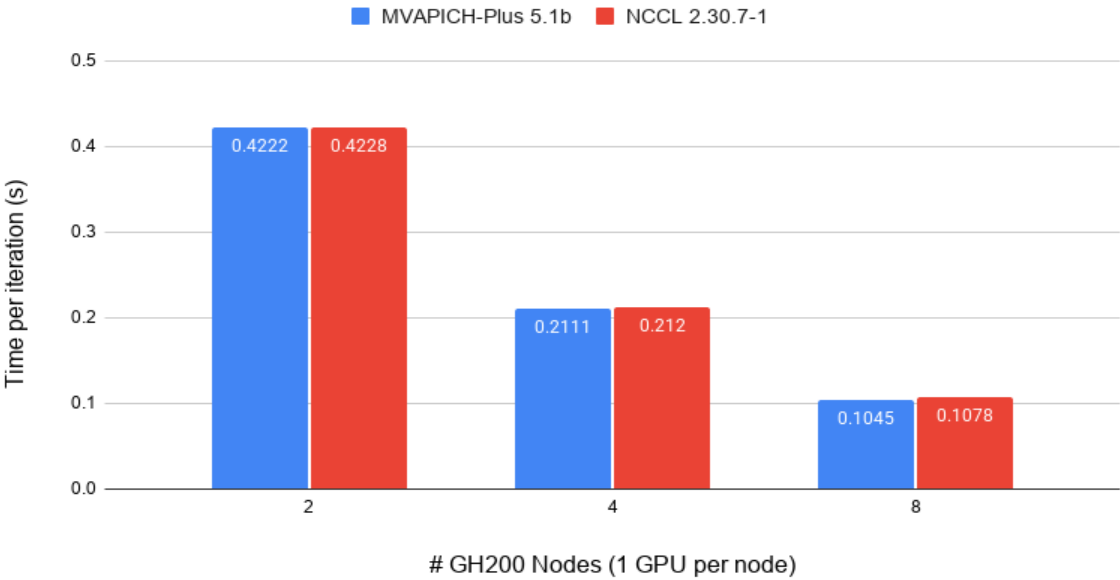
Machine Specifications: OLCF Frontier

CPU Model	CPU Core Info	Memory	GPU Model	GPU Memory	Interconnects
AMD EPYC 7A53 CPU	1x64@2GHz	512 GB DDR4	AMD MI250X (4/Node)	128 GB HBM 2e	HPE Slingshot (200 Gb/s)

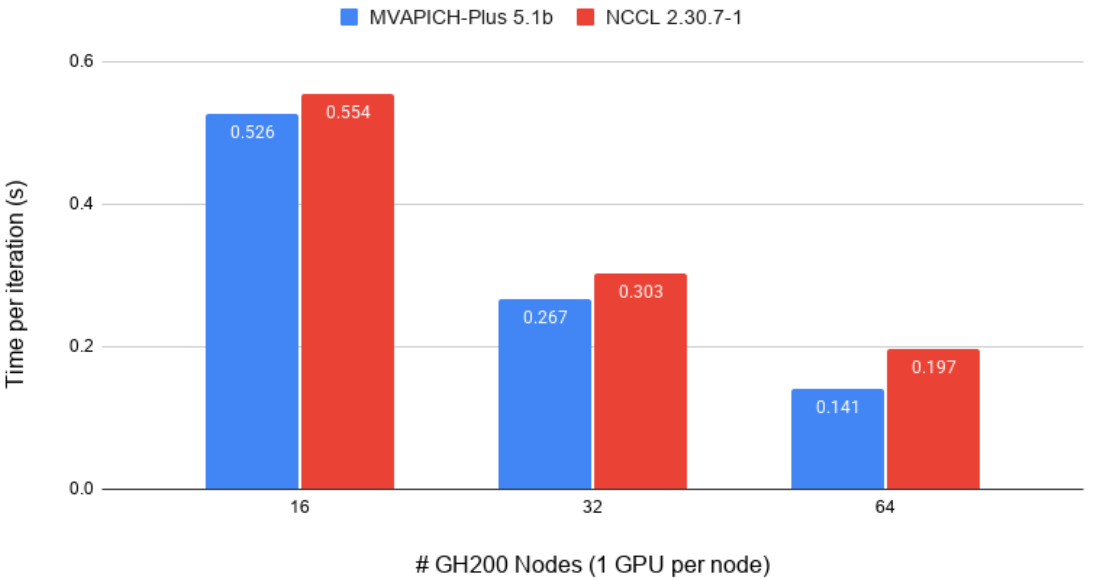
Model	Batch Size	Block Size	Benchmark	Dataset	DL Framework
GPT-2	12	1024	NanoGPT	OpenWebText	PyTorch 2.10.0

Distributed Data Parallel Training with BFLOAT16 on H200 (Vista)

MVAPICH-Plus 5.1b vs NCCL 2.30.7-1; Batch Size: 12; Block Size: 1024



MVAPICH-Plus 5.1b vs NCCL 2.30.7-1; Batch Size: 12; Block Size: 1024



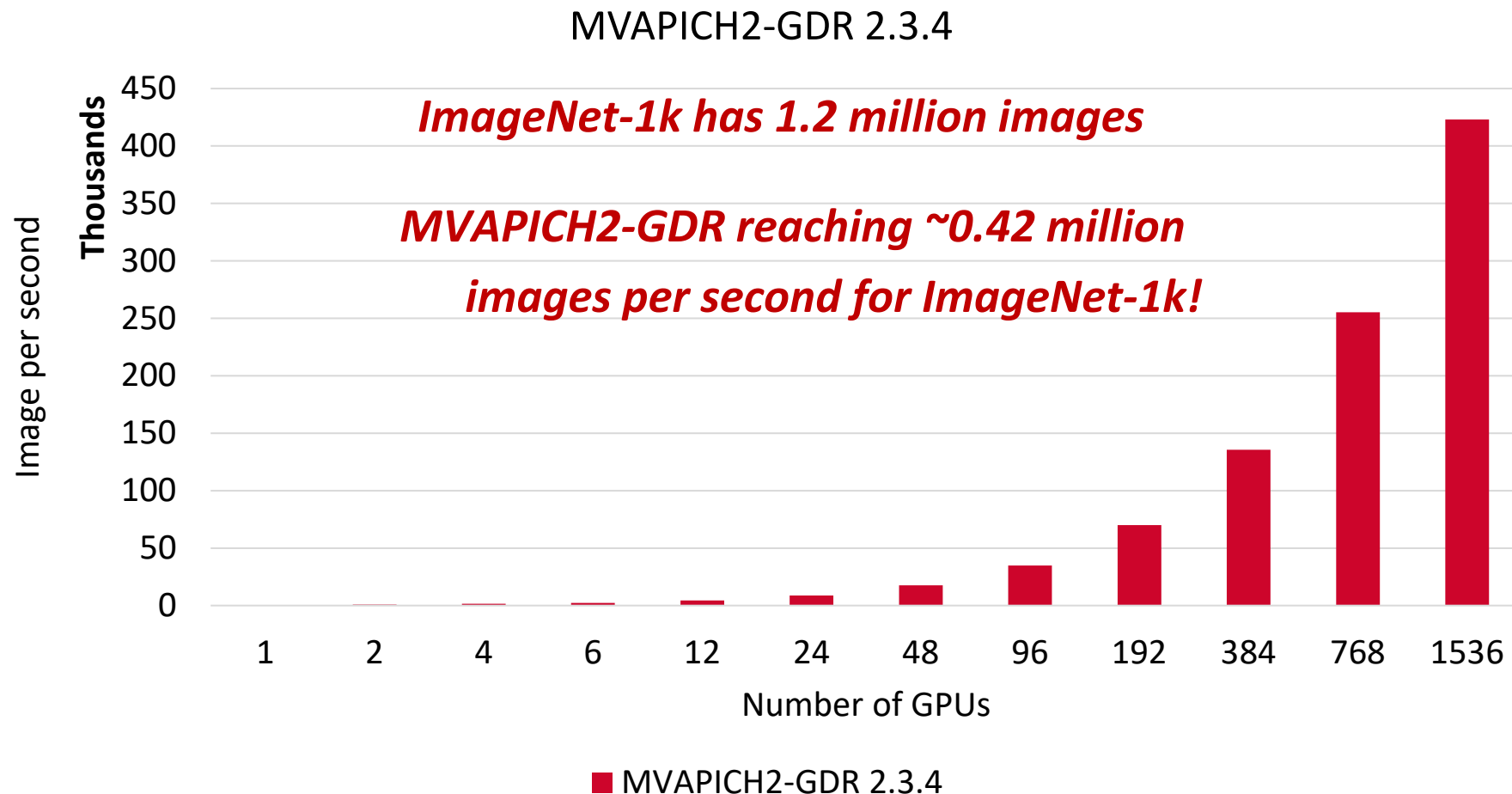
- **28.43% speedup**(compared to NCCL 2.30.7-1) on 64 GPUs with PyTorch 2.10.0

Machine Specifications: TACC Vista					
CPU Model	CPU Core Info	Memory	GPU Model	GPU Memory	Interconnects
NVIDIA Grace CPU	1x72@3.1 GHz	116 GB DDR5	NVIDIA H200 GPU (1/Node)	96 GB HBM 3	Mellanox NDR (400 Gb/s)

Model	Batch Size	Block Size	Benchmark	Dataset	DL Framework
GPT-2	12	1024	NanoGPT	OpenWebText	PyTorch 2.10.0

Distributed TensorFlow on ORNL Summit (1,536 GPUs)

- ResNet-50 Training using TensorFlow benchmark on SUMMIT -- 1536 Volta GPUs!
- 1,281,167 (1.2 mil.) images
- Time/epoch = 3 seconds
- Total Time (90 epochs) = $3 \times 90 = 270$ seconds = **4.5 minutes!**

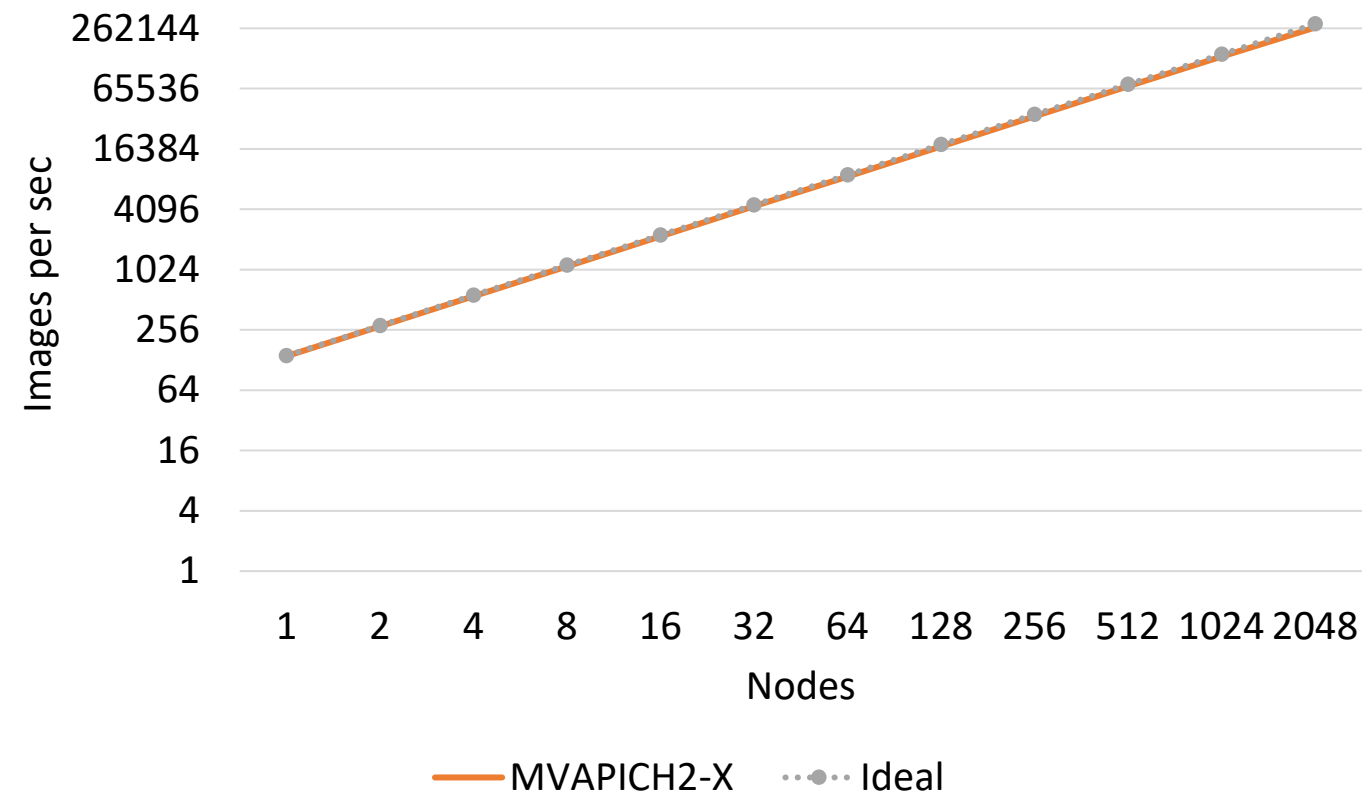


*We observed issues for NCCL2 beyond 384 GPUs

Platform: The Summit Supercomputer (#2 on Top500.org) – 6 NVIDIA Volta GPUs per node connected with NVLink, CUDA 10.1

Distributed TensorFlow on TACC Frontera (2048 CPU nodes)

- Scaled TensorFlow to 2048 nodes on Frontera using MVAPICH2 and IntelMPI
- MVAPICH2 delivers close to the ideal performance for DNN training
- Report a peak of **260,000 images/sec** on 2048 nodes
- On 2048 nodes, ResNet-50 can be trained in **7 minutes!**



A. Jain, A. A. Awan, H. Subramoni, DK Panda, "Scaling TensorFlow, PyTorch, and MXNet using MVAPICH2 for High-Performance Deep Learning on Frontera", DLS '19 (SC '19 Workshop).

Outline

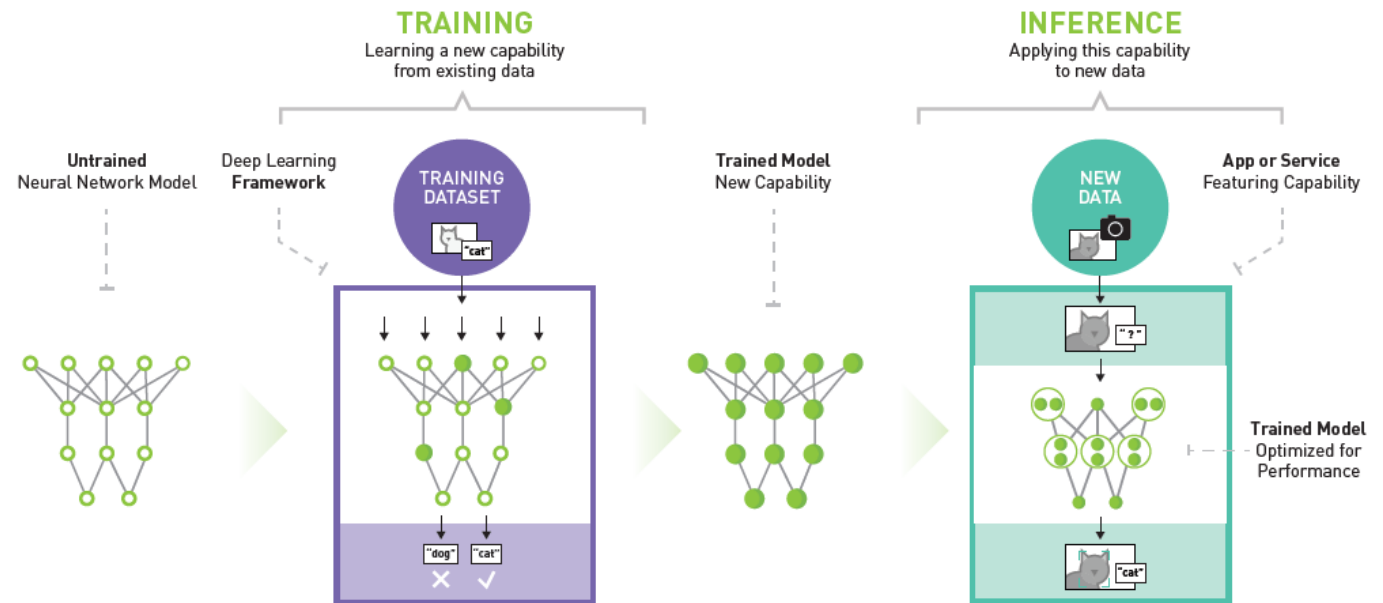
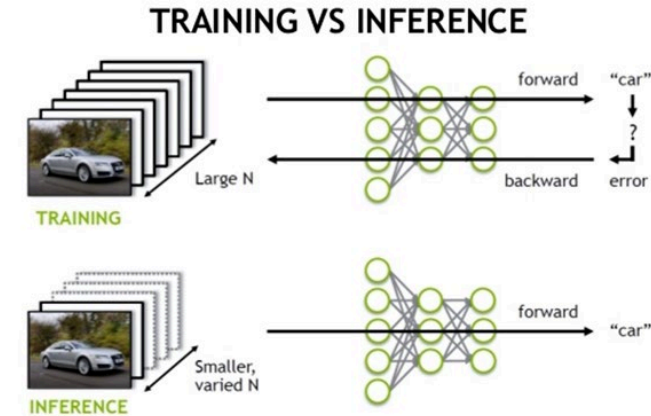
- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- **Distributed Inference Systems**
- Open Issues and Challenges
- Conclusion

What is Deep Learning Inference?

- Deep learning Training & Inference

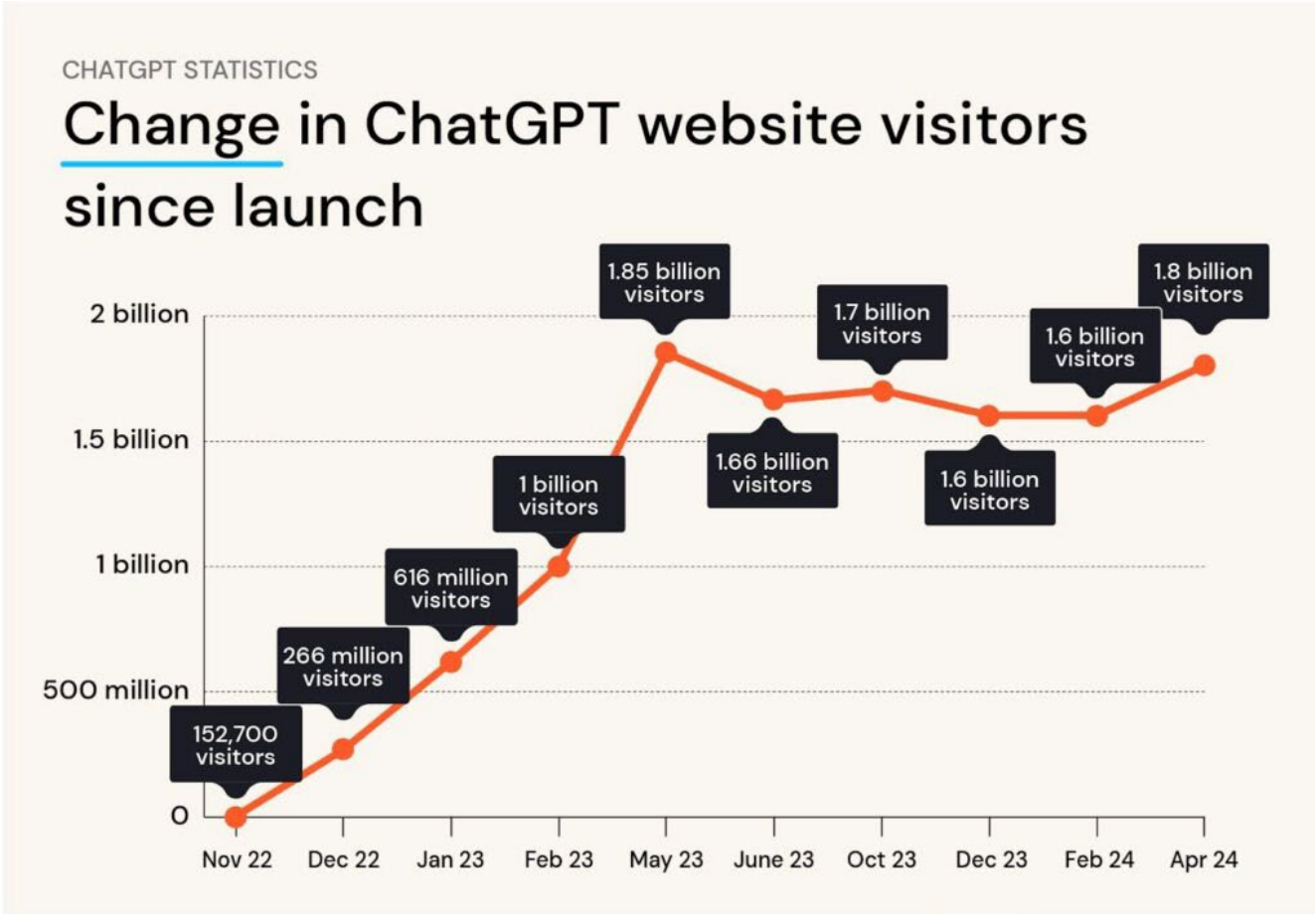
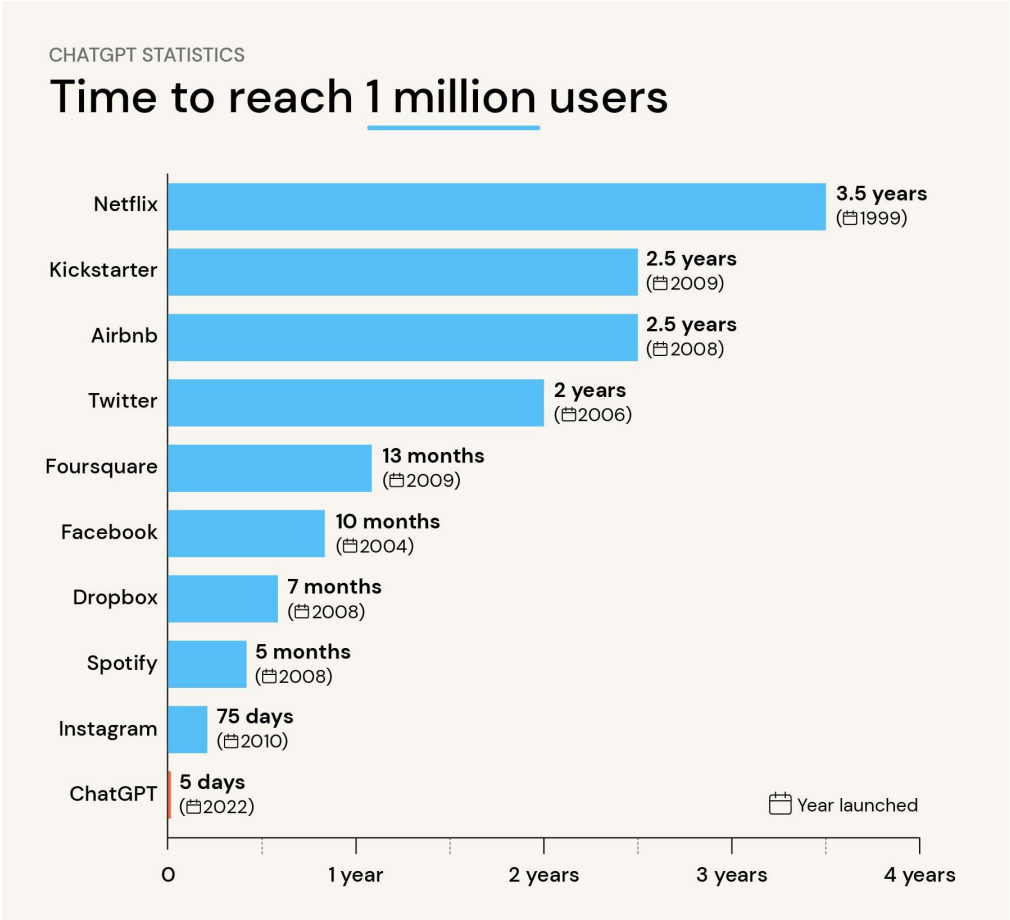
	Phase	Sensitivity
Training	Model-learning	Throughput
Inference	User-facing	Latency

- Inference: Latency-sensitive
 - Final Phase of Deep Learning
 - The closest end to users**
- Smaller batch size in the workflow
- User-end requests arrive randomly
- No need for model weights update
- Response time is the most crucial



Courtesy: <https://developer.nvidia.com/blog/nvidia-deep-learning-inference-platform-performance-study/>; <https://www.exxactcorp.com/blog/HPC/discover-the-difference-between-deep-learning-training-and-inference>

The mania of ChatGPT



Autoregressive property of GPT model

- For a sequence with N tokens, training only takes 1 run on the model.
- However, inference takes N runs on the model. Totally sequential!!!

Human: Hello.

AI: How

AI: How can

AI: How can I

AI: How can I help

AI: How can I help you

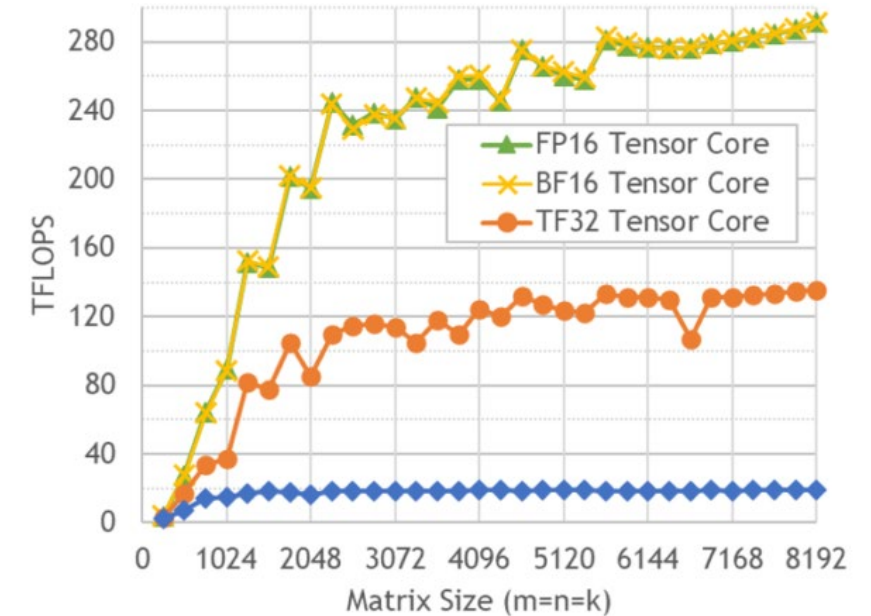
AI: How can I help you today?

```
while True:
    new_word = None
    query = new_word
    key = output_sentence
    value = output_sentence

    query = model.inference(query, key, value)

    if query != "$":
        output_sentence.append(query)
    else:
        print("Generation finishes.")
        break
```

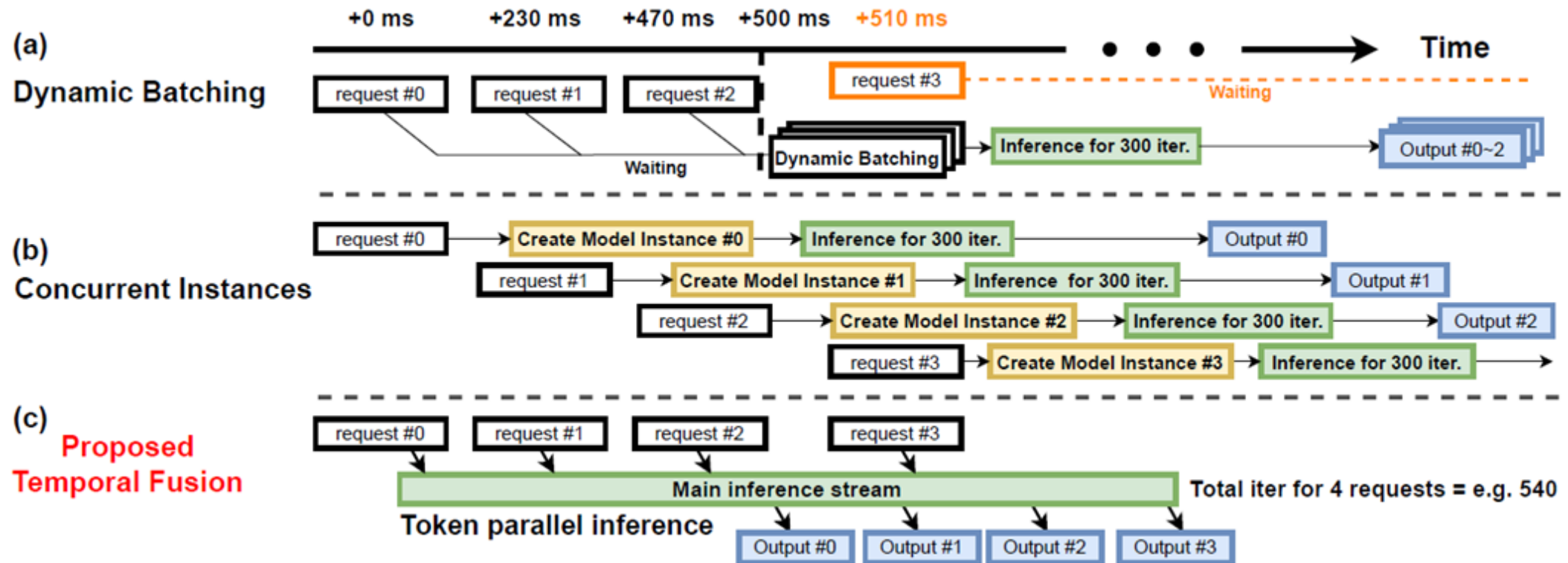
Mixed Precision Matrix Multiply on A100



- It is wasting most GPU compute power!

Flover: Efficient parallel inference on LLMs with In-flight batching^[1]

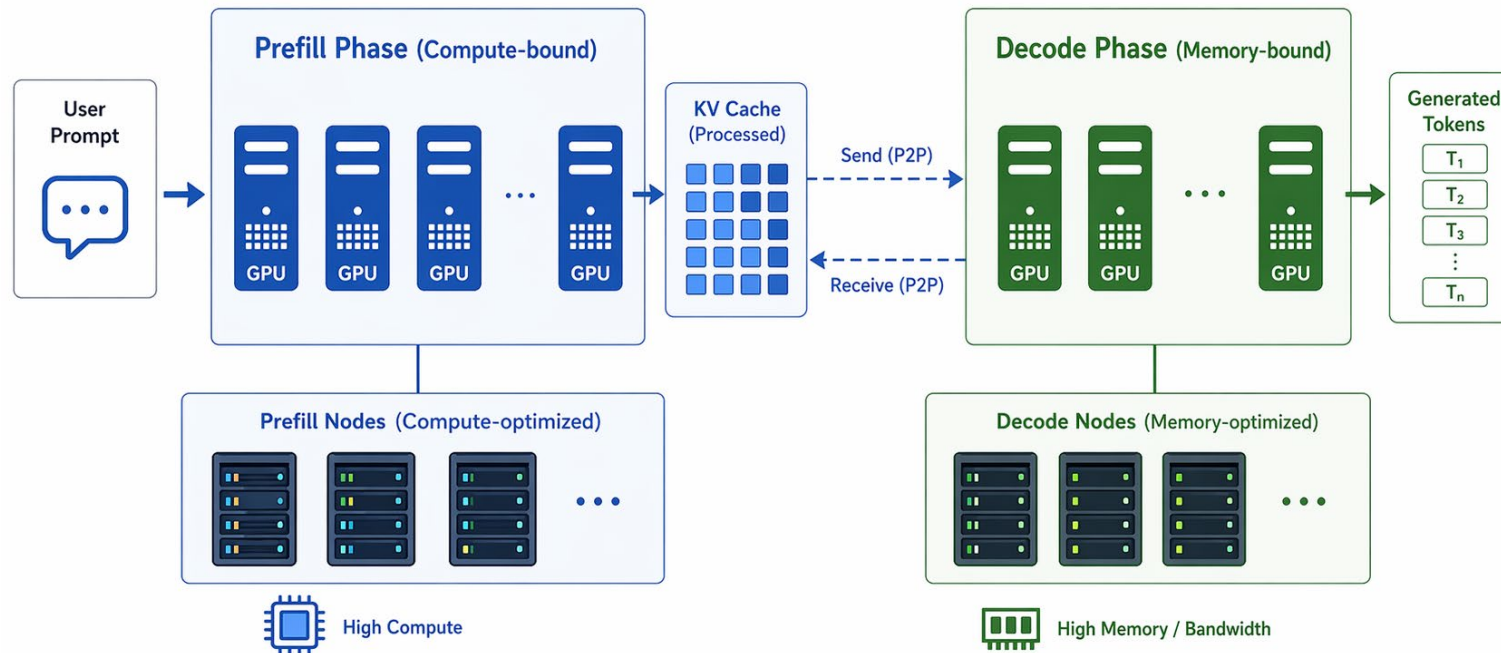
- When serving multiple requests, how to deliver both low-latency and high-throughput?
- For generative models such as GPT, LLaMA, the generation is sequential and regulated by 'for' loop.
 - For multiple requests that arrive at different time, how do we schedule the inference?



[1] Yao, Jinghan, Nawras Alnaasan, Tian Chen, Aamir Shafi, Hari Subramoni, D.K. Panda. "Flover: A Temporal Fusion Framework for Efficient Autoregressive Model Parallel Inference." *In Proceeding of HiPC 23*

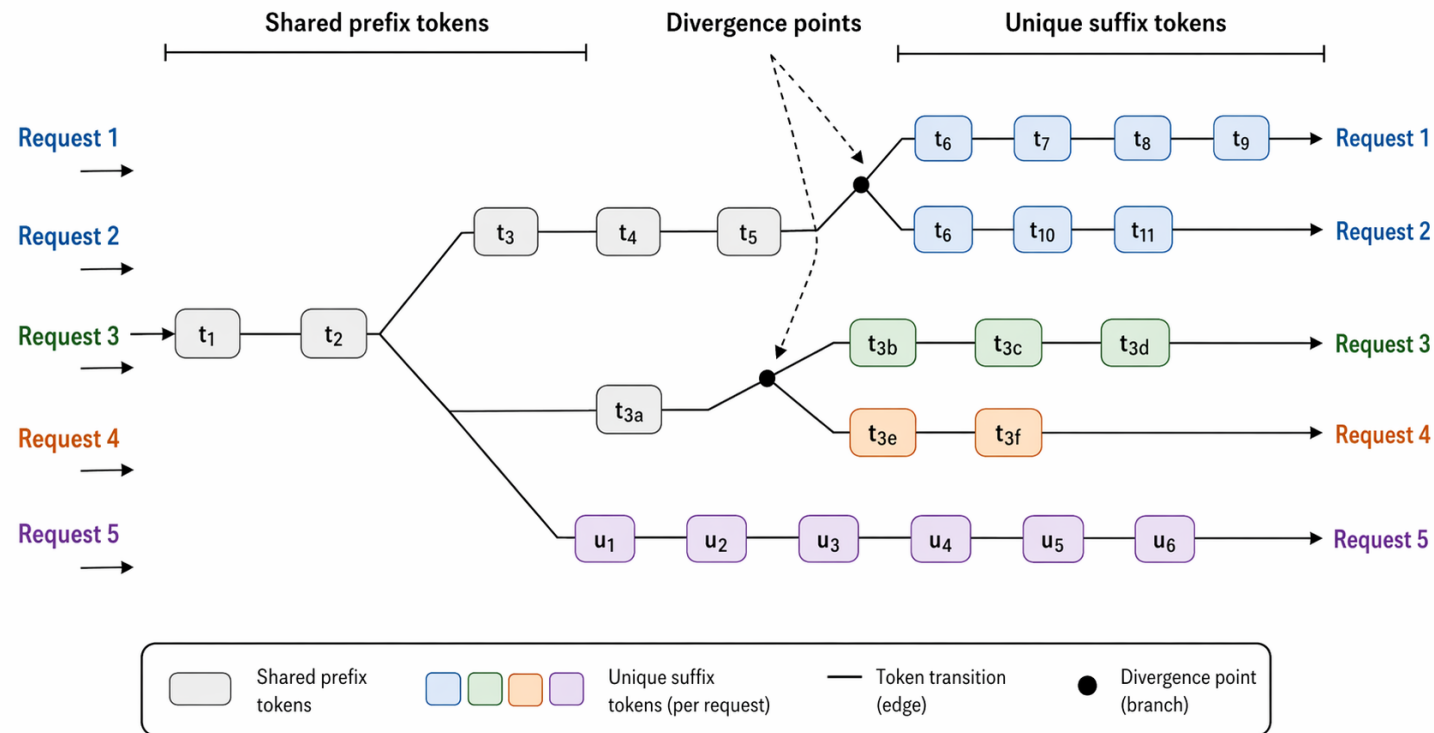
Multi-node Disaggregated Inference

- Prefilling is compute-bound, while decoding is memory bound. Deploying both phases on the same hardware is therefore not the first choice.
- Prefill-Decode disaggregation allows the split of prefill nodes and decode nodes.
- The processed KV cache is transferred from prefill nodes to decode nodes using point-to-point send and receive.



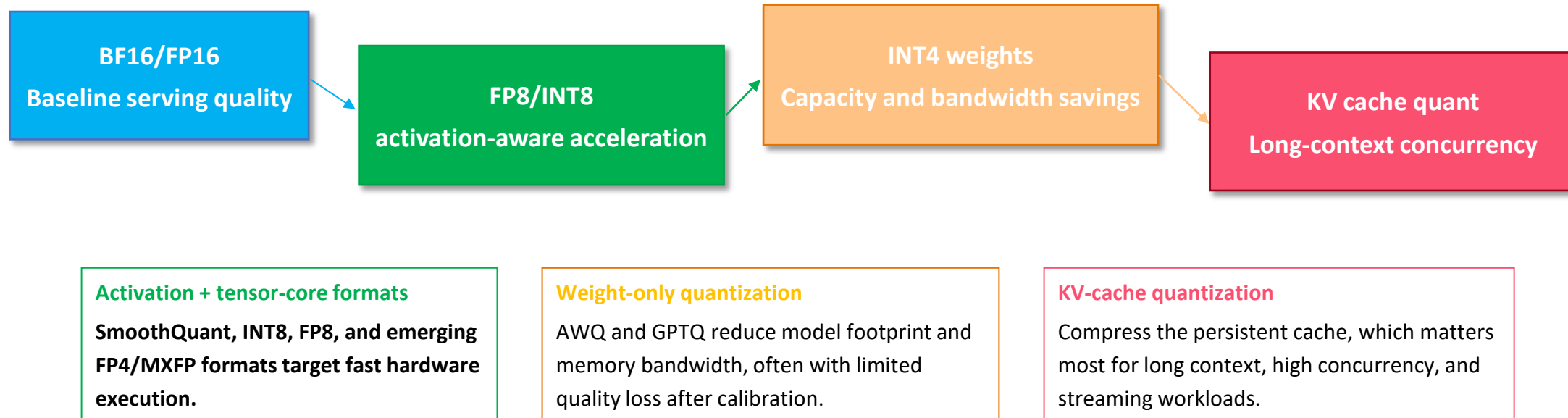
Memory Constraints: KV Cache

- Model weights mostly fixed; KV cache grows with active requests, context length, and output length.
- Instead of allocating a huge contiguous buffer for each request's KV cache, managing them as blocks (similar to page table) not only reduces fragmentation, enables larger dynamic batches, but also allows KV cache prefix sharing.



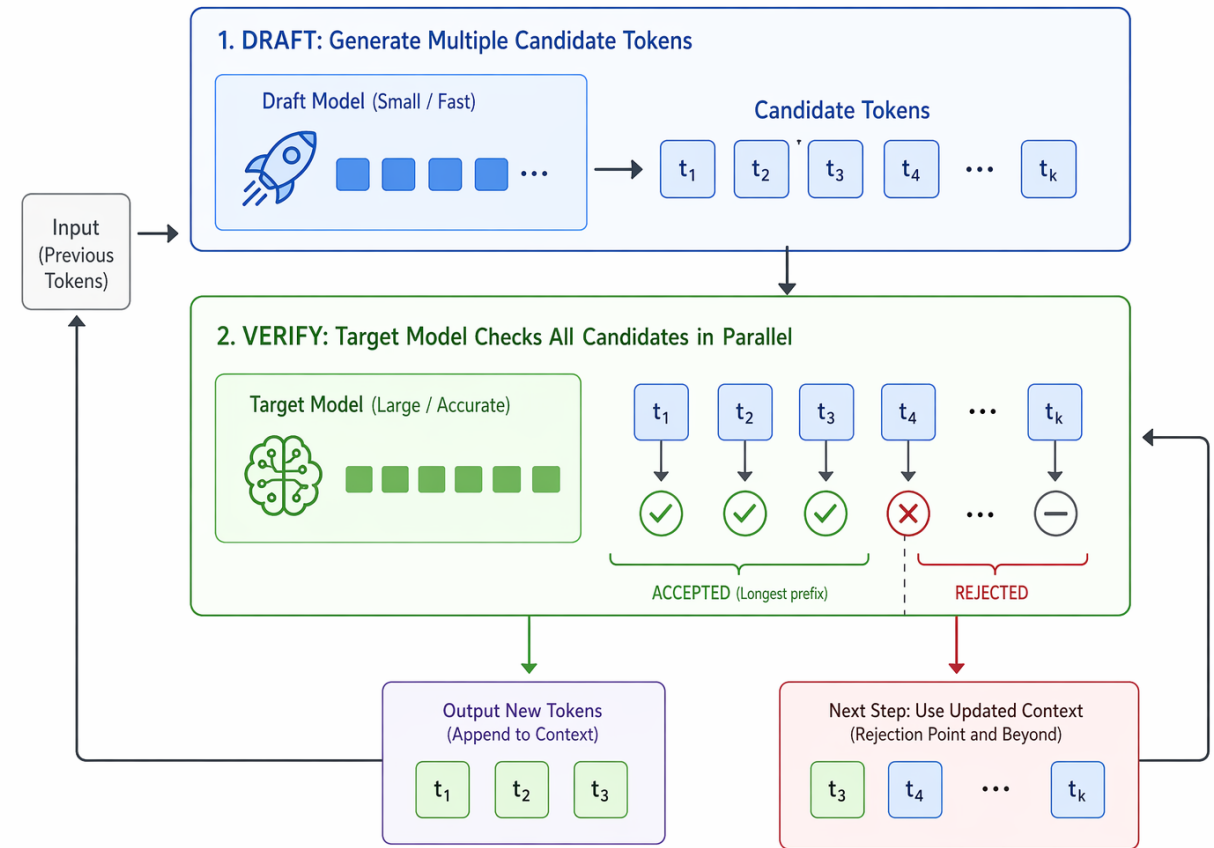
Quantization

- Quantization can reduce bytes, increase batch size.
- But it is also a systematic choice: quality, kernel support, calibration data, and target hardware all matter.



Speculative Decoding

- Speculative decoding parallelizes the serial token generation loop.
- Draft fast: A small model, auxiliary heads, or feature predictor proposes several candidate tokens.
- Verify in parallel: The target model checks multiple candidates in one pass, preserving output distribution for accepted variants.
- Skip serial work: Accepted tokens reduce the number of target-model iterations; rejected tokens fall back safely.



Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- **Open Issues and Challenges**
- Conclusion

Open Issues and Challenges

- Handling Trillion Parameter Models for Training and Inference
- Energy-aware and Fault-Tolerant DL training
- Low latency and high-throughput inference on a range of devices

Handling Trillion Parameter Models for Training and Inference

- The community has crossed models with Billion Parameters
- Already thinking about Models with Trillion Parameters
 - Trillion Parameter Consortium (<https://www.anl.gov/cels/trillion-parameter-consortium>)
- Model Training and Inference with Trillion Parameters will require
 - Extremely Large-scale datacenters (~1 million GPUs)
 - Accelerators and/or Memory subsystems to hold the model during training and inference
 - Next-generation of architectures (CPUs, GPUs, Interconnects) and algorithms for training and inference

Energy-aware and Fault-Tolerant DL Training

- Training Models with Billion Parameters requires
 - Extremely-large data centers with hundred thousands of GPUs
 - Months of training time
- Consumes significant energy
- GPUs go through failures
- Significant focus on
 - New generation of hardware and software for reducing energy consumption
 - Newer Checkpointing and fault-tolerant schemes

Low latency and high-throughput inference on a range of devices

- Wide range of needs for inference
 - Multiple disciplines (engineering, medicine, agriculture, ...)
 - Range of edge devices (laptops, smart phones, drones, dedicated devices)
- Require inference schemes with
 - Low-latency
 - High-throughput
 - Reduced cost
- The inference workflow pipeline involving edge devices, network, and back-end servers need to be heavily optimized based on the needs

Outline

- Introduction: The AI–HPC Convergence
 - Definitions: AI, Machine Learning, Deep Learning, and Generative AI
 - Overview: System-Aware AI Lifecycle and HPC Stack
- Challenges in Exploiting HPC Technologies for AI
- Modern HPC Architectures for AI
- Foundations of Deep Neural Networks
- Distributed Data Parallel Deep Learning
 - Basic Principles and Parallelism Strategies
 - Lab 1: Hands-on Exercises (Distributed Data Parallelism)
- Advanced Parallelism Strategies
 - Hybrid Parallelism Techniques
 - Case studies and solutions
 - Lab 2: Hands-on Exercises (Hybrid Parallelism)
- Distributed Inference Systems
- Open Issues and Challenges
- **Conclusion**

Conclusion

- Exponential growth in AI (training, inference, and generative AI)
- Presented an overview of the Life Cycle of the AI Models and various stages.
- Provided an overview of issues, challenges, and opportunities for designing efficient communication and accelerator runtimes
 - Efficient, scalable, and hierarchical designs are crucial for HPC-AI frameworks
 - Co-design of communication runtimes and AI frameworks will be essential
- Worked on a set of hands-on exercises to demonstrate the complex interaction between AI middleware with the underling HPC technologies and middleware
- Need collaborative efforts to achieve the full potential

Funding Acknowledgments

Funding Support by



Equipment Support by



Acknowledgments to all the Heroes (Past/Current Students and Staffs)

Current Students (Under/Graduate)

- | | | | |
|-----------------------|--------------------------|-----------------------|--------------------|
| – N. Alnaasan (Ph.D.) | – J. Hatef (Ph.D.) | – S. Mohammad (Ph.D.) | – S. Xu (Ph.D.) |
| – Q. Anthony (Ph.D.) | – G. Kuncham (Ph.D.) | – T. Tran (Ph.D.) | – J. Yao (Ph.D.) |
| – T. Chen (Ph.D.) | – S. Lee (Ph.D.) | – J. Wan (Ph.D.) | – S. Zhang (Ph.D.) |
| – S. Gumaste (Ph.D.) | – B. Michalowicz (Ph.D.) | – L. Xu (Ph.D.) | – N. Klein (M.S.) |

Current Research Specialist

- R. Motlagh

Current Software Engineers

- N. Shineman
- M. Lieber

Past Research Scientists

- K. Hamidouche
- S. Sur
- X. Lu
- M. Abduljabbar
- A. Shafi

Past Students

- | | | | | |
|-----------------------------|----------------------------|----------------------------|---------------------------------|---------------------------|
| – A. Awan (Ph.D.) | – C.-H. Chu (Ph.D.) | – P. Kousha (Ph.D.) | – S. Pai (M.S.) | – S. Sur (Ph.D.) |
| – A. Augustine (M.S.) | – T. Gangadharappa (M.S.) | – K. Kulkarni (M.S.) | – S. Potluri (Ph.D.) | – K. K. Suresh (Ph.D.) |
| – P. Balaji (Ph.D.) | – K. Gopalakrishnan (M.S.) | – R. Kumar (M.S.) | – J. Queiser (M.S.) | – K. Vaidyanathan (Ph.D.) |
| – M. Bayatpour (Ph.D.) | – R. Gulhane (M.S.) | – S. Krishnamoorthy (M.S.) | – K. Raj (M.S.) | – A. Vishnu (Ph.D.) |
| – R. Biswas (M.S.) | – J. Hashmi (Ph.D.) | – K. Kandalla (Ph.D.) | – R. Rajachandrasekar (Ph.D.) | – J. Wu (Ph.D.) |
| – S. Bhagvat (M.S.) | – M. Han (M.S.) | – M. Li (Ph.D.) | – B. Ramesh (Ph.D.) | – W. Yu (Ph.D.) |
| – A. Bhat (M.S.) | – W. Huang (Ph.D.) | – P. Lai (M.S.) | – D. Shankar (Ph.D.) | – J. Zhang (Ph.D.) |
| – D. Buntinas (Ph.D.) | – A. Jain (Ph.D.) | – J. Liu (Ph.D.) | – G. Santhanaraman (Ph.D.) | – Q. Zhou (Ph.D.) |
| – L. Chai (Ph.D.) | – J. Jani (M.S.) | – M. Luo (Ph.D.) | – N. Sarkauskas (B.S. and M.S.) | – N. Chmura (B.S.) |
| – B. Chandrasekharan (M.S.) | – W. Jiang (M.S.) | – A. Mamidala (Ph.D.) | – V. Sathu (M.S.) | – S. Tilford (M.S.) |
| – S. Chakraborty (Ph.D.) | – J. Jose (Ph.D.) | – G. Marsh (M.S.) | – N. Senthil Kumar (M.S.) | |
| – C.-C. Chen (Ph.D.) | – M. Kedia (M.S.) | – V. Meshram (M.S.) | – A. Singh (Ph.D.) | |
| – N. Contini (Ph.D.) | – K. S. Khorassani (Ph.D.) | – A. Moody (M.S.) | – J. Sridhar (M.S.) | |
| – N. Dandapanthula (M.S.) | – S. Kini (M.S.) | – S. Naravula (Ph.D.) | – S. Srivastava (M.S.) | |
| – V. Dhanraj (M.S.) | – M. Koop (Ph.D.) | – R. Noronha (Ph.D.) | – H. Subramoni (Ph.D.) | |
| | | – X. Ouyang (Ph.D.) | | |

Past Faculty

- H. Subramoni

Past Senior Research Associate

- J. Hashmi

Past Programmers

- A. Reifsteck
- D. Bureddy
- J. Perkins
- B. Seeds
- A. Gupta
- N. Pavuk

Past Research Specialist

- M. Arnold
- J. Smith

Past Post-Docs

- | | | | |
|-----------------------|-------------|-----------------|-------------|
| – D. Banerjee | – H.-W. Jin | – E. Mancini | – A. Ruhela |
| – X. Besson | – J. Lin | – K. Manian | – J. Vienne |
| – M. S. Ghazimirsaeed | – M. Luo | – S. Marcarelli | – H. Wang |

Interested in learning more?

1. Join our half-day tutorials at IEEE Hot Interconnects 2026!

- **Tutorial: Principles and Practice of Scalable and Distributed AI Training and Inference**
 - Friday, August 21, 13:30 - 16:30 Pacific Time (PT)
- **Tutorial: High Performance and Smart Networking Technologies for HPC and AI**
 - Friday, August 21, 09:00 - 12:00 Pacific Time (PT)

2. Join our full-day tutorials at Supercomputing (SC) 2026!

- **Tutorial: Principles and Practice of Scalable and Distributed AI Training and Inference**
 - Nov 15-20 (Exact date/time TBD)
- **Tutorial: High Performance and Smart Networking Technologies for HPC and AI**
 - Nov 15-20 (Exact date/time TBD)

3. CARLA Conference in Córdoba, Argentina!

- **Tutorial: Principles and Practice of Scalable and Distributed AI Training and Inference**
 - September 21, 2026, 13:30 - 17:00 Eastern Time (ET) – 14:30 – 18:00 Córdoba Time
- **Tutorial: High Performance and Smart Networking Technologies for HPC and AI**
 - Monday, September 21, 2026, 8:00 – 11:00 Eastern Time (ET) – 9:00 – 12:00 Córdoba Time

14th Annual MVAPICH User Group (MUG) Conference

<http://mug.mvapich.cse.ohio-state.edu/>

- August 17-19, 2025, Columbus, Ohio, USA

- **Keynote Speakers:**

- Yutong Lu, Sun Yat-Sen University
- Dan Stanzione, Texas Advanced Computing Center (TACC)

- **Invited Speakers:**

- Ahmad Abdelfattah and Natalie Beams, University of Tennessee, Knoxville
- Sameh Abdulah, King Abdullah University of Science and Technology (KAUST), Saudi Arabia
- HooYoung Ahn, ETRI, South Korea
- Jeff Hammond, NVIDIA
- Jithin Jose, Microsoft
- Niranjana Vaidya and Hemal Shah, Broadcom
- Solyman Ashrafi, Fujitsu/1Finity
- Soham Ghosh and Kyle Schaefer, X-ScaleSolutions
- Charles Archer and Brian Hwang, Cornelis Networks
- Ankur Sheth, Dan Mihailescu, and Eric Yu, Keysight; Mihai-Gabriel Constantin, UPB Romania

- **Invited Speakers (Cont'd):**

- Vipin Chaudhary, Case Western Reserve University
- Richard Graham, NVIDIA
- Rakesh Kumar Yadav, CDAC, India
- Sameer Shende, ParaTools and University of Oregon
- Mahidhar Tatineni, San Diego Supercomputer Center (SDSC)
- Xiaoyi Lu, University of Florida
- Sangamesh Kodge, Sai Rahul Chalamalasetti, and David Karlov, D-Matrix
- Hari Subramoni and Brijesh Nanda, OSU

- **Tutorials and Demos:**

- 1Finity
- Keysight
- D-matrix
- OSU MVAPICH
- OSU HiDL
- ParaTools
- X-ScaleSolutions

**Slides and Videos of the
Talks are being uploaded!**

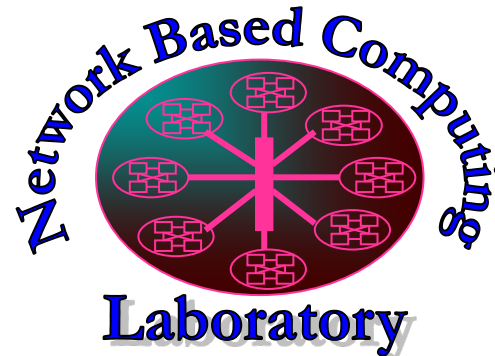
Thank You!

panda@cse.ohio-state.edu

alnaasan.1@osu.edu



<https://twitter.com/mvapich>



Network-Based Computing Laboratory

<http://nowlab.cse.ohio-state.edu/>



MVAPICH

MPI, PGAS and Hybrid MPI+PGAS Library

The MVAPICH2 Project

<http://mvapich.cse.ohio-state.edu/>



The High-Performance Deep Learning Project

<http://hidl.cse.ohio-state.edu/>